

ΣΑΚΕΛΛΑΡΟΠΟΥΛΟΣ ΣΠΥΡΙΔΩΝ

PHYSICS INFORMED NEURAL
NETWORKS (PINNs) ΚΑΙ ΟΙ
ΕΦΑΡΜΟΓΕΣ ΤΟΥΣ ΣΤΗΝ ΕΠΙΛΥΣΗ
ΚΥΜΑΤΙΚΩΝ ΜΕΡΙΚΩΝ
ΔΙΑΦΟΡΙΚΩΝ ΕΞΙΣΩΣΕΩΝ

ΜΕΤΑΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΑΙΓΑΙΟΥ

ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ
ΤΜΗΜΑ ΜΑΘΗΜΑΤΙΚΩΝ

Είμαι συγγραφέας αυτής της εργασίας και κάθε βοήθεια την οποία είχα για την προετοιμασία της είναι πλήρως αναγνωρισμένη και αναφέρεται στην εργασία. Επίσης, έχω αναφέρει τις όποιες πηγές από τις οποίες έκανα χρήση δεδομένων ή ιδεών, είτε αυτές αναφέρονται ακριβώς είτε παραφρασμένες. Επίσης, βεβαιώνω ότι αυτή η εργασία προετοιμάστηκε από εμένα προσωπικά, ειδικά για τη συγκεκριμένη Μεταπτυχιακή Εργασία.

ΕΙΣΗΓΗΤΗΣ: Βασίλειος Κουκουλογιάννης

ΕΠΙΤΡΟΠΗ

Βασίλειος Κουκουλογιάννης

Χρήστος Νικολόπουλος

Παναγιώτης Νάστου

End of line, man.
- *CLU*

Περιεχόμενα

Περίληψη στα ελληνικά xi

Περίληψη στα αγγλικά xiii

Εισαγωγή xv

1	Ιστορική Αναδρομή της Τεχνητής Νοημοσύνης	1
1.1	Από την Αρχαιότητα...	1
1.2	...στην Αναγέννηση και τον Διαφωτισμό...	2
1.3	...στον Alan Turing...	3
1.4	...στο μεσοδιάστημα...	3
1.5	...στο Σήμερα...	4
1.6	...στο Τώρα	5
2	Τεχνητή Νοημοσύνη - Νευρωνικά Δίκτυα - PINNs	7
2.1	Τεχνητή Νοημοσύνη	7
2.2	Νευρωνικά Δίκτυα	8
2.2α'	Αρχιτεκτονική ενός Τεχνητού Νευρώνα	8
2.2β'	Περιορισμοί του ενός Τεχνητού Νευρώνα	10
2.2γ'	Περισσότεροι του ενός Τεχνητού Νευρώνα	11
2.2δ'	Εμπρόσθια Προώθηση (Forward Propagation)	13
2.2ε'	Συνάρτηση Απώλειας (Loss Function)	15
2.2ϛ'	Οπισθοδιάδοση Σφάλματος (Back Propagation)	15
2.2ζ'	Πολυεπίπεδο Perceptron (Multi-Layer Perceptron - MLP)	17
2.2η'	Γεωμετρία των FNN	18
3	Νευρωνικά Δίκτυα Εμπλουτισμένα με Φυσικούς Νόμους (Physics-Informed Neural Networks - PINNs)	19
3.0α'	Ορισμός Προβλήματος που Προσεγγίζουν τα PINNs	20
3.0β'	Βασικές Διαφορές μεταξύ FNN και PINNs	20

3.0γ'	Συνάρτηση Απώλειας των PINNS	21
3.0δ'	Κατηγορίες και Μοντελοποίηση Προβλημάτων	22
3.0ε'	Συνεχή Χρονικά Μοντέλα (Continuous-Time Models)	23
3.0ζ'	Αυτόματη Παραγωγή (Automatic Differentiation)	24
3.0ζ'	Συναρτήσεις Βελτιστοποίησης στα PINNs (Adam και L-BFGS)	26
3.0η'	Εκπαίδευση των PINNs	26
3.0θ'	Αρχιτεκτονική των PINNS	29
3.0ι'	Σύγχρονες Βελτιώσεις των PINNs	29

4 Υπολογιστικό Περιβάλλον Εκτέλεσης των PINNs 31

5 Εξίσωση Κύματος 33

5.1	Περιοδικές Συνοριακές Συνθήκες	33
5.1α'	Εκπαίδευση σε CPU	34
5.1β'	Εκπαίδευση σε CUDA	39
5.2	Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες	40
5.2α'	Εκπαίδευση σε CPU	40
5.2β'	Εκπαίδευση σε CUDA	43
5.3	Συνοριακές Συνθήκες Dirichlet	45
5.3α'	Εκπαίδευση σε CPU	45
5.3β'	Εκπαίδευση σε CUDA	49
5.4	Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Συνοριακές Συνθήκες Dirichlet	51
5.4α'	Εκπαίδευση σε CPU	51
5.4β'	Εκπαίδευση σε CUDA	54
5.5	Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet	55
5.5α'	Εκπαίδευση σε CPU	55
5.5β'	Εκπαίδευση σε CUDA	58
5.6	Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet	60
5.6α'	Εκπαίδευση σε CPU	60
5.6β'	Εκπαίδευση σε CUDA	63
5.7	Συνοριακές Συνθήκες Neumann	64
5.7α'	Εκπαίδευση σε CPU	64
5.7β'	Εκπαίδευση σε CUDA	68
5.8	Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Συνοριακές Συνθήκες Neumann	70
5.8α'	Εκπαίδευση σε CPU	71
5.8β'	Εκπαίδευση σε CUDA	73
5.9	Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Neumann	74
5.9α'	Εκπαίδευση σε CPU	75
5.9β'	Εκπαίδευση σε CUDA	78
5.10	Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet	80
5.10α'	Εκπαίδευση σε CPU	81

5.10β'	Εκπαίδευση σε CUDA	83
6	Εξίσωση Korteweg - de Vries (KdV)	85
6.1	Περιοδικές Συνοριακές Συνθήκες	85
6.1α'	Εκπαίδευση σε CPU	86
6.1β'	Εκπαίδευση σε CUDA	90
6.2	Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες	92
6.2α'	Εκπαίδευση σε CPU	93
6.2β'	Εκπαίδευση σε CUDA	95
7	Εξίσωση Non-Linear Schrödinger (NLS)	97
7.1	Περιοδικές Συνοριακές Συνθήκες	97
7.1α'	Εκπαίδευση σε CPU	98
7.1β'	Εκπαίδευση σε CUDA	102
7.2	Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες	104
7.2α'	Εκπαίδευση σε CPU	105
7.2β'	Εκπαίδευση σε CUDA	107
8	Συνοπτικά Συμπεράσματα	109
	Παράρτημα Α: Κώδικας PINN για την Εξίσωση Κύματος	111
	Βιβλιογραφία	131

Περίληψη στα ελληνικά

Η παρούσα διπλωματική εργασία εξετάζει τη μεθοδολογία των Physics-Informed Neural Networks (PINNs) για την επίλυση Μερικών Διαφορικών Εξισώσεων και την εφαρμογή τους σε κλασικά προβλήματα φυσικής. Τα PINNs αποτελούν μια σύγχρονη προσέγγιση που συνδυάζει τεχνικές βαθιάς μάθησης ενσωματώνοντας τις διαφορικές εξισώσεις και τις αρχικές και συνοριακές συνθήκες στη συνάρτηση απώλειας (Loss Function) του νευρωνικού δικτύου.

Αρχικά παρουσιάζεται η ιστορική εξέλιξη της Τεχνητής Νοημοσύνης και των νευρωνικών δικτύων καθώς και οι βασικές αρχές αρχιτεκτονικής και εκπαίδευσης πολυεπίπεδων νευρωνικών δικτύων (MLP). Γίνεται αναφορά του Θεωρήματος Καθολικής Σύγκλισης. Στη συνέχεια παρουσιάζονται τα PINNs, αναλύοντας τη δομή και την αρχιτεκτονική τους, το πώς διαμορφώνεται η συνάρτηση απώλειας, τον ρόλο της αυτόματης παραγώγισης και τη διαδικασία εκπαίδευσης που ακολουθεί το δίκτυο.

Η αποτελεσματικότητα της μεθόδου αξιολογείται μέσω εφαρμογών σε κυματικές εξισώσεις. Σε γραμμικές εξισώσεις, όπως η εξίσωση Κύματος, και μη γραμμικές όπως η εξίσωση Korteweg-de Vries (KdV) και η μη γραμμική εξίσωση Schrödinger, για διαφορετικές αρχικές και συνοριακές συνθήκες, συμπεριλαμβανομένων και τμηματικά ορισμένων δεδομένων. Παρουσιάζεται συγκριτική ανάλυση της απόδοσης σε περιβάλλον CPU και GPU μέσω CUDA, εξετάζοντας το υπολογιστικό κόστος της εκπαίδευσης και τη συμπεριφορά σύγκλισης. Επιπλέον, οι προσεγγιστικές λύσεις του PINN, για κάθε πρόβλημα, συγκρίνονται με τις αντίστοιχες αναλυτικές λύσεις και με αριθμητικές λύσεις που προκύπτουν από τη μέθοδο πεπερασμένων διαφορών με χρήση Runge Kutta 4ης τάξης, όπου αυτό είναι δυνατόν.

Η μελέτη των PINNs παρουσιάζει ιδιαίτερο ενδιαφέρον, καθώς προσφέρουν μια εναλλακτική προσέγγιση στην αριθμητική επίλυση διαφορικών εξισώσεων χωρίς την ανάγκη ρητής διακριτοποίησης πλέγματος. Στόχος της εργασίας είναι η αξιολόγηση της ικανότητας των PINNs να προσεγγίζουν λύσεις κυματικών εξισώσεων.

ΛΕΞΕΙΣ ΚΛΕΙΔΙΑ: PINNs, Μερικές Διαφορικές Εξισώσεις, Κυματικές Εξισώσεις, Νευρωνικά Δίκτυα, CUDA, KdV, NLS

Περίληψη στα αγγλικά

This thesis investigates the methodology of Physics-Informed Neural Networks (PINNs) for solving Partial Differential Equations (PDEs) and their application to classical problems in physics. PINNs represent a modern approach that combines deep learning techniques with physical laws by embedding differential equations as well as initial and boundary conditions directly into the network's loss function.

The work begins with a historical overview of Artificial Intelligence and neural networks, including the fundamental principles of architecture and training of Multi-Layer Perceptrons (MLPs). Reference is made to the Universal Approximation Theorem. Subsequently, PINNs are presented in detail, analyzing their structure and architecture, the formulation of the loss function, the role of automatic differentiation, and the training procedure of the network.

The effectiveness of the method is evaluated through applications to wave equations. Linear problems, such as the Wave Equation, as well as nonlinear ones, including the Korteweg-de Vries (KdV) and the Nonlinear Schrödinger equations, are considered under various initial and boundary conditions, including piecewise-defined data. A comparative analysis of performance on CPU and GPU (via CUDA) is provided, assessing computational cost and convergence behavior. Additionally, the PINN approximations are compared with corresponding analytical solutions and numerical solutions obtained through the finite difference method with a 4th-order Runge-Kutta scheme, where applicable.

The study of PINNs is of particular interest as they offer an alternative approach for the numerical solution of differential equations without requiring explicit mesh discretization. The goal of this thesis is to evaluate the capability of PINNs to accurately approximate solutions to wave equations.

KEYWORDS: PINNs, Partial Differential Equations, Wave Equations, Neural Networks, CUDA, KdV, NLS

Εισαγωγή

Οι μερικές διαφορικές εξισώσεις (Partial Differential Equations – PDEs) αποτελούν θεμελιώδες εργαλείο περιγραφής φυσικών φαινομένων στη μηχανική, στη ρευστοδυναμική, στη θεωρία κυμάτων, στη χβαντική μηχανική και σε πλήθος άλλων επιστημονικών πεδίων. Στις περισσότερες περιπτώσεις, αναλυτικές λύσεις δεν είναι διαθέσιμες, γεγονός που καθιστά απαραίτητη την ανάπτυξη αριθμητικών μεθόδων προσέγγισης.

Κλασικές προσεγγίσεις, όπως οι μέθοδοι πεπερασμένων διαφορών, πεπερασμένων στοιχείων και πεπερασμένων όγκων, βασίζονται στη διακριτοποίηση του χωρικού ή χρονικού πεδίου και στην επίλυση μεγάλων αλγεβρικών συστημάτων. Παρότι ιδιαίτερα αποτελεσματικές, παρουσιάζουν περιορισμούς σε προβλήματα υψηλής διάστασης, σύνθετης γεωμετρίας ή όταν απαιτείται προσαρμοστικότητα σε δεδομένα.

Η εξέλιξη της Τεχνητής Νοημοσύνης και ειδικότερα των τεχνητών νευρωνικών δικτύων οδήγησε σε νέες μεθόδους προσέγγισης συναρτήσεων υψηλής πολυπλοκότητας. Τα πολυεπίπεδα νευρωνικά δίκτυα (MLPs) διαθέτουν καθολική ικανότητα προσέγγισης συναρτήσεων, γεγονός που τα καθιστά κατάλληλα για την προσεγγιστική επίλυση διαφορικών εξισώσεων.

Η εισαγωγή των Physics-Informed Neural Networks αποτέλεσε ένα σημαντικό βήμα προς την ενοποίηση της μηχανικής μάθησης με τη μαθηματική φυσική. Στα PINNs, η διαφορική εξίσωση, οι αρχικές και οι συνοριακές συνθήκες ενσωματώνονται απευθείας στη συνάρτηση απώλειας μέσω αυτόματης παραγώγισης, επιτρέποντας την εκπαίδευση του δικτύου χωρίς την ανάγκη ρητής διακριτοποίησης πλέγματος.

Στόχος της παρούσας εργασίας είναι η θεωρητική μελέτη και η υπολογιστική διερεύνηση της μεθόδου των PINNs στην επίλυση κυματικών μερικών διαφορικών εξισώσεων. Η μελέτη εφαρμόζεται σε εξισώσεις μαθηματικής φυσικής, όπως η εξίσωση κύματος, η Korteweg–de Vries, η Non-Linear Schrödinger, η Burgers και η Camassa–Holm, επιτρέποντας την αξιολόγηση των PINNs σε γραμμικά και μη γραμμικά φαινόμενα.

Στο σημείο αυτό θα ήθελα να ευχαριστήσω τον επιβλέποντα καθηγητή μου, Βασίλειο Κουκουλογιάννη, για την υποστήριξή του καθ' όλη τη διάρκεια της εργασίας καθώς και τα τεράστια αποθέματα υπομονής που διαθέτει. Ιδιαίτερα ευχαριστώ για την ευκαιρία να δουλέψω πάνω σε ένα τόσο σύγχρονο και ενδιαφέρον θέμα.

Κεφάλαιο 1

Ιστορική Αναδρομή της Τεχνητής Νοημοσύνης

1.1 Από την Αρχαιότητα...

Ο άνθρωπος είναι ένα ον γεμάτο περιέργεια και βαθιά ανάγκη για την κατανόηση του κόσμου γύρω του. Προσπαθώντας να ερμηνεύσει τα φαινόμενα που εκτυλίσσονται γύρω του άρχισε σταδιακά να διατυπώνει τους νόμους της φύσης, οι οποίοι εκφράστηκαν μέσω εξισώσεων μεταφράζοντας τα φαινόμενα σε μαθηματική μορφή. Ταυτόχρονα άρχισε και την προσπάθεια κατανόησης του εαυτού του.

Οι Πυθαγόριοι (6ο και 5ο αιώνα π.Χ.) σύνδεσαν φαινόμενα με αριθμούς (π.χ. μουσική) αλλά όχι με γενικούς φυσικούς νόμους. Ο Αριστοτέλης (384 π.Χ. - 322 π.Χ.) περιέγραψε φυσικά φαινόμενα, αλλά ποιοτικά, χωρίς μαθηματική διατύπωση. Επίσης δημιούργησε την Συλλογιστική, τη μελέτη των συλλογισμών, η οποία θεωρείται ο πρόγονος του τρόπου που “σκέφτονται” οι υπολογιστές. Ο Αρχιμήδης (287 π.Χ. - 212 π.Χ.) θεωρείται ο πρώτος που διατύπωσε φυσικό νόμο με μαθηματική ακρίβεια και με δυνατότητα επαλήθευσης, τον Νόμο της Άνωσης, γνωστός και ως Νόμος του Αρχιμήδη. [1], [2]

Ταυτόχρονα, η σκέψη και η φαντασία του ανθρώπου οραματίστηκε μηχανισμούς και τεχνητά όντα που θα μπορούσαν να προσομοιώσουν την ζωή και την ανθρώπινη νοημοσύνη. Τεχνητά όντα που θα κινούνταν “μόνα τους” από κάποια εσωτερική ενέργεια ή “ψυχή” κατά τους προσωκρατικούς φιλοσόφους.

Το πρώτο τεχνητό ον, αν και “έζησε” στην αρχαία ελληνική μυθολογία, ήταν ο Τάλως. Ένας ανθρωπόμορφος γίγαντας εξ ολοκλήρου από χαλκό είχε ως “λειτουργία” να περιπολεί τις ακτές της Κρήτης και να επιτηρεί την εφαρμογή των νόμων, κουβαλώντας τους μαζί του σε χάλκινες πλάκες. Πηγή ενέργειας του ήταν το ιχώρ, ένα θεϊκό υγρό αντί για αίμα. Ο Τάλως τοποθετείται στη Μινωική Εποχή από το 3000 π.Χ. έως το 1450 π.Χ. και, λόγω της αρχαιότητας της αφήγησης, υπάρχουν διαφορετικές εκδοχές της ιστορίας του. [3] [4]

Οι πρώτοι πραγματικοί πρόγονοι των τεχνητών όντων χρονολογούνται τον 5ο αιώνα π.Χ. με το Περιστέρι του Αρχύτα, τον 3ο αιώνα π.Χ. με τους Αυτόματους Καθρέπτες του Φίλωνα και τον 1ο αιώνα π.Χ. με τα Αυτόματα Θέατρα του Ήρωνα. [5]

1.2 ...στην Αναγέννηση και τον Διαφωτισμό...

Κατά την Αναγέννηση (14ος μ.Χ. - 16ος μ.Χ. αιώνας) και τον Διαφωτισμό (17ος μ.Χ. - 18ος μ.Χ. αιώνας) η επιστημονική σκέψη καθώς και η έννοια της Μηχανικής Νοημοσύνης εξελίχθηκε σημαντικά.

Αν όχι ο σημαντικότερος, ένας από τους πιο σημαντικούς και πιο σπουδαίους επιστήμονες που έζησαν ήταν ο Ser Isaac Newton (Σερ Ισαάκ Νεύτων 1643 - 1727). Διατύπωσε τους τρεις Νόμους της Κίνησης (Νόμος της Αδράνειας, Νόμος της Δύναμης και Νόμος Δράσης-Αντίδρασης) και τον Νόμο της Παγκόσμιας Έλξης (Νόμος της Βαρύτητας) καθώς και πολλές ακόμα μελέτες. Οι νόμοι αυτοί εκφράστηκαν μέσω Διαφορικών Εξισώσεων, καθιστώντας δυνατή την ακριβή περιγραφή της κίνησης των σωμάτων και της αλληλεπίδρασής τους. [6]

Ο René Descartes (Καρτέσιος) (Ρενέ Ντεκάρτ 1596 - 1650) υποστήριξε πως μόνο ο άνθρωπος έχει την δυνατότητα να σκέφτεται καθώς η σκέψη είναι η αλληλεπίδραση ψυχής και σώματος. Τα ζώα δεν έχουν ψυχή, τα θεωρούσε αυτόματα, χωρίς ψυχή ή συνείδηση. Ωστόσο αναγνώρισε ότι κάποτε μία μηχανή θα μπορούσε να μιμηθεί την ανθρώπινη σκέψη αλλά ποτέ να έχει αληθινή κατανόηση ή συνείδηση. [7]

Ο Gottfried Wilhelm Leibniz (Γκότφριντ Βίλχελμ Λάιμπνιτς 1646 - 1716) ήταν από τους πρώτους που οραματίστηκε μια Μηχανή η οποία θα μπορούσε να κάνει υπολογισμούς σκέψεων, τυποποιώντας την ανθρώπινη σκέψη σε κατάλληλο αλφάβητο και μαζί με λογικούς κανόνες να μπορεί αυτός ο Μηχανισμός να τους εκτελεί γρήγορα και αξιόπιστα, σαν αριθμητικές πράξεις. Ήταν μόλις 20 χρονών. Φαντάστηκε πως αν υπήρχε κάποια λογική διαφωνία μεταξύ ανθρώπων θα μπορούσαν απλώς να λυθεί με την φράση: «Απλά θα λέγαμε “Ας υπολογίσουμε”, και χωρίς περαιτέρω συζήτηση θα δούμε ποιος έχει δίκιο.».

Πίστευε ότι το δυαδικό σύστημα είναι η απόλυτη γλώσσα της λογικής. Ο αριθμός 0 αντιπροσώπευε το “Τίποτα” και ο αριθμός 1 τον “Θεό”. Ενώ ο Blaise Pascal (Μπλεζ Πασκάλ 1623 - 1662) κατασκεύασε την Pascaline, μια μηχανική αριθμομηχανή που εκτελούσε τις πράξεις της πρόσθεσης και της αφαίρεσης ο Leibniz βελτίωσε την ιδέα, κατασκευάζοντας τον Step Reckoner, ο οποίος εκτελούσε και τις πράξεις του πολλαπλασιασμού και της διαίρεσης. [8] [9]

Αν και δεν κατασκεύασε πραγματική “Λογική Μηχανή”, όπως θα την ορίζαμε σήμερα, θεωρείται πρόδρομος της Σύγχρονης Μαθηματικής Λογικής (Modern Mathematical Logic), της Υπολογιστικής Σκέψης (Computational Thinking) και, κατ’επέκταση, της Τεχνητής Νοημοσύνης (Artificial Intelligence - AI).

Ο Charles Babbage (Τσαρλς Μπάμπατζ 1791 - 1871) θεωρείται ο “πατέρας” του υπολογιστή. Σχεδίασε την Μηχανή Διαφορών 1 και 2 (Difference Engine 1 and 2, 1822) και την Αναλυτική Μηχανή (Analytical Engine, 1934). Η δεύτερη θεωρείται ως ο πρώτος υπολογιστής γενικού σκοπού καθώς είχε μνήμη, επεξεργαστή και είσοδο δεδομένων διάτριτες κάρτες, οι οποίες κατεύθυναν την μηχανή στην εκτέλεση πολύπλοκων μαθηματικών πράξεων. [10]

Η Ada Lovelace (Augusta Ada King, Countess of Lovelace) (Άντα Λάβλεϊς - Ωγκάστα Έιντα Κινγκ, Κόμισσα του Λάβλεϊς 1815 - 1852) θεωρείται η πρώτη προγραμματίστρια στην ιστορία. Μαθηματικός και συνεργάτιδα του Babbage κατάλαβε πως η Αναλυτική Μηχανή δεν ήταν απλώς για αριθμητικούς υπολογισμούς, αλλά μπορούσε να επεξεργαστεί οποιοδήποτε περιεχόμενο (μουσική, σύμβολα, εικόνες). Το 1843 μετέφραζε ένα άρθρο για την Αναλυτική Μηχανή και πρόσθεσε δικές τις σημειώσεις οι οποίες συμπεριλάμβαναν τον πρώτο αλγόριθμο. [11], [12]

Ο Babbage δεν κατάφερε να δει τις Μηχανές του πλήρως λειτουργικές. Σε

αντίθεση με την Διαφορική Μηχανή 2,η οποία ολοκληρώθηκε το 1991 από το Μουσείο Επιστήμης του Λονδίνου (Science Museum, London) η Αναλυτική Μηχανή παραμένει ένα κενοτόμο σχέδιο σε χιλιάδες σελίδες λεπτομερών σχεδίων και σημειώσεων, εκτός από κάποια τμήματα της που κατασκευάστηκαν μεμονωμένα. Αν η Μηχανή είχε ολοκληρωθεί εγγαίως το πρόγραμμα της Lovelace θα υπολόγιζε τους αριθμούς Bernoulli χωρίς κανένα λογικό σφάλμα. [13], [14]

1.3 ...στον Alan Turing...

Ο Alan Matheson Turing (Άλαν Μάθσον Τιούρινγκ 1912 - 1954) θεωρείται ο “πατέρας” της Επιστήμης των Υπολογιστών και της Τεχνητής Νοημοσύνης και ένας από τους σημαντικότερους επιστήμονες του 20ου αιώνα.

Εκτός από την θετική συμβολή του στον Β' Παγκόσμιο Πόλεμο, με την αποκρυπτογράφηση των μηνυμάτων της μηχανής Enigma από τους ναζί, με την συσκευή Bombe που σχεδίασε, ο Turing περιέγραψε την ιδέα της Καθολικής Μηχανής Τιούρινγκ (Universal Turing Machine 1936), μια θεωρητική μηχανή που θα μπορούσε να επιλύσει οποιοδήποτε μαθηματικό πρόβλημα, αν και μόνο αν αυτό μπορεί να αποτυπωθεί από έναν αλγόριθμο που τερματίζει.

Ο Turing πίστευε ότι «οι Μηχανές θα ανταγωνιστούν τελικά τους ανθρώπους σε όλα τα καθαρά διανοητικά πεδία». Εισαγάγει το Παιχνίδι Μίμησης (Imitation Game 1950) ή όπως έχει καθιερωθεί σήμερα Τεστ Τιούρινγκ (Turing Test). Είναι ένας πρακτικός τρόπος αξιολόγησης της Μηχανικής Νοημοσύνης. Το πείραμα αποτελούταν από έναν αμερόληπτο εξεταστή που επικοινωνούσε γραπτώς με έναν άνθρωπο και μία Μηχανή. Αν ο αμερόληπτος εξεταστής δεν ήταν σε θέση να διακρίνει την Μηχανή από τον άνθρωπο, τότε πρέπει να της αποδοθεί νοημοσύνη. Βέβαια, δεν είχε σημασία για τον Turing αν η Μηχανή, “καταλαβαίνει” ή “προσποιείται” κατά την αλληλεπίδρασή της με τον άνθρωπο, επομένως το Τεστ Τιούρινγκ δεν μετράει την εσωτερική κατανόηση της Μηχανής αλλά την ικανότητα της να επιδεικνύει συμπεριφορές συγκρίσιμες με αυτές του ανθρώπου.

Στο Intelligent Machinery (1948) ο Turing αναφέρει ότι «Αντί να προσπαθούμε να παράγουμε ένα πρόγραμμα που να προσομοιώνει την ενήλικη διάνοια, γιατί να μην προσπαθούμε να παράγουμε ένα που να προσομοιώνει την παιδική; Αν αυτό υποβαλλόταν σε μια κατάλληλη πορεία εκπαίδευσης, θα προέκυπτε ο ενήλικος εγκέφαλος.». Υποστήριζε, επίσης, ότι η Μηχανή θα μπορούσε να τροποποιεί τον δικό της κώδικα μέσω διαδικασιών εκπαίδευσης με “ανταμοιβές” και “τιμωρίες” ακόμα και βάσει εμπειρίας. [15], [16], [17], [18], [19]

1.4 ...στο μεσοδιάστημα...

Το 1943 συναντάμε τις ρίζες της Τεχνητής Νοημοσύνης. Ο Warren Sturgis McCulloch (Ουάρεν Μακάλοχ 1898 - 1969) και ο Walter Harry Pitts Jr. (Ουόλτερ Πιτς 1923 - 1969) δημοσίευσαν ότι οι βιολογικοί νευρώνες μπορούν να μοντελοποιηθούν με την χρήση Μαθηματικής Λογικής. Πρότειναν ένα απλοποιημένο μοντέλο βιολογικού νευρώνα, τον Νευρώνα McCulloch-Pitts. Η σύνδεση βιολογίας με την λογική αποτέλεσε την βάση για τα Νευρωνικά Δίκτυα και τον σχεδιασμό της Βαθιάς Μάθησης (Deep Learning) όπως τα γνωρίζουμε σήμερα. [20] [21]

Σε εργασία του το 1948, ο Turing αν και περιέχει την έκφραση “learning machine” ή στα ελληνικά “μηχανή που μαθαίνει” σε κείμενο του, δεν την χρησιμοποιεί με τον τρόπο που το χρησιμοποιούμε σήμερα, αν και είναι ο πρόδρομος του

Machine Learning. [22]

Το 1951 ο Marvin Lee Minsky (Μάρβιν Μίνσκι 1927 - 2016) και ο Dean S. Edmonds (Ντιν Έντμοντς 1924 - 2018) κατασκευάζουν το πρώτο γνωστό τεχνητό Νευρωνικό Δίκτυο αποτελούμενο από 40 νευρώνες και 3000 λυχνίες, το SNARC (Stochastic Neural Analog Reinforcement Calculator). Σκοπός ήταν η μελέτη του πώς ένα Νευρωνικό Δίκτυο θα μπορούσε να μάθει εμπειρικά, από “επιβράβευση” ή “τιμωρία” (Reinforcement Learning). Το SNARC ήταν αναλογικό, αφού λειτουργούσε με λυχνίες, και χρησιμοποιούσε τυχαία στοιχεία (Stochastic Elements) για να μιμηθεί τη νευρωνική δραστηριότητα. [23]

Το 1956 ο όρος Τεχνητή Νοημοσύνη (Artificial Intelligence - AI) γίνεται επίσημα επιστημονικό πεδίο στη Διάσκεψη του Νταρτμουθ (Dartmouth Conference). Με την πάροδο των δεκαετιών και με την ανάπτυξη της τεχνολογίας αλλά και του θεωρητικού υπόβαθρου, η Τεχνητή Νοημοσύνη γνωρίζει σημαντική πρόοδο. [24]

Το 1958 σηματοδοτεί την δημιουργία του πρώτου ψηφιακού Νευρωνικού Δικτύου, η πρώτη ουσιαστική εφαρμογή της θεωρίας των McCulloch και Pitts. Ο Frank Rosenblatt (Φρανκ Ροσενμπλατ 1928 - 1971) παρουσίασε τον αλγόριθμο Perceptron, ένα απλό Νευρωνικό Δίκτυο που μπορούσε να εκπαιδευτεί και μαθαίνοντας από τα λάθη του και να πάρει αποφάσεις βασισμένες στα δεδομένα εισόδου. [25]

Το 1980 αναπτύχθηκε ένα από τα πρώτα Πολυεπίπεδα Νευρωνικά Δίκτυα (Multi-Layer Neural Networks - MLNNs). Ο Kunihiko Fukushima (Κουνιχίκο Φουκουσίμα 1936 -) παρουσίασε το Neocognitron το οποίο σχεδιάστηκε για να αναγνωρίζει οπτικά πρότυπα, ανεξάρτητα από τη θέση τους ή την παραμόρφωση τους. Θεωρείται πρόδρομος των Συνελικτικών Νευρωνικών Δικτύων (Convolutional Neural Networks - CNNs). Πηγή έμπνευσης αποτέλεσε μια έρευνα του 1962 πάνω στην βιολογία των David Hunter Hubel (Ντέιβιντ Χιούμπελ 1926 - 2013) και Torsten Nils Wiesel (Τορστεν Ουίσελ 1924 -). Ανακάλυψαν ότι οι νευρώνες δεν αντιδρούν σε ολόκληρες εικόνες, αλλά σε συγκεκριμένα γεωμετρικά χαρακτηριστικά. [26] [27]

Το 1986 σημειώνεται σημαντική εξέλιξη στην εκπαίδευση των Νευρωνικών Δικτύων. Το μεγαλύτερο πρόβλημα, μέχρι εκείνη την περίοδο, ήταν το πώς μπορεί ένα Νευρωνικό Δίκτυο με πολλά Επίπεδα (Layers) να “μαθαίνει” αποτελεσματικά. Στα σύνθετα δίκτυα δεν υπήρχε τρόπος διόρθωσης των σφαλμάτων στα εσωτερικά επίπεδα. Την λύση του προβλήματος έδωσε ο αλγόριθμος της Οπισθοδιάδοσης Προσαρμογής Σφάλματος (Backpropagation), ο οποίος περιγράφεται σε επόμενο κεφάλαιο. Τον αλγόριθμο παρουσίασαν οι Geoffrey Everest Hinton (Τζέφρι Χιντον 1947 -), David Everett Rumelhart (Ντέιβιντ Ρούμελχαρτ 1942 - 2011) και Ronald James Williams (Ρόναλντ Ουίλιαμς 1945 - 2024) βάζοντας έτσι τις βάσεις για την σύγχρονη εποχή της Βαθιάς Μάθησης. [28]

1.5 ...στο Σήμερα...

Από το 2000 και ύστερα η Τεχνητή Νοημοσύνη γνωρίζει τεράστια ανάπτυξη. Η ευρεία χρήση του Διαδικτύου βοήθησε στη συλλογή τεράστιου όγκου δεδομένων (Big Data), η αύξηση της υπολογιστικής ισχύος των Κεντρικών Μονάδων Επεξεργασίας (CPUs - Central Processing Units), καθώς και η εκμετάλλευση των Μονάδων Επεξεργασίας Γραφικών (GPUs - Graphics Processing Units) έδωσαν την δυνατότητα για ταχύτερους υπολογισμούς και την εκτέλεση υπολογιστικά απαιτητικών αλγόριθμων.

Το 2006 ξεκινάει η αναγέννηση της Τεχνητής Νοημοσύνης με τους Hinton, Simon Osindero (Σάιμον Οσιντέρο) και Yee-Whye Teh (Γι Ουάι Τε) να αποδεικνύουν ότι η εκπαίδευση σε Νευρωνικά Δίκτυα με πολλά επίπεδα μπορεί να γίνει αποτελεσματικά, εισαγάγοντας την έννοια των Δίκτυων Βαθιάς Πίστης (Deep Belief Networks - DBNs), η οποία είναι θεμελιώδης για την Βαθιά Μάθηση. [29]

Το 2009 οι Yann André Le Cun (Γιαν ΛεΚαν 1960 -), Yoshua Bengio (Γιόσουα Μπέντζιο 1964 -) και Hinton παρουσίασαν μια γενική ανασκόπηση για τα Νευρωνικά Δίκτυα και την Βαθιά Μάθηση οξύνοντας το ενδιαφέρον της επιστημονικής κοινότητας θέτοντας τις βάσεις για τις επόμενες δεκαετίες. [30]

Μέχρι το 2015, όσο πιο πολλά επίπεδα προσθέταμε σε ένα Νευρωνικό Δίκτυο τόσο πιο δύσκολο ήταν να εκπαιδευτεί. Οι Kaiming He (Κέμινγκ Χι), Xiangyu Zhang (Σιανγκγιου Τζαν), Shaoqing Ren (Σαοτσίν Ρεν) και Jian Sun (Τζιάν Σαν), της Microsoft Research, ανέπτυξαν το Residual Neural Network (ResNet) εισάγοντας τις Συνδέσεις Παράκαμψης (Skip Connections) καταφέρνοντας να εκπαιδεύσουν Νευρωνικά Δίκτυα με πολλά επίπεδα, για την εποχή. [31]

Καθοριστικό ρόλο είχε και η ανάπτυξη Πλαισίων Βαθιάς Μάθησης (Deep Learning Frameworks), τα οποία περιέχουν ένα σύνολο εργαλείων για την δημιουργία, την εκπαίδευση και την ανάπτυξη Νευρωνικών Δικτύων. Τα πιο δημοφιλή πλαίσια είναι το TensorFlow, που αναπτύχθηκε από την ομάδα Google Brain και το Pytorch που αναπτύχθηκε από το Facebook AI Research. Επίσης, σημαντικό είναι και το Keras από τον François Chollet (Φρανσουά Σολέ) το οποίο είναι βιβλιοθήκη υψηλού επιπέδου που ενσωματώθηκε στο TensorFlow. [32]

1.6 ...στο Τώρα

Μέχρι πρότινος, η Τεχνητή Νοημοσύνη ήταν βασισμένη μόνο στον τεράστιο όγκο δεδομένων για την έκβαση αποτελέσματος. Αυτό πολλές φορές την καθιστούσε κοστοβόρα, απαιτούσε σημαντική υπολογιστική ισχύ και καταναλάωνε πολύ χρόνο και ενέργεια. Επιπλέον, αν τα δεδομένα ήταν περιορισμένα ήταν δύσκολη η σωστή εκπαίδευση του Νευρωνικού Δικτύου.

Το 2017 συναντάμε μια πραγματική καινοτομία. Οι Maziar Raissi (Μαζιάρ Ραΐσι), Paris Perdikaris (Πάρης Περδικάρης) και George Karniadakis (Γιώργος Καρνιαδάκης) παρουσίασαν Νευρωνικά Δίκτυα συνδυάζοντας στην εκπαίδευση τους θεμελιώδεις νόμους της φυσικής, τα Physics-Informed Neural Networks (PINNs). Έτσι, αντί να μαθαίνουν από παραδείγματα και δεδομένα, η εκπαίδευση τους γίνεται ικανοποιώντας τις Μερικές Διαφορικές Εξισώσεις (Partial Differential Equations) που περιγράφουν το υπό μελέτη φυσικό φαινόμενο. Με αυτόν τον τρόπο απαιτούν λιγότερα δεδομένα, προσφέρουν μεγαλύτερη ακρίβεια στις προβλέψεις και οδηγούν σε πιο αξιόπιστα αποτελέσματα, ειδικά σε προβλήματα όπου τα δεδομένα είναι περιορισμένα ή η Αναλυτική Λύση (Analytical Solution) είναι άγνωστη. [33], [34], [35]

Η αποτελεσματικότητα των PINNs συνδέεται άμεσα με το Θεώρημα Καθολικής Προσέγγισης (Universal Approximation Theorem - UAT), σύμφωνα με το οποίο ένα Νευρωνικό Δίκτυο με επαρκή αριθμό νευρώνων μπορεί να προσεγγίσει οποιαδήποτε συνεχής συνάρτηση με όσο μεγάλη ακρίβεια επιθυμούμε. Η επικρατέστερη διατύπωση βασίζεται στο έργο του George V. Cybenko (Τζόρτζ Σάιμπένκο) 1989 και του Kurt Hornik (Κερτ Χόρνικ) 1991. [36], [37]

Θεώρημα 1.6.1 (Θεώρημα Καθολικής Προσέγγισης - Universal Approximation Theorem). Έστω $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ μια μη σταθερή, φραγμένη και μονοτονικά αύξουσα

συνεχή συνάρτηση. Έστω $I_m = [0, 1]^m$ ο μοναδιαίος υπερκύβος στις m διαστάσεις και $C(I_m)$ ο χώρος των συνεχών συναρτήσεων στο I_m .

Τότε, για οποιαδήποτε συνάρτηση $f \in C(I_m)$ και για οποιοδήποτε $\epsilon > 0$, υπάρχει ένας ακέραιος N , πραγματικές σταθερές $v_i, b_i \in \mathbb{R}$ και διανύσματα βαρών $w_i \in \mathbb{R}^m$ τέτοια ώστε η συνάρτηση:

$$(1.1) \quad F(x) = \sum_{i=1}^N v_i \sigma(w_i^T x + b_i)$$

να ικανοποιεί τη συνθήκη:

$$(1.2) \quad \sup_{x \in I_m} |F(x) - f(x)| < \epsilon$$

όπου x το διάνυσμα εισόδου, N αριθμός Νευρώνων - το πλάτος του Νευρωνικού Δικτύου, w_i τα διανύσματα των βαρών της εισόδου (*Input Weights*), b_i η μεροληψία (*Bias*), $\sigma(\cdot)$ η συνάρτηση ενεργοποίησης (*Activation Function*), v_i τα διανύσματα των βαρών εξόδου (*Output Weights*), $F(x)$ η προσέγγιση του Νευρωνικού Δικτύου, ϵ το σφάλμα προσέγγισης του Νευρωνικού Δικτύου.

Το θεώρημα μας εγγυάται ότι υπάρχει ένα, ρηχό, Νευρωνικό Δίκτυο το οποίο μπορεί να προσεγγίσει ένα πρόβλημα που μπορεί να περιγραφεί από μία μαθηματική σχέση. Όμως, δεν μας λέει πόσο μεγάλο πρέπει να είναι το N , πώς να βρούμε τα κατάλληλα βάρη ή πόσα δεδομένα χρειάζονται για την εκπαίδευση.

Η διατύπωση του Cybenko αναφέρεται σε Νευρωνικά Δίκτυα ενός επιπέδου με πλάτος $N \rightarrow \infty$. Στην πράξη αλλά και σε αναφορές των Raissi, Perdikari και Karniadaki χρησιμοποιούμε Βαθιά Νευρωνικά Δίκτυα, πολλών επιπέδων, καθώς αυτό προσφέρει βελτίωση της προσέγγισης σε σχέση με το πλάτος.

Ο Hornik το 1991 απέδειξε ότι αν η σ είναι k -φορές διαφορίσιμη, το Νευρωνικό Δίκτυο μπορεί να προσεγγίσει την f και όλες τος παραγώγους της έως τάξης k , προσφέροντας την δυνατότητα στα PINNs να ελαχιστοποιήσουν τη Συνάρτηση Απώλειας (*Loss Function*). [37]

Κεφάλαιο 2

Τεχνητή Νοημοσύνη - Νευρωνικά Δίκτυα - PINNs

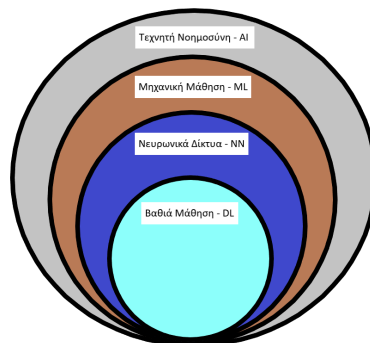
2.1 Τεχνητή Νοημοσύνη

Η Τεχνητή Νοημοσύνη, επισήμως από το 1956, αποτελεί επιστημονικό κλάδο της Πληροφορικής που μελετά και αναπτύσσει αλγοριθμικές μεθόδους ικανές να επιδεικνύουν συμπεριφορές που σχετίζονται με τη νοημοσύνη, όταν πραγματοποιούνται από ανθρώπους. Συνδυάζει στοιχεία από τη Μαθηματική Λογική, της Επιστήμης των Υπολογιστών, τη Στατιστική και την Νευροεπιστήμη.

Στόχος της Τεχνητής Νοημοσύνης είναι η δημιουργία αλγορίθμων και υπολογιστικών μοντέλων που μπορούν να αναπαριστούν γνώση, να εξάγουν συμπεράσματα και να προσαρμόζονται σε νέα δεδομένα, βελτιώνοντας την απόδοσή τους χωρίς την επέμβαση ανθρώπου για επαναπρογραμματισμό.

Η Μηχανική Μάθηση (Machine Learning) είναι υποπεδίο της Τεχνητής Νοημοσύνης. Είναι ο το αντικείμενο της ανάπτυξης αλγορίθμων που μαθαίνουν από εμπειρία.

Η Βαθιά Μάθηση (Deep Learning) αποτελεί υποκατηγορία της Μηχανικής Μάθησης και βασίζεται σε χρήση πολυεπίπεδων Νευρωνικών Δικτύων.



Σχήμα 2.1: Τα υποπεδία της Τεχνητής Νοημοσύνης

2.2 Νευρωνικά Δίκτυα

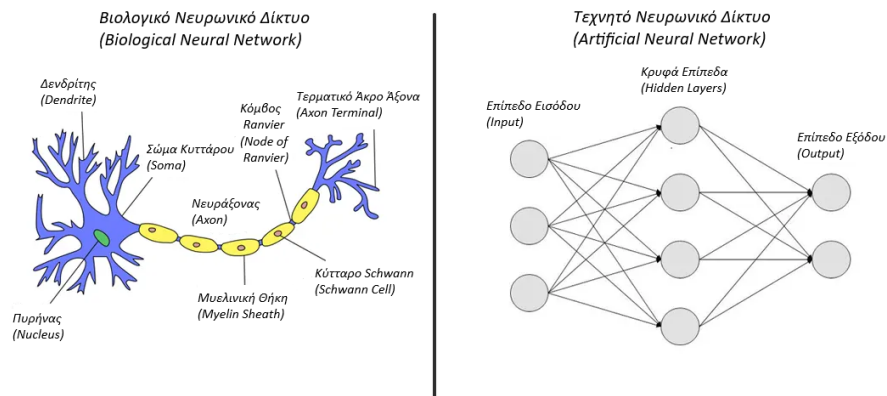
Μηχανική Μάθηση ονομάζεται η ικανότητα ενός υπολογιστικού συστήματος να δημιουργεί μοντέλα ή πρότυπα από ένα σύνολο δεδομένων. Ασχολείται με την μελέτη αλγορίθμων, οι οποίοι μπορούν να βελτιστοποιήσουν το αποτέλεσμα τους μέσω της αξιοποίησης προηγούμενης γνώσης και εμπειρίας.

Τα Νευρωνικά Δίκτυα αποτελούν υποσύνολο της Μηχανικής Μάθησης, όπως είδαμε πιο πάνω, και βασίζονται σε μαθηματικά μοντέλα που προσεγγίζουν μη γραμμικές συναρτήσεις.

Τα Νευρωνικά Δίκτυα προσπαθούν να μιμηθούν το βιολογικό νευρωνικό δίκτυο του ανθρώπινου εγκεφάλου. Ένας τυπικός εγκεφαλος περιέχει περίπου 10^{11} νευρώνες. Κάθε νευρώνας αποτελείται από το κυτταρικό σώμα και τις συνδετικές δομές του: πολυάριθμους δενδρίτες, οι οποίοι λειτουργούν ως είσοδος, μεταφέροντας σήματα προς το κυτταρικό σώμα, και έναν μοναδικό άξονα, που λειτουργεί ως έξοδος, μεταδίδοντας σήματα προς άλλους νευρώνες.

Οι δενδρίτες λαμβάνουν και μεταφέρουν ηλεκτρικά και χημικά σήματα από άλλους νευρώνες, ενώ οι άξονες προωθούν ηλεκτρικούς παλμούς που διευκολύνουν τη μετάδοση πληροφοριών. Αυτή η διαδικασία αποτελεί τη βάση για βασικές νευροβιολογικές λειτουργίες, όπως η αποθήκευση και ανάκτηση μνημών, η ρύθμιση της μυϊκής δραστηριότητας και η ολοκλήρωση σύνθετων γνωστικών διαδικασιών.

Στα Βιολογικά Νευρωνικά Δίκτυα, οι συνάψεις (Synapses) λειτουργούν ως τα βασικά σημεία επεξεργασίας και μετάδοσης πληροφορίας. Κάθε σύναψη έχει συναπτικό βάρος (Synaptic Weight) που καθορίζει πόσο ισχυρά το σήμα από τον προσαγωγό νευρώνα επηρεάζει τον μεταγενέστερο νευρώνα. Η προσαρμογή αυτών των βαρών με την εμπειρία, μέσω της συναπτικής πλαστικότητας (Synaptic Plasticity), είναι η βάση της μάθησης και της μνήμης στον εγκεφαλο 2.2. [38]



Σχήμα 2.2: BNΔ και TNΔ

2.2α' Αρχιτεκτονική ενός Τεχνητού Νευρώνα

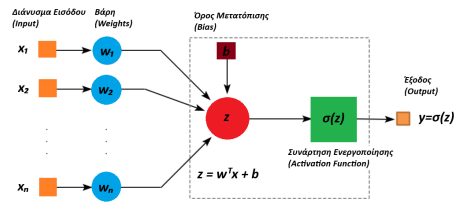
Στα Τεχνητά Νευρωνικά Δίκτυα, η αντίστοιχη λειτουργία των συνάψεων υλοποιείται μέσω των βαρών (Weights) που συνδέουν τους Τεχνητούς Νευρώνες (Neurons ή Nodes). Κάθε Τεχνητός Νευρώνας δέχεται πολλαπλές εισόδους, πολλαπλασιάζει κάθε εισροή με το αντίστοιχο βάρος, και εφαρμόζει μια Συνάρτηση

Ενεργοποίησης (Activation Function) για να υπολογίσει την έξοδο του.

Η συνάρτηση ενεργοποίησης απαιτείται να είναι μια μη γραμμική συνάρτηση, καθώς αυτό επιβάλλεται από το UAT (1.6.1) αφού για γραμμική συνάρτηση δεν θα ίσχυε, αλλά επίσης είναι η μη γραμμικότητα που βοηθάει στην μάθηση πολύπλοκων μοτίβων και στην επίλυση μη γραμμικών προβλημάτων.

Τα βάρη προσαρμόζονται κατά την διάρκεια της εκπαίδευσης, επιτρέποντας στο δίκτυο να βελτιώνει σταδιακά την απόδοσή του και να παράγει πιο ακριβείς προβλέψεις.

Η εικόνα (2.3) παρουσιάζει την αρχιτεκτονική ενός μόνο Τεχνητού Νευρώνα, η οποία είναι η βασική υπολογιστική μονάδα. [38]



Σχήμα 2.3: Αρχιτεκτονική Τεχνητού Νευρώνα (ενός επιπέδου)

Η λειτουργία των συνάψεων, που πραγματοποιείται στα Βιολογικά Νευρωνικά Δίκτυα, μοντελοποιείται μέσω του διανύσματος βαρών

$$(2.1) \quad w = [w_1, w_2, \dots]^T,$$

το οποίο καθορίζει την συνεισφορά κάθε εισόδου. Κάθε νευρώνας δέχεται ένα διάνυσμα εισόδου

$$(2.2) \quad x = [x_1, x_2, \dots]^T,$$

και υπολογίζει το σταθμισμένο άθροισμα

$$(2.3) \quad z = w^T x + b$$

όπου b ο όρος της μεροληψίας (Bias). Στη συνέχεια εφαρμόζεται μια μη γραμμική συνάρτηση ενεργοποίησης $\sigma(\cdot)$.

$$(2.4) \quad y = \sigma(z)$$

ώστε να υπολογιστεί η έξοδος y .

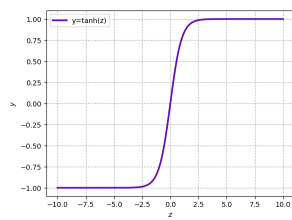
Συναρτήσεις ενεργοποίησης που συναντάμε συχνά είναι η Υπερβολική Εφαπτομένη. Επιστρέφει τιμές στο διάστημα $(-1, 1)$, συμμετρική γύρω από το 0 βοηθώντας στη σταθερή εκπαίδευση και είναι ομαλή και άπειρα διαφορίσιμη. Χρησιμοποιείται συχνά σε PINNs για κυματικές εξισώσεις. Στην πράξη, υπάρχουν αρκετές κλασικές συναρτήσεις ενεργοποίησης εκτός της Tanh, όπως η Σιγμοειδής (Sigmoid) και η ReLU (Rectified Linear Unit). Η Sigmoid επιστρέφει τιμές στο διάστημα $(0, 1)$ και χρησιμοποιείται συχνά σε προβλήματα δυαδικής ταξινόμησης λόγω της ομαλής

παραγώγου και της δυνατότητας ερμηνείας ως πιθανότητα. Η ReLU (Rectified Linear Unit) είναι απλή και αποδοτική, επιτρέποντας ταχύτερη σύγκλιση σε βαθιά νευρωνικά δίκτυα λόγω της γραμμικής συμπεριφοράς για θετικές εισόδους. Ακολουθεί μια σύντομη παρουσίαση τους.

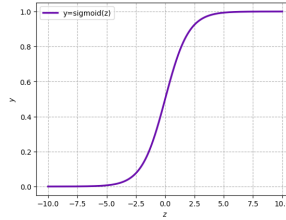
$$(2.5) \quad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$(2.6) \quad \text{Sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

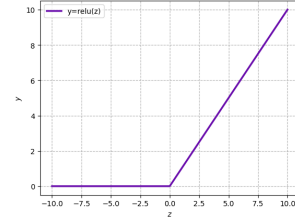
$$(2.7) \quad \text{ReLU}(z) = \max(0, z)$$



(α') Υπερβολική Εφαπτομένη



(β') Σιγμοειδής



(γ') ReLU

Στο επόμενο κεφάλαιο θα μελετηθούν οι Συναρτήσεις Ενεργοποίησης με βάση την ομαλότητά τους, δηλαδή συναρτήσεις που είναι συνεχείς και παραγωγίσιμες σε όλο το πεδίο ορισμού τους, η οποία είναι καθοριστικά σημαντική.

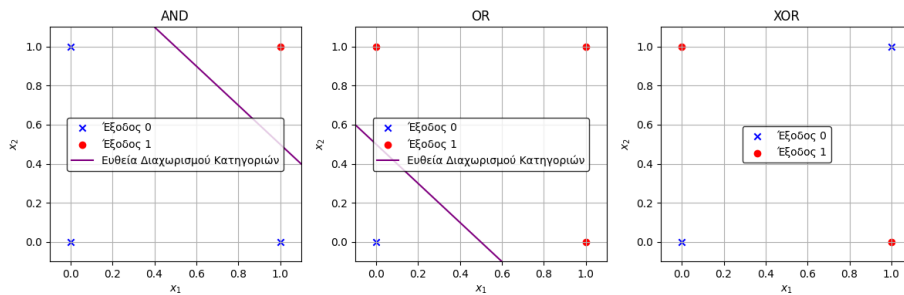
2.2β' Περιορισμοί του ενός Τεχνητού Νευρώνα

Ένα Νευρωνικό Δίκτυο που αποτελείται μόνο από έναν Τεχνητό Νευρώνα, χωρίς μια μη-γραμμική συνάρτηση ενεργοποίησης (Perceptron) δεν θα αρκούσε για την προσέγγιση σύνθετων προβλημάτων, καθώς ο Perceptron είναι γραμμικός διαχωριστής και δεν μπορεί να προσεγγίσει μη γραμμικές συναρτήσεις.

Γραμμικά Διαχωρίσιμο είναι ένα σύνολο δεδομένων όταν μπορεί να χωριστεί πλήρως σε κατηγορίες από μία ευθεία σε δύο διαστάσεις, από ένα επίπεδο σε τρεις διαστάσεις ή γενικότερα, από ένα υπερεπίπεδο σε υψηλότερες διαστάσεις. [39]

Ένα χαρακτηριστικό παράδειγμα που αναδεικνύει τους περιορισμούς του Perceptron είναι οι Λογικές Συναρτήσεις (Boolean Functions) Λογική Σύζευξη (ΚΑΙ - AND), Λογική Διάζευξη (Ή - OR) και Αποκλειστική Διάζευξη (Αποκλειστικό Ή - XOR).

Στα γραφήματα (2.5) απεικονίζονται τα αντίστοιχα σύνολα δεδομένων, αποτελούμενα από 0 και 1, καθώς και οι ευθείες διαχωρισμού όπου αυτές υπάρχουν.



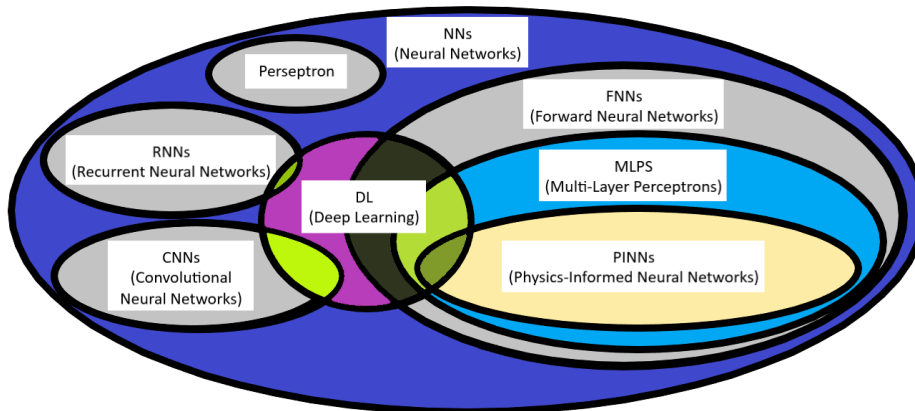
Σχήμα 2.5: AND, OR, XOR με την Ευθεία Διαχωρισμού

Οι Λογικές Συναρτήσεις AND και OR είναι γραμμικά διαχωρίσιμες, καθώς υπάρχει ευθεία που διαχωρίζει πλήρως τις δύο κατηγορίες. Η Λογική Συναρτηση XOR δεν είναι γραμμικά διαχωρίσιμη, καθώς δεν υπάρχει ευθεία που διαχωρίζει πλήρως τις δύο κατηγορίες.

Τελικά, ο Perceptron αν και μπορεί να αναπαραστήσει τη συνάρτηση AND και OR, δεν μπορεί να αναπαραστήσει τη συνάρτηση XOR, συνεπώς δεν επαρκεί για την προσέγγιση μη γραμμικών συναρτήσεων.

2.2γ' Περισσότεροι του ενός Τεχνητού Νευρώνα

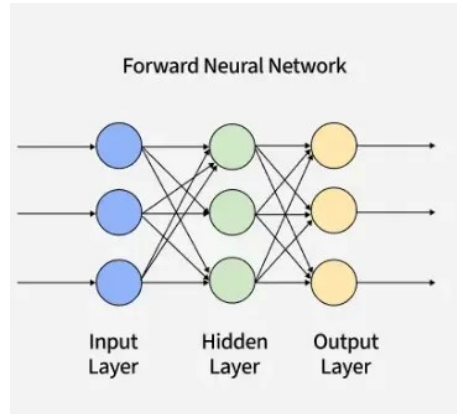
Η προσθήκη περισσότερων Τεχνητών Νευρώνων, σε συνδυασμό μη γραμμικής συνάρτησης ενεργοποίησης, επιτρέπει στο δίκτυο να προσεγγίζει σύνθετες μη γραμμικές σχέσεις, αποτελώντας τη βάση των Νευρωνικών Δικτύων Εμπρόσθιας Τροφοδότησης (Feedforward Neural Network - FNN). Τα FNN περιλαμβάνουν την υποκατηγορία των Πολυεπίπεδων Νευρωνικών Δικτύων (Multi-Layer Perceptron - MLP), όπου αυτά με την σειρά τους επεκτείνονται στην υποκατηγορία των Νευρωνικών Δικτύων Εμπλουτισμένα με Φυσικούς Νόμους (Physics-Informed Neural Networks - PINNs).



Σχήμα 2.6: Σχέσεις μεταξύ NN, Perceptron, RNN, FNN, DL και PINNs

Οι Τεχνητοί Νευρώνες οργανώνονται σε επίπεδα, αποτελούμενα από έναν αριθμό Τεχνητών Νευρώνων που λειτουργούν παράλληλα, ορίζοντας τη δομή του

FNN. Το Επίπεδο Εισόδου (Input Layer) δέχεται τα αρχικά δεδομένα και τα προωθεί προς τα επόμενα Κρυφά Επίπεδα (Hidden Layers). Στη συνέχεια τα δεδομένα μετασχηματίζονται και περνούν στο Επίπεδο Εξόδου (Output Layer), το οποίο παρέχει τα τελικά αποτελέσματα της διαδικασίας.



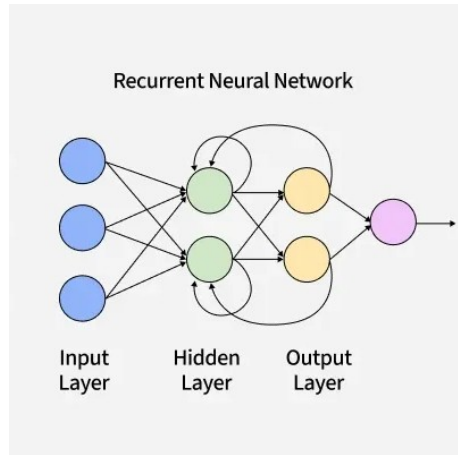
Σχήμα 2.7: Αρχιτεκτονική FNN

Στο Σχήμα (2.7), περιγράφεται ένα FNN το οποίο έχει ένα Κρυφό Επίπεδο, άρα έχει βάθος (depth) ίσο με 1. Επιπλέον, έχει τρεις Τεχνητούς Νευρώνες στο ένα Κρυφό Επίπεδο, άρα έχει πλάτος (width) ίσο με 3.

Αριστερά απεικονίζεται το διάνυσμα εισόδου. Το διάνυσμα αυτό υφίσταται γραμμικό μετασχηματισμό μέσω του πίνακα βαρών w_1 και του διανύσματος μεροληψίας b_1 και στη συνέχεια εφαρμόζεται η συνάρτηση ενεργοποίησης $\sigma(\cdot)$, παράγοντας το διάνυσμα του κρυφού επιπέδου, το οποίο βρίσκεται στην μέση.

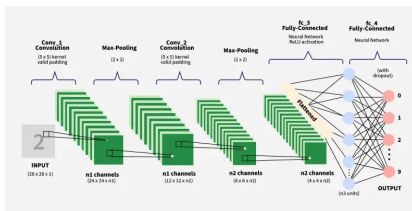
Αντίστοιχα, το διάνυσμα του κρυφού επιπέδου μετασχηματίζεται εκ νέου μέσω του πίνακα βαρών w_2 και του διανύσματος μεροληψίας b_2 και της συνάρτησης ενεργοποίησης, οδηγώντας στον υπολογισμό του διανύσματος εξόδου του δικτύου, το οποίο βρίσκεται στα δεξιά.

Εκτός από τα FNN, έχουν αναπτυχθεί και άλλες αρχιτεκτονικές. Ενδεικτικά, τα Επαναληπτικά Νευρωνικά Δίκτυα (Recurrent Neural Networks - RNNs) διαθέτουν επαναληπτικές συνδέσεις και αξιοποιούν την πληροφορία παραηγόμενων χρονικών βημάτων για τον υπολογισμό της εξόδου, όπως παρουσιάζεται στο σχήμα (2.8)



Σχήμα 2.8: Αρχιτεκτονική RNN

Μία ακόμη δημοφιλής αρχιτεκτονική Νευρωνικού Δικτύου είναι τα Συνελικτικά Νευρωνικά Δίκτυα (Convolutional Neural Networks - CNNs), τα οποία είναι σχεδιασμένα κυρίως για την ανάλυση δεδομένων με δομημένη χωρική και χρονική πληροφορία, όπως είναι οι εικόνες και τα βίντεο. Χρησιμοποιώντας ως δεδομένα εκπαίδευσης μεγάλους όγκους εικόνων, τα CNNs εκπαιδεύονται να αναγνωρίζουν χαρακτηριστικά και μοτίβα, διευκολύνοντας την ταξινόμηση ή την αναγνώριση αντικειμένων. Σε αντίθεση με τα κλασικά MLP, τα CNN δεν είναι fully connected, δεν συνδέουν κάθε νευρώνα με όλους τους προηγούμενους. Στο σχήμα (2.9) παρουσιάζεται ένα CNN το οποίο υπολογίζει αν σε μια εικόνα εμφανίζεται ο αριθμός 2, χρησιμοποιώντας για την εκπαίδευση του δεδομένα από το σύνολο MNIST (Modified National Institute of Standards and Technology). [40], [41]



(α') Αρχιτεκτονική CNN



(β') Δεδομένα του αριθμού 2 από το σύνολο MNIST

Σχήμα 2.9: Παράδειγμα CNN

2.2δ' Εμπρόσθια Προώθηση (Forward Propagation)

Ένα FNN μπορεί να οριστεί ως μια παραμετρική συνάρτηση

$$(2.8) \quad f_{\theta} : \mathbb{R}^M \rightarrow \mathbb{R}^N$$

Το αποτελείται από το σύνολο των παραμέτρων των βαρών και όρων μεροληψίας, $\theta = \{W, b\}$, καθώς και M, N οι διαστάσεις της εισόδου και της εξόδου,

αντίστοιχα.

Ένα Νευρωνικό Δίκτυο Εμπρόσθιας Τροφοδότησης με ένα κρυφό επίπεδο H νευρώνων μπορεί να οριστεί ως

$$(2.9) \quad f_{\theta}(x) = \bar{y}(x) = \sigma(W_2 \sigma(W_1 x + b_1) + b_2)$$

Το $x \in \mathbb{R}^M$ είναι το διάνυσμα εισόδου και $\bar{y} \in \mathbb{R}^N$ είναι το διάνυσμα της εκτιμώμενης εξόδου. Το βάρος και ο όρος μεροληψίας που εφαρμόζονται στην είσοδο είναι $W_1 \in \mathbb{R}^{H \times M}$ και $b_1 \in \mathbb{R}^H$ αντίστοιχα, ενώ στο κρυφό επίπεδο το βάρος και ο όρος μεροληψίας που εφαρμόζονται είναι $W_2 \in \mathbb{R}^{N \times H}$ και $b_2 \in \mathbb{R}^N$ αντίστοιχα. Η $\sigma(\cdot) : \mathbb{R} \rightarrow \mathbb{R}$ είναι η συνάρτηση ενεργοποίησης, η οποία εφαρμόζεται σε κάθε στοιχείο.

Η διαδικασία όπου το Νευρωνικό Δίκτυο Εμπρόσθιας Τροφοδότησης υπολογίζει την έξοδό του, δεδομένου διανύσματος εισόδου, ονομάζεται Εμπρόσθια Προώθηση (Forward Propagation).

Στην περίπτωση του (2.9) τα βήματα της διαδικασίας μπορούν να περιγραφούν ως εξής.

- (i) **Βήμα 1:** Δεδομένης εισόδου $x \in \mathbb{R}^M$, υπολογίζεται ο γραμμικός μετασχηματισμός μέσω των $W_1 \in \mathbb{R}^{H \times M}$ και $b_1 \in \mathbb{R}^H$

$$(2.10) \quad W_1 x + b_1$$

- (ii) **Βήμα 2:** Εφαρμογή της συνάρτησης ενεργοποίησης $\sigma(\cdot) : \mathbb{R} \rightarrow \mathbb{R}$ σε κάθε στοιχείο

$$(2.11) \quad \sigma(W_1 x + b_1)$$

- (iii) **Βήμα 3:** Με είσοδο το διάνυσμα $\sigma(W_1 x + b_1) \in \mathbb{R}^H$, υπολογίζεται ο γραμμικός μετασχηματισμός μέσω των $W_2 \in \mathbb{R}^{N \times H}$ και $b_2 \in \mathbb{R}^N$

$$(2.12) \quad W_2 \sigma(W_1 x + b_1) + b_2$$

- (iv) **Βήμα 4:** Εφαρμογή της συνάρτησης ενεργοποίησης $\sigma(\cdot) : \mathbb{R} \rightarrow \mathbb{R}$ σε κάθε στοιχείο

$$(2.13) \quad \sigma(W_2 \sigma(W_1 x + b_1) + b_2)$$

- (v) **Βήμα 5:** Έχοντας μόνο ένα κρυφό επίπεδο, η παραπάνω έξοδος ταυτίζεται με την τελική έξοδο

$$(2.14) \quad f_{\theta}(x) = \bar{y}(x) = \sigma(W_2 \sigma(W_1 x + b_1) + b_2)$$

Το αποτέλεσμα της Forward Propagation είναι η προσέγγιση $\bar{y}(x) \in \mathbb{R}^N$ της πραγματικής λύσης $y(x)$.

2.2ε' Συνάρτηση Απώλειας (Loss Function)

Η σχέση (2.9) ορίζει τη μαθηματική μορφή της εξόδου ενός FNN. Ωστόσο, προκειμένου το Νευρωνικό Δίκτυο να μπορέσει να προσεγγίσει την πραγματική λύση με επαρκή ακρίβεια, απαιτείται η εισαγωγή ενός κριτηρίου αξιολόγησης της απόδοσής του.

Η συνάρτηση απώλειας (Loss Function) εφαρμόζεται για το σύνολο της εκπαίδευσης ενός FNN μέσω του Μέσου Τετραγωνικού Σφάλματος (Mean Squared Error - MSE).

Έστω σύνολο εκπαίδευσης $\{(x_i, \bar{y}_i)\}_{i=1}^K$, με $x_i \in \mathbb{R}^M$ και $y_i \in \mathbb{R}^N$, δηλαδή

$$(2.15) \quad x = \begin{bmatrix} | & | & \cdots & | \\ x_1 & x_2 & \cdots & x_K \\ | & | & \cdots & | \end{bmatrix} \in \mathbb{R}^{M \times K}$$

$$(2.16) \quad \bar{y} = \begin{bmatrix} | & | & \cdots & | \\ \bar{y}_1 & \bar{y}_2 & \cdots & \bar{y}_K \\ | & | & \cdots & | \end{bmatrix} \in \mathbb{R}^{N \times K}$$

Η συνάρτηση απώλειας ορίζεται ως

$$(2.17) \quad \mathcal{L}(\theta) = \frac{1}{2K} \sum_{i=1}^K \|\bar{y}(x_i|\theta) - y_i\|_2^2$$

Ο συντελεστής $\frac{1}{2}$ εισάγεται για υπολογιστική ευκολία, $\|\cdot\|_2$ είναι το Μέσω Τετραγωνικό Σφάλμα (Mean Square Error - MSE), $\bar{y}(x_i|\theta)$ είναι η έξοδος που εξαρτάται από το $\theta = \{W_1, b_1, W_2, b_2\}$, το σύνολο των παραμέτρων του FNN.

Η εκπαίδευση ενός FNN βασίζεται στην ελαχιστοποίηση μιας Loss Function, η οποία ποσοτικοποιεί την απόκλιση μεταξύ της εξόδου του FNN $\bar{y}$ και της πραγματικής λύσης y . Κατά τη διάρκεια της εκπαίδευσης, τα βάρη W_i και οι όροι μεροληψίας b_i προσαρμόζονται κατάλληλα, με στόχο τη βελτίωση της προσέγγισης της πραγματικής λύσης.

Αυτό αποτελεί ένα παράδειγμα της Επιβλεπόμενης Εκπαίδευσης (Supervised Learning).

Στην Επιβλεπόμενη Εκπαίδευση, το FNN εκπαιδεύεται χρησιμοποιώντας ένα σύνολο δεδομένων, που το ονομάζουμε σύνολο εκπαίδευσης (2.15), (2.16), με σκοπό την εκμάθηση μιας βέλτιστης συνάρτησης που αντιστοιχίζει μια είσοδο σε μια επιθυμητή έξοδο.

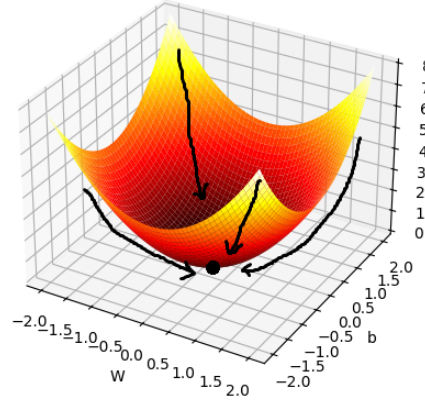
Άλλες δύο μορφές μάθησης, με τις οποίες δεν θα ασχοληθούμε σε αυτήν την εργασία, είναι οι εξής. Η πρώτη είναι η Εκπαίδευση χωρίς Επίβλεψη (Unsupervised Learning), όπου δεν παρέχουμε την έξοδο και η μάθηση γίνεται μέσω ανακάλυψης μοτίβων από το σύνολο εκπαίδευσης. Η δεύτερη είναι η Ενισχυτική Εκπαίδευση (Reinforcement Learning), όπου η εκπαίδευση γίνεται μέσω δοκιμών και λαθών. [42]

2.2ς' Οπισθοδιάδοση Σφάλματος (Back Propagation)

Η ελαχιστοποίηση της Loss Function (2.17) επιτυγχάνεται με μια επαναληπτική διαδικασία που ονομάζεται Μέθοδος Καθόδου Κλίσης (Gradient Descent). Είναι

μια επαναληπτική Μέθοδος Βελτιστοποίησης της Loss Function ενός Νευρωνικού Δικτύου, μέσω του υπολογισμού της επίδρασης κάθε παραμέτρου στην έξοδο του δικτύου.

Για να κατανοήσουμε την μέθοδο Gradient Descent, ας υποθέσουμε ότι το γράφημα της Loss Function (2.17) εκφράζεται στο Σχήμα (2.10).



Σχήμα 2.10: Υποθετικό Γράφημα Loss Function

Για υψηλές τιμές της Loss Function, η εκτιμώμενη έξοδος του δικτύου αποκλίνει περισσότερο από την πραγματική τιμή, ενώ για χαμηλές τιμές η απόκλιση είναι μικρότερη. Ο στόχος της εκπαίδευσης είναι να μετακινήσουμε τις παραμέτρους του δικτύου έτσι ώστε η Loss Function να γίνει όσο το δυνατόν μικρότερη.

Η κλίση (Gradient), δηλαδή το διάνυσμα των μερικών παραγώγων της Loss Function ως προς κάθε παράμετρο, υποδεικνύει την κατεύθυνση της μέγιστης αύξησης καθώς και το μέγεθος της αύξησης της Loss Function. Για να ελαχιστοποιηθεί η Loss Function, ενημερώνονται οι παράμετροι $\theta = \{W_1, b_1, W_2, b_2\}$ στην αντίθετη κατεύθυνση της κλίσης, προσεγγίζοντας σταδιακά το ελάχιστο της Loss Function.

$$(2.18) \quad \theta_{i+1} = \theta_i - \eta \frac{\partial \mathcal{L}(\theta_i)}{\partial \theta_i}$$

Η παράμετρος $\eta > 0$ ονομάζεται Ρυθμός Μάθησης (Learning Rate) και καθορίζει το μέγεθος του βήματος με το οποίο ενημερώνονται οι παράμετροι κατά την διάρκεια της Μεθόδου Βελτιστοποίησης. Όσο μεγαλύτερη είναι η τιμή του η , τόσο μεγαλύτερο είναι το βήμα προς το ελάχιστο, με αποτέλεσμα μεγαλύτερης πιθανότητας μη σύγκλισης στο ελάχιστο. Αντίθετα, όσο μικρότερη είναι η τιμή του η , τόσο πιο μικρά είναι τα βήματα, εξασφαλίζοντας πιο σταθερή σύγκλιση στο ελάχιστο με κόστος την πιο αργή εκπαίδευση.

Ο υπολογισμός των παραγώγων της Loss Function ως προς τις παραμέτρους του FNN πραγματοποιείται μέσω του Κανόνα της Αλυσίδας (Chain Rule).

Η μέθοδος, όπου οι παράγωγοι (gradients) ενημερώνονται από το τελευταίο επίπεδο προς τα πίσω, χρησιμοποιώντας της μεθόδου Gradient Descent για την προσαρμογή τους, είναι γνωστή ως Οπισθοδιάδοση Σφάλματος (Back Propagation).

2.2ζ' Πολυεπίπεδο Perceptron (Multi-Layer Perceptron - MLP)

Το Πολυεπίπεδο Perceptron (Multi-Layer Perceptron - MLP) είναι η γενίκευση του FNN με περισσότερα Κρυφά Επίπεδα. Κάθε νέο Κρυφό Επίπεδο εισάγει επιπλέον βάρη και όρους μεροληψίας. Η σχέση (2.9) γενικεύεται ως εξής

- (i) **Βήμα 1:** Δεδομένης εισόδου $x \in \mathbb{R}^M$, υπολογίζεται ο γραμμικός μετασχηματισμός μέσω των W_1 και b_1

$$(2.19) \quad W_1 x + b_1$$

- (ii) **Βήμα 2:** Εφαρμογή της συνάρτησης ενεργοποίησης

$$(2.20) \quad \sigma_1(W_1 x + b_1)$$

- (iii) **Βήμα 3:** Υπολογίζεται ο γραμμικός μετασχηματισμός μέσω των W_2 και b_2

$$(2.21) \quad W_2 \sigma_1(W_1 x + b_1) + b_2$$

- (iv) **Βήμα 4:** Εφαρμογή της συνάρτησης ενεργοποίησης

$$(2.22) \quad \sigma_2(W_2 \sigma_1(W_1 x + b_1) + b_2)$$

...

- (v) **Βήμα d:** Υπολογίζεται ο γραμμικός μετασχηματισμός μέσω των W_d και b_d καθώς και εφαρμογή της συνάρτησης ενεργοποίησης

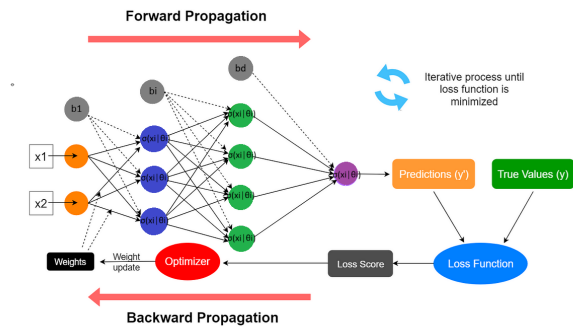
$$(2.23) \quad \sigma_d(W_d \sigma_{d-1}(\dots \sigma_3(W_2 \sigma(W_1 x + b_1) + b_2) \dots) + b_d)$$

Για ένα MLP με βάθος d , δηλαδή d Κρυφά Επίπεδα, βάρη $W_i \in \mathbb{R}^{n_{i+1} \times n_i}$ και όρους μεροληψίας $b_i \in \mathbb{R}^{n_{i+1}}$, για κάποιο $n_i \geq 1$ και $n_d = N$ σταθερό, δίνεται από την σχέση

$$(2.24) \quad f_\theta(x) = \bar{y}(x) = \sigma_d(W_d \sigma_{d-1}(\dots \sigma_3(W_2 \sigma(W_1 x + b_1) + b_2) \dots) + b_d)$$

όπου $\theta = (W_i, b_i)$ οι παράμετροι. Η συνάρτηση ενεργοποίησης μπορεί να είναι ίδια ή διαφορετική σε κάθε Επίπεδο.

Στα MLP η Loss Function (2.17) καθώς και η επαναληπτική διαδικασία ενημέρωσης του συνόλου των παραμέτρων θ μέσω της μεθόδου Back Propagation (2.18) συμπίπτουν με εκείνες των FNN.



Σχήμα 2.11: Ολοκληρωμένη Δομή MLP

Στο Σχήμα (2.11) παρουσιάζεται όλη η δομή των FNN.

2.2η' Γεωμετρία των FNN

Σύμφωνα με τα κλασικά αποτελέσματα των Cybenko [36] και Hornik [37], ένα FNN με ένα μόνο Κρυφό Επίπεδο και επαρκές πλάτος είναι καθολικός προσεγγιστής.

Νεότερα αποτελέσματα δείχνουν ότι η καθολική προσέγγιση μπορεί να επιτευχθεί και με αυθαίρετο βάθος, ακόμα και όταν το πλάτος παραμένει περιορισμένο. [43]

Παρά τα παραπάνω θεωρητικά αποτελέσματα, στην πράξη η αύξηση του βάθους οδηγεί σε δυσκολίες στην εκπαίδευση, όπως τα Vanishing Gradients (Μηδενισμός των Κλίσεων) και τα Exploding Gradients (Απειρισμός των Κλίσεων). [42]

Για τον λόγο αυτό, τα FNN που χρησιμοποιούνται σε προβλήματα προσέγγισης συναρτήσεων τείνουν να διατηρούν σχετικά μικρό βάθος, συνήθως από 4 έως 8 Κρυφά Επίπεδα, ενώ σε πολλές εφαρμογές επαρκούν ακόμη και 1–3 επίπεδα. [42]

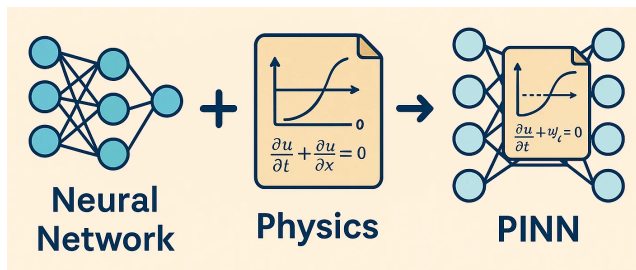
Στα πλαίσια της σύγχρονης Βαθιάς Μάθησης έχουν αναπτυχθεί βαθιές αρχιτεκτονικές (Deep MLP), οι οποίες μπορούν να ξεπεράσουν το βάθος των 100 Κρυφών Επιπέδων μέσω εξειδικευμένων τεχνικών. Ωστόσο, οι τεχνικές αυτές δεν είναι συμβατές με εφαρμογές που απαιτούν ομαλές παραγώγους υψηλής τάξης. [31]

Κεφάλαιο 3

Νευρωνικά Δίκτυα Εμπλουτισμένα με Φυσικούς Νόμους (Physics-Informed Neural Networks - PINNs)

Όπως φαίνεται και στο Σχήμα (2.6), τα Νευρωνικά Δίκτυα Εμπλουτισμένα με Φυσικούς Νόμους (Physics-Informed Neural Networks - PINNs) είναι FNN τα οποία εκπαιδεύονται ώστε η προσεγγιστική λύση τους να ικανοποιεί μια Διαφορική Εξίσωση (Differential Equation). Τον όρο PINNs εισήγαγαν οι Raissi, Perdikakis και Karniadakis με εργασία τους. [35]

Τα PINNs έχουν αποδειχθεί ένα εξαιρετικά αποτελεσματικό και χρήσιμο εργαλείο στην προσέγγιση λύσεων Συνήθων Διαφορικών Εξισώσεων (Ordinary Differential Equations - ODEs), Μερικών Διαφορικών Εξισώσεων (Partial Differential Equations - PDEs) και Κλασματικής Τάξης Διαφορικών Εξισώσεων (Fractional Differential Equations) καθώς και στην προσομοίωση μη σταθερών ρευστοδυναμικών μοντέλων με μειωμένο υπολογιστικό κόστος. [44]



Σχήμα 3.1: Physics-Informed Neural Networks

3.0α' Ορισμός Προβλήματος που Προσεγγίζουν τα PINNs

Η γενική μορφή μιας (μη γραμμικής) Μερικής Διαφορικής Εξίσωσης μπορεί να διατυπωθεί ως

$$(3.1) \quad \begin{cases} \mathcal{N}[u] = 0, & x \in \Omega, t \in [0, T], \\ u(x, 0) = f(x), & x \in \Omega, t = 0, \\ u(x, t) = g(x, t), & x \in \partial\Omega, t \in [0, T] \end{cases}$$

Το σύνολο $\Omega \subset \mathbb{R}^N$ είναι ένα ανοιχτό και φραγμένο σύνολο, $\partial\Omega$ το σύνορό του συνόλου και $T \in \mathbb{R}^+$ το μέγιστο του χρόνου.

Ο τελεστής $\mathcal{N}[\cdot]$ είναι ένας (μη γραμμικός) μερικός διαφορικός τελεστής και οι συναρτήσεις f και g αποτελούν τις δοθείσες Αρχικές Συνθήκες (Initial Conditions) και Συνοριακές Συνθήκες (Boundary Conditions) αντίστοιχα.

Στο πλαίσιο της παρούσας εργασίας μελετώνται Μερικές Διαφορικές Εξισώσεις, στις οποίες η άγνωστη συνάρτηση είναι της μορφής $u(x, t)$, με $x \in \Omega \subset \mathbb{R}^N$ και $t \in [0, T]$. Αναλόγως την εξίσωση που μελετάται, θεωρούνται Συνοριακές Συνθήκες Dirichlet (Dirichlet Boundary Conditions), Συνοριακές Συνθήκες Neumann (Neumann Boundary Conditions) και Περιοδικές Συνοριακές Συνθήκες (Periodic Boundary Conditions).

Σκοπός ενός PINN είναι η προσέγγιση της λύσης της εξίσωσης (3.1), με επαρκή ακρίβεια, λαμβάνοντας ως είσοδο τα ζεύγη (x, t) και έχοντας ως έξοδο το $\bar{u}_\theta(x, t)$. Η πραγματική λύση συμβολίζεται $u(x, t)$. [45], [35]

3.0β' Βασικές Διαφορές μεταξύ FNN και PINNs

Τα περισσότερα PINNs βασίζονται σε πλήρως συνδεδεμένα FNN, με πλήθος Κρυφών Επίπεδων και Τεχνητών Νευρώνων ανάλογο με την φύση της Διαφορικής Εξίσωσης.

Μία διαφορά των PINNs με τα FNN είναι στην διαδικασία εκπαίδευσης, όπου η φυσική πληροφορία της εξίσωσης ενσωματώνεται στη Loss Function.

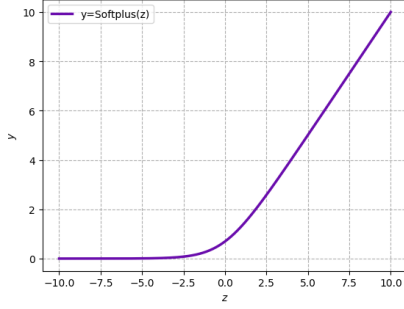
Μία ακόμα διαφορά εντοπίζεται στην Activation Function. Στα FNN αναφέραμε ότι η εφαρμογή της ReLU παρουσιάζει σημαντικά πλεονεκτήματα, ωστόσο στα PINNs η χρήση της είναι περιορισμένη, καθώς η επίλυση ΜΔΕ απαιτεί τον υπολογισμό παραγώγων υψηλής τάξης. Η ReLU, αν και είναι συνεχής παντού, δεν είναι παραγωγίσιμη στο 0.

Στα PINNs οι Activation Function που χρησιμοποιούνται απαιτείται να είναι ομαλές, δηλαδή $\sigma \in \mathcal{C}^\infty(\mathbb{R})$. Συχνή επιλογή, καθώς και σε αυτή την εργασία, είναι η Υπερβολική Εφαπτομένη, καθώς είναι συνεχής και απείρως παραγωγίσιμη με συνεχείς παραγώγους, δηλαδή $\tanh \in \mathcal{C}^\infty(\mathbb{R})$. [35]

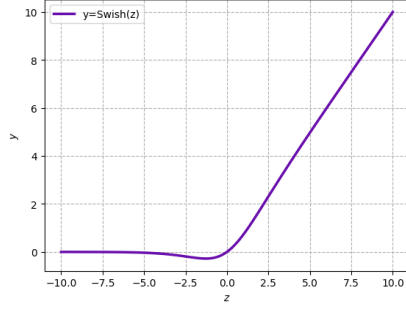
Πέραν της Υπερβολικής Εφαπτομένης, χρησιμοποιούνται και άλλες Activation Functions στα PINNs οι οποίες είναι ομαλές. Ενδεικτικά αναφέρονται η λογιστική συνάρτηση (sigmoid), η softplus, η οποία αποτελεί ομαλή προσέγγιση της ReLU, καθώς και η συνάρτηση Swish, οι οποίες ανήκουν στον χώρο $\mathcal{C}^\infty(\mathbb{R})$.

$$(3.2) \quad \text{Softplus}(z) = \ln(1 + e^{-z})$$

$$(3.3) \quad \text{Swish}(z) = z \text{ Sigmoid}(\beta z) = \frac{z}{1 + e^{-\beta z}}$$



(α') Softplus



(β') Swish

3.0γ' Συνάρτηση Απώλειας των PINNs

Έχουμε αναφέρει ότι η εκπαίδευση των FNN εξαρτάται από την ελαχιστοποίηση της Loss Function.

Στην περίπτωση των PINNs, η Loss Function για την προσέγγιση $\bar{u}_\theta(x, t)$ κατασκευάζεται ως το άθροισμα τριών επιμέρους όρων απώλειας. Η συνολική Loss Function για τα PINNs ορίζεται ως εξής

$$(3.4) \quad \mathcal{L} = \mathcal{L}_u + \mathcal{L}_b + \mathcal{L}_0$$

Με $\mathcal{L}_u$ ορίζεται η Loss Function της Διαφορικής Εξίσωσης. Με $\mathcal{L}_b$ ορίζεται η Loss Function των Συνοριακών Συνθηκών της Διαφορικής εξίσωσης. Με $\mathcal{L}_0$ ορίζεται η Loss Function της Αρχικής Συνθήκης της Διαφορικής Εξίσωσης. Με $\mathcal{L}$ ορίζεται η συνολική Loss Function του PINN.

Σε αντίθεση με τα κλασικά FNN που εκπαιδεύονται σε ζεύγη δεδομένων (x_i, y_i) , η εκπαίδευση των PINNs πραγματοποιείται σε ένα πεπερασμένο σύνολο σημείων στο εσωτερικό χωρίο $\Omega \times (0, T)$ (Collocation Points). [35]

Έστω $N_u \geq 1$ το πλήθος των σημείων εκπαίδευσης $\{(x_i^u, t_i^u)\}_{i=1}^{N_u} \subset \Omega \times (0, T]$. Η Loss Function της Διαφορικής Εξίσωσης ορίζεται ως εξής

$$(3.5) \quad \mathcal{L}_u = \frac{1}{2N_u} \sum_{i=1}^{N_u} \|\mathcal{N}[\bar{u}_\theta((x_i^u, t_i^u))]\|_2^2$$

Έστω $N_b \geq 1$ το πλήθος των σημείων εκπαίδευσης $\{(x_i^b, t_i^b)\}_{i=1}^{N_b} \subset \partial\Omega \times [0, T]$. Η Loss Function των Συνοριακών Συνθηκών της Διαφορικής Εξίσωσης ορίζεται ως εξής

$$(3.6) \quad \mathcal{L}_b = \frac{1}{2N_b} \sum_{i=1}^{N_b} \|\bar{u}_\theta(x_i^b, t_i^b) - g(x_i^b, t_i^b)\|_2^2$$

Τέλος, έστω $N_0 \geq 1$ το πλήθος των σημείων εκπαίδευσης $\{x_i^0\}_{i=1}^{N_0} \subset \Omega$. Η Loss Function των Αρχικών Συνθηκών της Διαφορικής Εξίσωσης ορίζεται ως εξής

$$(3.7) \quad \mathcal{L}_0 = \frac{1}{2N_0} \sum_{i=1}^{N_0} \|\bar{u}_\theta(x_i^0, 0) - f(x_i^0)\|_2^2$$

Οπότε η συνολική Loss Function ορίζεται:

$$(3.8) \quad \mathcal{L} = \underbrace{\frac{1}{2N_u} \sum_{i=1}^{N_u} \|\mathcal{N}[\bar{u}_\theta((x_i^u, t_i^u))]\|_2^2}_{\text{Self-Supervised (MΔE)}} + \underbrace{\frac{1}{2N_b} \sum_{i=1}^{N_b} \|\bar{u}_\theta(x_i^b, t_i^b) - g(x_i^b, t_i^b)\|_2^2}_{\text{Supervised (Σ.Σ.)}} + \underbrace{\frac{1}{2N_0} \sum_{i=1}^{N_0} \|\bar{u}_\theta(x_i^0, 0) - f(x_i^0)\|_2^2}_{\text{Supervised (Α.Σ.)}}$$

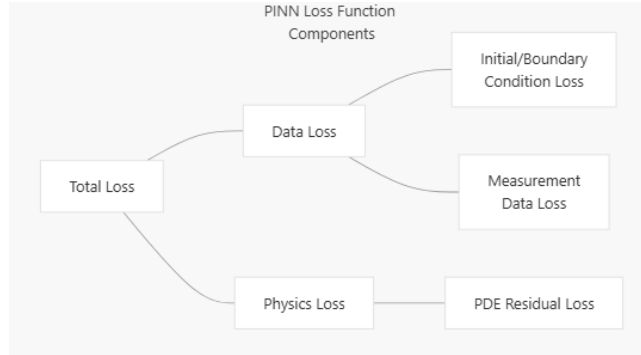
- **Supervised:** Το δίκτυο διαθέτει τις τιμές των Αρχικών Συνθηκών και Συνοριακών Συνθηκών και συγκρίνει την προσεγγιστική λύση του με αυτά τα γνωστά δεδομένα.
- **Self-supervised:** Η απώλεια υπολογίζεται μέσω του residual της διαφορικής εξίσωσης σε σημεία όπου δεν έχουμε δεδομένα. Το δίκτυο καθοδηγείται από την επαλήθευση της εξίσωσης, χωρίς γνωστά δεδομένα.

Όλοι οι όροι της Loss Function υπολογίζονται ως Μέσο Τετραγωνικό Σφάλμα (Mean Square Errors - MSE).

Η απώλεια $\mathcal{L}_u$ διασφαλίζει ότι η προσέγγιση $\bar{u}_\theta(x_i, t_i)$ συμμορφώνεται με την επαλήθευση της Διαφορικής Εξίσωσης.

Η απώλεια $\mathcal{L}_b$ διασφαλίζει ότι η προσέγγιση $\bar{u}_\theta(x_i, t_i)$ συμμορφώνεται με τις Συνοριακές Συνθήκες της Διαφορικής Εξίσωσης.

Η απώλεια $\mathcal{L}_0$ διασφαλίζει ότι η προσέγγιση $\bar{u}_\theta(x_i, t_i)$ συμμορφώνεται με τις Αρχικές Συνθήκες της Διαφορικής Εξίσωσης.



Σχήμα 3.3: Loss Function των PINNs

3.0δ' Κατηγορίες και Μοντελοποίηση Προβλημάτων

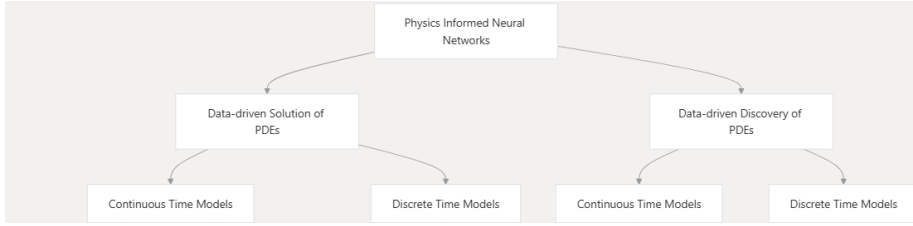
Το πλαίσιο των PINNs οργανώνεται γύρω από δύο βασικές κατηγορίες προβλημάτων και δύο κύριες προσεγγίσεις μοντελοποίησης. [34], [33], [35]

- **Ευθύ Πρόβλημα:** Στην πρώτη κατηγορία προβλημάτων, Data-Driven Solution of PDEs (επίλυση ΜΔΕ με βάση τα δεδομένα), η ΜΔΕ είναι γνωστή. Το Νευρωνικό Δίκτυο εκπαιδεύεται λαμβάνοντας ως είσοδο ζεύγη (x, t) , έτσι ώστε να ικανοποιεί τόσο τα διαθέσιμα δεδομένα, όσο και τους φυσικούς περιορισμούς της εξίσωσης.

- **Αντίστροφο Πρόβλημα:** Στην δεύτερη κατηγορία προβλημάτων, Data-Driven Discovery of PDEs (ανακάλυψη ΜΔΕ με βάση τα δεδομένα), η ΜΔΕ δεν είναι γνωστή. Το Νευρωνικό Δίκτυο, αξιοποιώντας δεδομένα από ένα φυσικό σύστημα, εκπαιδεύεται ώστε να ανακαλύψει μία προσεγγιστική μορφή της πραγματικής ΜΔΕ.

Η μοντελοποίηση των PINNs μπορεί να διαφοροποιηθεί ανάλογα με τη διαχείριση του χρόνου και του χώρου στο πεδίο λύσης.

- Στην πρώτη κατηγορία προσέγγισης της λύσης, Continuous Time Models (Συνεχή Χρονικά Μοντέλα), χρησιμοποιείται ένα μόνο Νευρωνικό Δίκτυο για την προσέγγιση της λύσης σε όλο το χωροχρονικό πεδίο.
- Στην δεύτερη κατηγορία προσέγγισης της λύσης, Discrete Time Models (Διακριτά Χρονικά Μοντέλα), η προσέγγιση βασίζεται σε κλασικές Αριθμητικές Μεθόδους, όπως οι μέθοδοι Runge-Kutta και Πεπερασμένες Διαφορές (Finite Differences), σε συνδυασμό με Νευρωνικά Δίκτυα.



Σχήμα 3.4: Κατηγορίες και Μοντελοποίηση Προβλημάτων

Στην παρούσα εργασία θα εστιάσουμε στην πρώτη κατηγορία προβλημάτων, επιλύοντας ΜΔΕ με βάση δεδομένα εκπαίδευσης, χρησιμοποιώντας την πρώτη κατηγορία προσέγγισης της λύσης, κατασκευάζοντας ένα μόνο Νευρωνικό Δίκτυο για την προσέγγιση της λύσης $u(x, t)$.

3.0ε' Συνεχή Χρονικά Μοντέλα (Continuous-Time Models)

Τα Continuous-Time Models στα PINNs ακολουθούν μια θεμελιώδης διαφορετική προσέγγιση για την επίλυση ΜΔΕ σε σύγκριση με τις παραδοσιακές Αριθμητικές Μεθόδους.

Αντί να διακριτοποιούν το χώρο και να προσεγγίζουν τις παραγώγους χρησιμοποιώντας Πεπερασμένες Διαφορές (Finite Differences) ή άλλες Αριθμητικές Μεθόδους, θεωρούν ότι η ίδια η λύση παριστάνεται από ένα Νευρωνικό Δίκτυο, του οποίου οι παράμετροι ρυθμίζονται κατάλληλα.

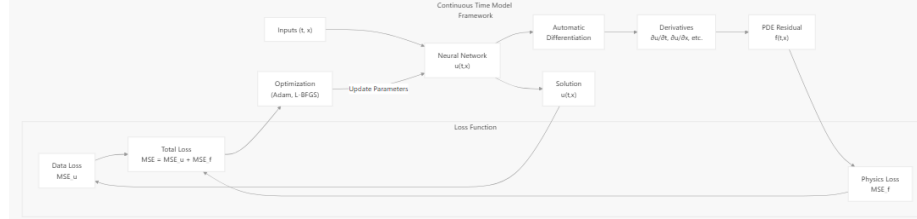
Οι παράγωγοι που εμφανίζονται στη Διαφορική Εξίσωση προκύπτουν άμεσα από το Νευρωνικό Δίκτυο μέσω Αυτόματης Παραγωγίσης, χωρίς την ανάγκη αριθμητικών προσεγγίσεων. [34], [33], [35]

Τα Continuous-Time Models είναι δομημένα έτσι ώστε να προσεγγίζουν ΜΔΕ της μορφής

$$(3.9) \quad u_t + \mathcal{N}[u] = 0, x \in \Omega, t \in [0, T]$$

Ο $\mathcal{N}$ είναι ένας (μη γραμμικός) τελεστής και περιλαμβάνει μερικούς παραγώγους της λύσης $u(x, t)$ ως προς τη χωρική μεταβλητή, όπως $u_x, u_{xx}, \dots$, καθώς και μερικούς παραγώγους ως προς τη χωρική και τη χρονική μεταβλητή, όπως $u_{tx}, u_{xxt}, \dots$.

Στο Σχήμα (3.5) παρουσιάζεται το πλαίσιο λειτουργίας των Continuous-Time Models.



Σχήμα 3.5: Πλαίσιο CTM

Ακολουθεί συνοπτική παρουσίαση των κύριων πλεονεκτημάτων και περιορισμών που σχετίζονται με την εφαρμογή των Continuous-Time Models [34], [33], [35]

- Πλεονεκτήματα χρήσης Continuous-Time Models
 - Παρέχουν συνεχείς και παραγωγίσιμες λύσεις ως προς τον χώρο και τον χρόνο.
 - Δεν απαιτούν την χρήση Αριθμητικού Πλέγματος (Mesh-Free).
 - Μπορούν να διαχειριστούν διάφορους τύπους Συνοριακών Συνθηκών και πολύπλοκες γεωμετρίες.
 - Μπορούν να διαχειριστούν κακώς τοποθετημένα προβλήματα.
- Περιορισμοί χρήσης Continuous-Time Models
 - Απαιτούν μεγάλο πλήθος σημείων εκπαίδευσης (Collocation Points) για την επιβολή των φυσικών περιορισμών της Διαφορικής Εξίσωσης στο σύνολο του πεδίου, κάνοντας τον υπολογισμό της προσέγγισης της λύσης να έχει μεγάλο υπολογιστικό κόστος για προβλήματα υψηλότερης διάστασης.

Οι FEM υπερτερούν σε χαμηλές διαστάσεις όσον αφορά την ακρίβεια και την αποδοτικότητα. Ωστόσο, στα προβλήματα υψηλής διάστασης το υπολογιστικό κόστος των FEM αυξάνεται εκθετικά, καθιστώντας τα PINNs πιο προτιμητέα, λόγω της ικανότητάς τους να προσεγγίζουν λύσεις χωρίς εκτεταμένη διακριτοποίηση πλέγματος. [46]

3.07' Αυτόματη Παραγωγή (Automatic Differentiation)

Για την ΜΔΕ (3.1), το PINN υπολογίζει την προσέγγιση $\bar{u}_\theta(x, t)$. Στη συνέχεια υπολογίζεται η Loss Function στο εσωτερικό του χωροχρόνου μέσω της (3.5). Η ποσότητα $f(x, t)$ καλείται στη βιβλιογραφία Residual (Υπόλειμμα) και συχνά συμβολίζεται ως

$$(3.10) \quad f(x, t) := \mathcal{N}[\bar{u}_\theta(x, t)]$$

Το Residual μετρά την απόκλιση της προσέγγισης $\bar{u}_\theta(x, t)$ από την ακριβή ικανοποίηση της ΜΔΕ σε κάθε σημείο στο εσωτερικό του χωροχρόνου της ΜΔΕ, δηλαδή για $x \in \Omega$ και $t \in [0, T]$. [35]

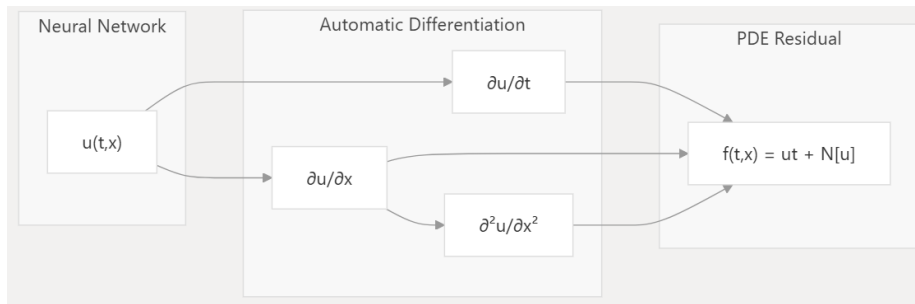
Η Αυτόματη Παραγωγή (Automatic Differentiation - AD) αποτελεί βασικό εργαλείο στα PINNs, καθώς επιτρέπει τον ακριβή υπολογισμό των παραγώγων που εμφανίζονται στις ΜΔΕ, χωρίς να απαιτείται πλέγμα ή γειτονικά σημεία.

Αυτό είναι δυνατό διότι η προσέγγιση της λύσης $\bar{u}_\theta(x, t)$ είναι μια ομαλή συνάρτηση, κατασκευασμένη από γραμμικούς συνδυασμούς και ομαλών συναρτήσεων ενεργοποίησης. Οι παράγωγοι της εξόδου του Νευρωνικού Δικτύου ως προς τις εισόδους του μπορούν να υπολογιστούν αναλυτικά μέσω του κανόνα της αλυσίδας, ακολουθώντας την αλυσίδα των υπολογιστικών πράξεων που οδήγησαν από την είσοδο (x, t) στην έξοδο $\bar{u}_\theta(x, t)$. [35], [47]

Με αυτόν τον τρόπο, το AD επιτρέπει τον υπολογισμό παραγώγων οποιασδήποτε τάξης που απαιτούνται από τη Διαφορική Εξίσωση, χωρίς την ανάγκη χειροκίνητης παραγωγής ή υλοποίησης των αντίστοιχων τύπων. Αυτό το καθιστά πιο ακριβές σε σχέση με τις κλασικές μεθόδους Πεπερασμένων Διαφορών (Finite Differences - FD). [48]

Η παράγωγος της προσεγγιστικής λύσης δείχνει πώς αλλάζει η έξοδος όταν αλλάζει λίγο η είσοδος

- Η παράγωγος u_x δείχνει πόσο αλλάζει το $\bar{u}_\theta$, όταν αλλάξει λίγο το x .
- Η παράγωγος u_t δείχνει πόσο αλλάζει το $\bar{u}_\theta$, όταν αλλάξει λίγο το t
- Η παράγωγος u_{xx} δείχνει πόσο αλλάζει η αλλαγή του $\bar{u}_\theta$, όταν αλλάξει λίγο το x
- Και ούτω καθεξής



Σχήμα 3.6: Πλαίσιο AD για τον υπολογισμό της 3.10

Αφού υπολογιστούν οι παράγωγοι και το Residual, ορίζεται η συνολική Loss Function 3.4, η οποία καθοδηγεί την εκπαίδευση του PINN. Κατά την εκπαίδευση, οι παράμετροι του δικτύου προσαρμόζονται ώστε το Residual καθώς και η συνολική Loss Function να ελαχιστοποιηθούν.

Μετά την εκπαίδευση, το PINN μπορεί να παράγει νέες τιμές σε οποιοδήποτε σημείο (x, t) , ακόμα και εκεί που δεν υπάρχουν δεδομένα εκπαίδευσης. Αυτό συμβαίνει επειδή το PINN έχει μάθει μια συνεχόμενη προσεγγιστική λύση $\bar{u}_\theta(x, t)$ της ΜΔΕ, η οποία ικανοποιεί τόσο τα δεδομένα όσο και τους νόμους της φυσικής που επιβάλλει η Διαφορική Εξίσωση.

Έτσι, το PINN μπορεί να συμπληρώσει κενά, να προβλέψει καταστάσεις ή να δημιουργήσει συνεχές πεδίο λύσεων, βασισμένο στη γνώση που απέκτησε από τα δεδομένα εκπαίδευσης και τους νόμους της φυσικής της Διαφορικής Εξίσωσης.

3.0ζ' Συναρτήσεις Βελτιστοποίησης στα PINNs (Adam και L-BFGS)

Όπως είχαμε αναφέρει στα FNN, η ενημέρωση των παραμέτρων θ γίνεται μέσω Αλγορίθμων Βελτιστοποίησης, όπως το Gradient Descent (Μέθοδος Καθόδου Κλίσης).

Στα PINNs η μέθοδος αυτή παρουσιάζει αργή σύγκλιση, ιδιαίτερα σε προβλήματα με έντονη δυσκαμψία στις κλίσεις της Loss Function (Stiff Loss Landscape).

Μία συχνά χρησιμοποιούμενη Μέθοδος Βελτιστοποίησης είναι η Adam (Adaptive Moments = Προσαρμοστικές Ροπές), η οποία προσαρμόζει τα βήματα ενημέρωσης για κάθε παράμετρο αξιοποιώντας εκθετικά κινούμενους μέσους όρους των βαθμίδων και των τετραγώνων τους., βελτιώνοντας τη σταθερότητα και την ταχύτητα σύγκλισης.

Η μέθοδος L-BFGS (Limited-memory Broyden–Fletcher–Goldfarb–Shanno), μια quasi-Newton μέθοδος, που αξιοποιεί προσεγγίσεις της Hessian για ταχύτερη σύγκλιση. Μπορεί να χρησιμοποιηθεί ως τελική φάση εκπαίδευσης, μετά από αρχική εκπαίδευση με την Adam, για την επίτευξη υψηλής ακρίβειας. Ο συνδυασμός αυτός επιτρέπει την αποφυγή τοπικών ελαχίστων και την ταχύτερη σύγκλιση σε προβλήματα με έντονες διαφορές κλίσεων. [49] [42] [35] [47] [50]

3.0η' Εκπαίδευση των PINNs

Η διαδικασία εκπαίδευσης ενός PINN μπορεί να περιγραφεί μέσω των ακόλουθων βημάτων, η οποία συνδυάζει τεχνικές Νευρωνικών Δικτύων σε συνδυασμό την επιβολή των φυσικών νόμων μέσω της Διαφορικής Εξίσωσης [35]

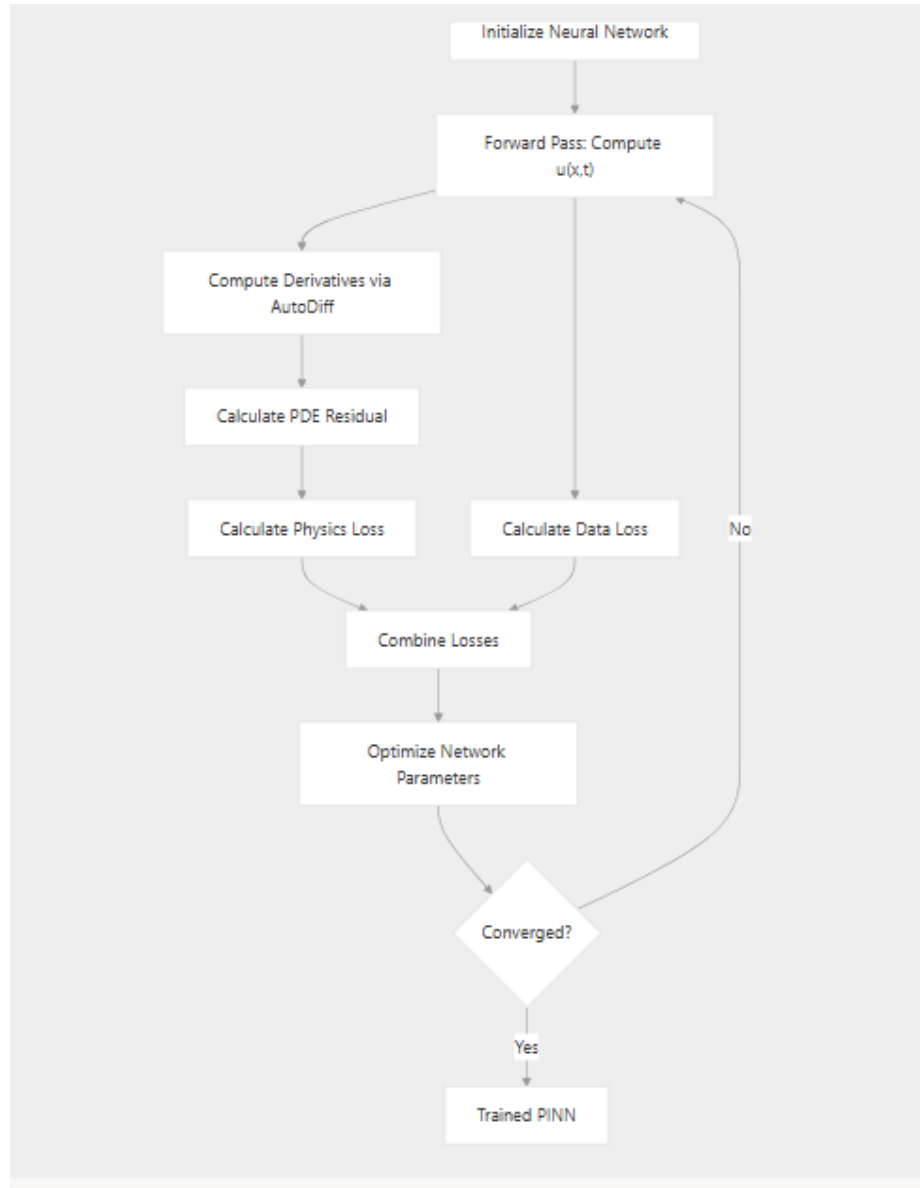
- 1. Επιλογή των σημείων για την εκπαίδευση (Collocation Points)
Γίνεται επιλογή σημείων στο εσωτερικό του χωρίου για την Διαφορική εξίσωση, στα σύνορα για τις Συνοριακές Συνθήκες και για αρχικό χρόνο για Αρχικές Συνθήκες. Τα σημεία αυτά αποτελούν το σύνολο εκπαίδευσης.
- 2. Εμπρόσθια Διάδοση (Forward Propagation)
Για κάθε σημείο του συνόλου εκπαίδευσης, το PINN υπολογίζει την προσεγγιστική λύση $\bar{u}_\theta(x, t)$, εφαρμόζοντας διαδοχικά τα τις παραμέτρους $\theta = [W, b]$ και τις μη γραμμικές συναρτήσεις ενεργοποίησης $\sigma(\cdot)$.
- 3. Υπολογισμός Παραγώγων μέσω Αυτόματης Διαφοροποίησης (Automatic Differentiation)
Με την χρήση της Αυτόματης Διαφοροποίησης υπολογίζονται οι απαιτούμενες παράγωγοι της προσέγγισης $\bar{u}_\theta(x, t)$, όπως $\partial_x \bar{u}_\theta$, $\partial_{xx} \bar{u}_\theta$, $\partial_{xxx} \bar{u}_\theta$, $\partial_t \bar{u}_\theta$, $\partial_{tt} \bar{u}_\theta$, $\partial_{xt} \bar{u}_\theta$ και ούτω καθεξής.

- 4. Υπολογισμός της Απόκλισης της Προσέγγισης $\bar{u}_\theta(x, t)$ (Residual)
Οι παράγωγοι εισάγονται στην Διαφορική εξίσωση (3.1) ώστε να υπολογιστεί το (3.10), το οποίο πρέπει να μηδενίζεται στο εσωτερικό του χωρίου.

- 5. Υπολογισμός Συναρτήσεων Απώλειας (Loss Function)
Υπολογίζονται οι όροι της συνολικής απώλειας, της Loss Function της Μερικής Διαφορικής Εξίσωσης $\mathcal{L}_u$ (3.5), της Συνάρτησης Απώλειας της Αρχικής Συνθήκης $\mathcal{L}_0$ (3.7) καθώς και η Συνάρτηση Απώλειας των Συνοριακών Συνθηκών $\mathcal{L}_b$ (3.6).

Οι όροι αυτοί, συχνά, συνδυάζονται με βάρη, ℓ_u, ℓ_0 και ℓ_b βοηθώντας την σύγκλιση.

- 6. Εφαρμογή Μεθόδων Βελτιστοποίησης (Optimization Methods)
Η ελαχιστοποίηση της συνολικής Loss Function πραγματοποιείται με (Gradient-Based Methods), όπως η Adam και η L-BFGS.
- 7. Οπισθοδιάδοση Σφάλματος (Back Propagation)
Αν η συνολική Loss Function δεν έχει φτάσει σε επαρκές ελάχιστο, εφαρμόζεται η Back Propagation για την ενημέρωση των παραμέτρων θ .
- 8. Έλεγχος Σύγκλισης
Τα παραπάνω βήματα επαναλαμβάνονται μέχρι το ελάχιστο της συνολικής Loss Function να γίνει αποδεκτά μικρό.



Σχήμα 3.7: Βήματα Προσέγγισης της Λύσης από τα PINNs

Η σύγκλιση της εκπαίδευσης που περιγράφηκε δεν είναι πάντοτε εξασφαλισμένη. Ένα από τα βασικά προβλήματα που εμφανίζονται στην πράξη είναι η Stiff Loss Landscape (Έντονη Δυσκαμψία στις Κλίσεις την Συνάρτησης Απώλειας).

Για να το αναλύσουμε θα χρειαστούμε την έννοια του Εσσιανού Πίνακα

Ορισμός: Εσσιανός Πίνακας (Hessian Matrix)

Έστω μια συνάρτηση $f : \mathbb{R}^n \rightarrow \mathbb{R}$, διαφορίσιμη δύο φορές. Ο Εσσιανός Πίνακας της f για $x \in \mathbb{R}^n$ είναι ο τετραγωνικός πίνακας όλων των δεύτερων μερικών παραγώγων:

$$(3.11) \quad H_f(x) = \left[\frac{\partial^2 f(x)}{\partial x_i \partial x_j} \right]_{i,j=1}^n$$

Μέσω του Εσσσιανού Πίνακα μπορούμε να ποσοτικοποιήσουμε τις Stiff Loss Landscape της Loss Function.

Ορίζουμε λ_{max} και λ_{min} είναι η μέγιστη και η ελάχιστη ιδιοτιμή του Εσσσιανού Πίνακα. Επίσης, ορίζουμε το πηλίκο

$$(3.12) \quad \kappa(H) = \frac{\lambda_{max}}{\lambda_{min}}$$

- Αν $\lambda_{max} \approx \lambda_{min} \approx 0$, τότε οι παράγωγοι είναι σχεδόν μηδέν (Vanishing Gradient). Η ενημέρωση των παραμέτρων είναι αργή ή και αδύνατη.
- Αν $\lambda_{max} \gg \lambda_{min}$, δηλαδή $\kappa(H) \gg 1$, τότε οι παράγωγοι έχουν μεγάλες τιμές (Exploding Gradients), υποδεικνύοντας έντονες διαφορές στην κλίση κατά μήκος διαφορετικών κατευθύνσεων, με αποτέλεσμα ορισμένοι όροι της Loss Function (3.4) να υπερσχύουν των άλλων κατά την διάρκεια της εκπαίδευσης, κάνοντας την σύγκλιση με τη μέθοδο Gradient Descent δύσκολη.
- Αν $\kappa(H) \approx 1$, τότε οι ιδιοτιμές είναι συγκρίσιμες, άρα οι κλίσεις είναι ομαλές και έχουμε σταθερή σύγκλιση.

Για την αντιμετώπιση των προβλημάτων χρησιμοποιούνται βάρη ℓ_u, ℓ_0 και ℓ_b που ισορροπούν τους όρους $\mathcal{L}_u$, $\mathcal{L}_b$ και $\mathcal{L}_0$.

Η κατάλληλη επιλογή αυτών των βαρών είναι κρίσιμη για τη σύγκλιση του δικτύου και την ορθή αναπαράσταση των φυσικών περιορισμών.

Έχουν αναπτυχθεί συγκεκριμένοι μέθοδοι υπολογισμού βέλτιστων βαρών για τους όρους της συνολικής Loss Function, όπως η NTK-Based Adaptive Loss Balancing, όμως στην παρούσα εργασία δεν θα τους αναφέρουμε. [51]

3.00' Αρχιτεκτονική των PINNS

Τα PINNs, στην πράξη, συνήθως βασίζονται σε πλήρως συνδεδεμένα FNN με 4-8 Κρυφά Επίπεδα και 20-50 Τεχνητούς Νευρώνες ανά επίπεδο. [35]

Η επιλογή ρηχών δικτύων οφείλεται σε διάφορους λόγους. Τα Βαθιά Πολυεπίπεδα Νευρωνικά Δίκτυα (Deep Multi-Layer Perceptrons) παρουσιάζουν δυσκολίες κατά την εκπαίδευση, οι οποίες σχετίζονται με αριθμητική αστάθεια, επιδείνωση την κλίσεων των παραγώγων υψηλής τάξης που υπολογίζονται μέσω της Αυτόματης Διαφοροποίησης, που με την σειρά τους ενισχύουν τις Stiff Loss Landscape της Loss Function. [49] [42] [35] [47]

3.0ι' Σύγχρονες Βελτιώσεις των PINNs

Τα κλασικά PINNs αποτελούν τη βασική μεθοδολογία, ωστόσο η πρόσφατη βιβλιογραφία έχει προτείνει πλήθος επεκτάσεων και παραλλαγών που στοχεύουν στη βελτίωση της ακρίβειας, της σταθερότητας και της αποδοτικότητας της εκπαίδευσης. Ενδεικτικά αναφέρονται οι ακόλουθες κατηγορίες:

- 1. XPINNs (Extended PINNs – Domain Decomposition) [52]

- 2. VPINNs (Variational PINNs) [53]
- 3. Adaptive Loss Weighting [51] [54]
- 4. Curriculum PINNs [55]
- 5. Fourier Neural Operators [51] [56]
- 6. Deep Operator Networks (DeepONets) [47]
- 7. Random Operator Networks (RandONets) [57]

Κεφάλαιο 4

Υπολογιστικό Περιβάλλον Εκτέλεσης των PINNs

Η εκπαίδευση των PINNs στην παρούσα εργασία πραγματοποιήθηκε τόσο σε CPU όσο και σε GPU, προκειμένου να αξιολογηθούν οι διαφορές στην απόδοση, την ταχύτητα εκπαίδευσης και στην ακρίβεια των αποτελεσμάτων.

Για την εκπαίδευση σε CPU, η διαδικασία υλοποιήθηκε με χρήση της βιβλιοθήκης PyTorch στο περιβάλλον Google Colab, σε λειτουργικό σύστημα Linux (64-bit), αρχιτεκτονικής x86_64.

Για την εκπαίδευση σε GPU, η διαδικασία υλοποιήθηκε με χρήση της βιβλιοθήκης PyTorch με υποστήριξη CUDA (2.8.0+cu129), σε λειτουργικό σύστημα Windows (64-bit), αξιοποιώντας την κάρτα γραφικών NVIDIA GeForce RTX 3060.

Κατά την εκτέλεση σε GPU εμφανίστηκαν προειδοποιητικά μηνύματα που σχετίζονται με την αρχικοποίηση του CUDA context και τον μηχανισμό αυτόματης διαφοροποίησης, τα οποία δεν επηρέασαν την ορθότητα ή τα αποτελέσματα της εκπαίδευσης.

```
1 1. UserWarning: Attempting to run cuBLAS, but there was no current CUDA
context! Attempting to set the primary context... (Triggered internally at C
:\actions-runner\work\pytorch\pytorch\pytorch\aten\src\ATen\cuda\
CublasHandlePool.cpp:179.)
```

Listing 4.1: Προειδοποιητικό μήνυμα CUDA κατά την εκτέλεση στη GPU

```
1 2. return Variable._execution_engine.run_backward( # Calls into the C++
engine to run the backward pass
```

Listing 4.2: Κλήση backward pass στο PyTorch engine

Αν και η CUDA επιτρέπει την εκμετάλλευση της μαζικής παραλληλίας της GPU, η χρήση της δεν εγγυάται πάντοτε μικρότερους χρόνους εκτέλεσης σε σύγκριση με την CPU. Για προβλήματα μικρού μεγέθους ή με περιορισμένο βαθμό παραλληλοποίησης, όπως PINNs με σχετικά μικρό αριθμό σημείων εκπαίδευσης ή βελτιστοποιητές τύπου L-BFGS, το υπολογιστικό κόστος μεταφοράς δεδομένων και συγχρονισμού μπορεί να εξουδετερώσει τα πλεονεκτήματα της GPU. Επιπλέον,

η CPU ενδέχεται να παρουσιάζει καλύτερη αριθμητική ακρίβεια, οδηγώντας σε μικρότερη τελική απόκλιση της Loss Function, με αντάλλαγμα μεγαλύτερο χρόνο εκπαίδευσης. [58]

Η διαφορά στην τελική ακρίβεια μεταξύ CPU και GPU μπορεί να αποδοθεί σε αριθμητικά φαινόμενα κινητής υποδιαστολής. Η παράλληλη φύση των υπολογισμών στη GPU, η διαφορετική σειρά εκτέλεσης των πράξεων και η εκτεταμένη χρήση Fused Multiply-Add (FMA) πράξεων οδηγούν σε διαφορετική συσσώρευση αριθμητικών σφαλμάτων σε σχέση με τη σειριακή εκτέλεση στην CPU, γεγονός που έχει τεκμηριωθεί εκτενώς στη βιβλιογραφία. [59]

Παρακάτω παρουσιάζονται τα αποτελέσματα των PINNs που υλοποιήθηκαν για την παρούσα εργασία και εκτελέστηκαν στο αναφερόμενο υπολογιστικό περιβάλλον, επιτρέποντας τη σύγκριση της απόδοσης των αλγορίθμων σε CPU και GPU μέσω CUDA, καθώς και την αξιολόγηση της ακρίβειας και των χρόνων εκπαίδευσης.

Επιπλέον, για την σύγκριση των προσεγγιστικών λύσεων των PINNs με κλασικές Αριθμητικές Μεθόδους, επιλέχθηκε η Runge Kutta 4ης τάξης (RK4), λόγω της υψηλής τάξης ακρίβειάς της, η οποία υλοποιήθηκε σε MATLAB (έκδοση R2025b).

Κεφάλαιο 5

Εξίσωση Κύματος

Όπως αναφέραμε στην παρούσα εργασία, μελετήθηκαν μοντέλα συνεχούς χρόνου στα Continuous-Time Models (3.9). Σε αυτό το κεφάλαιο εστιάζουμε σε CTM τα οποία είναι δεύτερης τάξης ως προς τον χρόνο και συγκεκριμένα ορίζουμε την (γραμμική) Μερική Διαφορική Εξίσωση Κύματος (Wave Equation) που ορίζεται ως εξής

$$(5.1) \quad u_{tt} - c^2 u_{xx} = 0, x \in \Omega, t \in [0, T].$$

Το σύνολο $\Omega \subset \mathbb{R}^N$ είναι ένα ανοιχτό και φραγμένο σύνολο, $\partial\Omega$ το σύνορό του συνόλου και $T \in \mathbb{R}^+$ το μέγιστο του χρόνου.

Σκοπός είναι να προσεγγίσουμε την πραγματική λύση $u(x, t)$ με ακριβώς μικρή απώλεια μέσω Physics-Informed Neural Networks.

Η Μερική Διαφορική Εξίσωση του Κύματος είναι η σημαντικότερη εξίσωση κυματικής διάδοσης και περιγράφει μια πληθώρα κυμάτων όπως ηχητικά κύματα, κύματα φωτός και κύματα στο νερό. Η αναλυτική μορφή (5.1) καθιερώθηκε το 1747 από τον Jean Le Rond d'Alembert (Τζιν Λεροντ Νταλαμπέρ 1717 - 1783). [60]

5.1 Περιοδικές Συνοριακές Συνθήκες

Στην Μερική Διαφορική Εξίσωση (5.1) εφαρμόζουμε δύο Αρχικές Συνθήκες (Initial Conditions), δύο Περιοδικές Συνοριακές Συνθήκες (Periodic Boundary Conditions) και ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN.

$$(5.2) \quad \begin{cases} u_{tt} - c^2 u_{xx} = 0, & x \in [-L, L], t \in [0, T], \\ u(x, 0) = A \sin\left(\frac{\pi}{L}x\right), & x \in [-L, L], \\ u_t(x, 0) = -A \frac{\pi}{L} c \cos\left(\frac{\pi}{L}x\right), & x \in [-L, L], \\ u(-L, t) = u(L, t), & t \in [0, T] \\ u_x(-L, t) = u_x(L, t), & t \in [0, T] \end{cases}$$

Για την (5.2) ορίσαμε τις μεταβλητές ως εξής

$$(5.3) \quad c = 1, L = 2, T = 5, A = 1$$

Για την προσέγγιση της λύσης της (5.2) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [61]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

5.1α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN έχει ως εξής

$$(5.4) \quad \begin{aligned} \text{Αριθμός Κρυφών Επιπέδων} &= 8, \\ \text{Αριθμός Νευρώνων ανά Επίπεδο} &= 20, \\ \text{Αριθμός Εποχών Εκπαίδευσης} &= 2000 \end{aligned}$$

Με τον όρο Εποχές (Epochs) αναφερόμαστε στο πλήθος των πλήρων επαναλήψεων που εκτελεί το PINN πάνω στο σύνολο των δεδομένων εκπαίδευσης. Αντί να έχουμε μια ατέρμονη επανάληψη της εκπαίδευσης, με κριτήριο τερματισμού μια επαρκή χαμηλή τιμή της συνολικής Loss Function, ο αριθμός των Εποχών καθορίζει a priori το μέγιστο πλήθος πλήρων επαναλήψεων εκπαίδευσης.

$$(5.5) \quad \text{Ρυθμός Μάθησης} = \begin{cases} 5 \times 10^{-4}, & \text{Εποχές} < 100, \\ 1 \times 10^{-4}, & 100 \leq \text{Εποχές} < 3000 \end{cases}$$

$$(5.6) \quad \text{Τα βάρη της ολικής Συνάρτησης Απώλειας} \begin{cases} w_u = 1, \\ w_b = 1, \\ w_0 = 1 \end{cases}$$

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.2) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.1

N_0	N_b	N_u	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
400	500	2000	5.09918761e-03	1.63952900e-05	1:57	0:30
300	500	2000	5.01715858e-03	1.29059854e-05	1:54	0:29
200	500	2000	8.33251514e-03	5.33379847e-04	1:54	0:30
100	500	2000	3.98602849e-03	3.03579855e-05	1:59	0:32
50	500	2000	3.97282047e-03	1.51251561e-05	1:55	0:30
400	400	2000	5.55277243e-03	1.64713783e-05	1:55	0:31
400	300	2000	8.23759474e-03	1.07215741e-03	1:54	0:30
400	200	2000	4.04443452e-03	1.37142652e-05	1:52	0:30
400	100	2000	5.26801404e-03	2.31999675e-05	1:51	0:29
400	50	2000	1.10043446e-03	7.41373788e-06	1:56	0:29
400	10	2000	4.90799220e-03	1.41180935e-05	1:53	0:31
400	500	1800	5.90554066e-03	1.82133863e-05	1:53	0:31
400	500	1600	3.65602691e-03	2.32615112e-05	1:47	0:28
400	500	1400	3.74418194e-03	3.02120134e-05	1:43	0:26
400	500	1200	4.28399304e-03	1.42207182e-05	1:36	0:25
400	500	1000	5.09349257e-03	1.27296889e-05	1:28	0:23
400	500	800	3.21554672e-03	9.27908786e-06	1:22	0:22
400	500	600	8.65831692e-03	8.59940192e-05	1:16	0:21
400	500	400	5.64896688e-03	5.48921416e-05	1:12	0:18
400	500	200	6.48668036e-03	1.55511952e-04	1:05	0:17
300	400	1500	3.16458032e-03	3.34631914e-05	1:42	0:27
300	400	1000	3.03304661e-03	2.70059245e-05	1:28	0:22
300	400	500	3.99246160e-03	6.32159717e-05	1:12	0:18
200	300	1500	4.15475946e-03	1.00968309e-05	1:38	0:26
200	300	1000	7.79440394e-03	8.48683794e-06	1:23	0:22
200	300	500	4.34123725e-03	1.14836694e-05	1:09	0:18
100	200	1500	4.78578173e-03	1.85736626e-05	1:38	0:26
100	200	1000	4.38405480e-03	1.35113405e-05	1:21	0:21
100	200	500	5.52610029e-03	1.98388196e-04	1:08	0:17
50	100	1500	5.80113754e-03	3.86848442e-05	1:37	0:25
50	100	1000	8.15270841e-03	1.13264077e-05	1:18	0:22
50	100	500	5.55884466e-03	8.39453878e-06	1:05	0:18

Πίνακας 5.1: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε CPU

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Από τον Πίνακα 5.1 μπορούν να εξαχθούν τα εξής συμπεράσματα

Τα αποτελέσματα της συνολικής Loss Function μέσω της Μεθόδου βελτιστοποίησης Adam παρατηρούμε ότι παραμένουν της ίδιας τάξης μεγέθους, 10^{-3} , για όλες τις εξεταζόμενες επιλογές σημείων εκπαίδευσης. Όμως, παρατηρούνται διακυμάνσεις που υποδηλώνουν ότι η σύγκλιση της Adam επηρεάζεται από τη σύνθεση του συνόλου εκπαίδευσης και όχι μόνο από το συνολικό πλήθος των σημείων.

Ως προς τον υπολογιστικό χρόνο της Adam, προκύπτει ότι η αύξηση του πλήθους των σημείων εκπαίδευσης οδηγεί γενικά σε μεγαλύτερους χρόνους εκπαίδευσης, με τον αριθμό των collocation points N_u να αποτελεί τον κυρίαρχο παράγοντα κόστους.

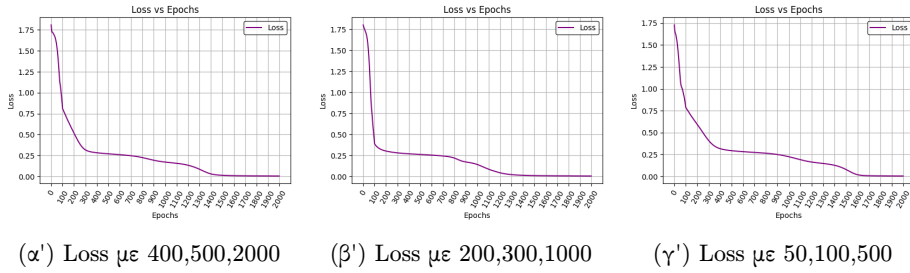
Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους που κυμαίνεται από 10^{-4} έως 10^{-5} .

Ο συνολικός χρόνος εκπαίδευσης της μεθόδου L-BFGS κυμαίνεται εντός ενός σχετικά στενού χρονικού διαστήματος, γεγονός που υποδηλώνει ότι επηρεάζεται σε μικρότερο βαθμό από το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Τα αποτελέσματα δείχνουν ότι η αύξηση του πλήθους των σημείων εκπαίδευσης δεν οδηγεί κατ' ανάγκη σε μείωση της συνολικής Loss Function, ενώ ταυτόχρονα μπορεί να αυξήσουν το υπολογιστικό κόστος σημαντικά, για το συγκεκριμένο πρόβλημα.

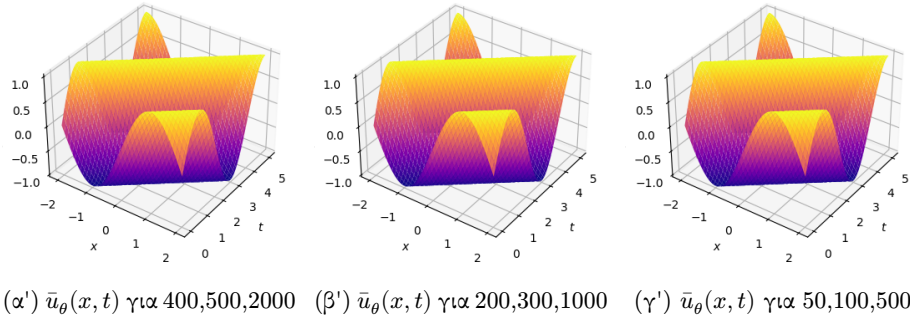
Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

Με βάση τα παραπάνω, παίρνουμε τα γραφήματα 5.1, 5.2, 5.3 και 5.4:

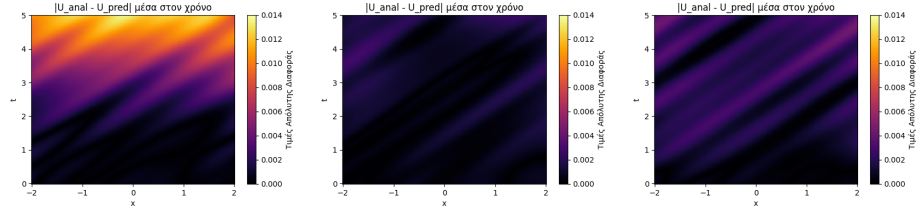


Σχήμα 5.1: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (5.1) παρατηρούμε ότι, για τις διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης, η εξέλιξη της συνολικής συνάρτησης απώλειας ως προς τις εποχές παρουσιάζει παρόμοια συμπεριφορά σύγκλισης. Το γεγονός αυτό υποδηλώνει ότι το δίκτυο μπορεί να εκπαιδευτεί αποτελεσματικά χωρίς την ανάγκη πολύ μεγάλου αριθμού σημείων, διατηρώντας την ακρίβεια της προσέγγισης, όπως φαίνεται και στο Σχήμα (5.2).



Σχήμα 5.2: Γραφήματα της Προσεγγιστικής Λύσης με PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

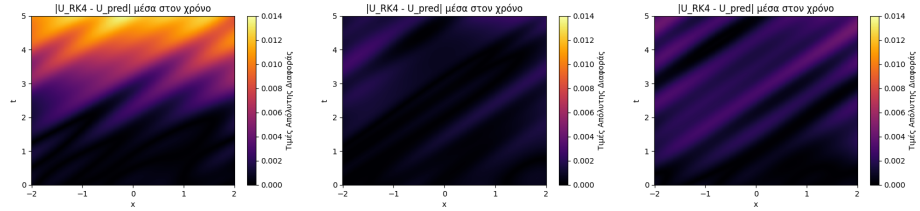


(α') $\max|Αναλυτική - PINN|$ για 400,500,2000 (β') $\max|Αναλυτική - PINN|$ για 200,300,1000 (γ') $\max|Αναλυτική - PINN|$ για 50,100,500

Σχήμα 5.3: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (5.3) παρατηρούμε ότι, ακόμη και για σχετικά μικρό αριθμό σημείων εκπαίδευσης, η προσεγγιστική λύση ακολουθεί ικανοποιητικά τη συμπεριφορά της αναλυτικής λύσης σε όλο τον εξεταζόμενο χωροχρόνο, με τις αποκλίσεις να παραμένουν αόρατες οπτικά. Ένα αποτέλεσμα το οποίο δεν ήταν εκ των προτέρων αναμενόμενο είναι ότι η μείωση του πλήθους των σημείων εκπαίδευσης, για το συγκεκριμένο παράδειγμα, οδηγεί σε βελτιωμένη προσέγγιση της πραγματικής λύσης, όπως προκύπτει από τη σύγκριση της πρώτης με τη δεύτερη εικόνα. Το φαινόμενο αυτό δεν παρατηρήθηκε σε άλλα παραδείγματα εφαρμογής των PINNs που εξετάστηκαν στην παρούσα εργασία.

Το αποτέλεσμα αυτό ενδέχεται να υποδηλώνει ότι, για το συγκεκριμένο πρόβλημα, η αύξηση των σημείων εκπαίδευσης δεν συνεπάγεται κατ' ανάγκη βελτίωση της γενίκευσης, αλλά μπορεί να επηρεάζει τη δυναμική της βελτιστοποίησης.



(α') $\max|RK4 - PINN|$ για 400,500,2000 (β') $\max|RK4 - PINN|$ για 200,300,1000 (γ') $\max|RK4 - PINN|$ για 50,100,500

Σχήμα 5.4: Μέγιστη Απόλυτη Διαφορά μεταξύ της Προσεγγιστικής Λύσης RK4 και της Προσεγγιστικής Λύσης PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (5.4) παρατηρούμε ότι, τα γραφήματα της μέγιστης απόλυτης διαφοράς μεταξύ της μεθόδου Runge–Kutta 4 και του PINN και εκείνα της μέγιστης απόλυτης διαφοράς μεταξύ της αναλυτικής λύσης και του PINN είναι πρακτικά ταυτόσημα. Το αποτέλεσμα αυτό υποδηλώνει ότι η μέθοδος Runge–Kutta 4 προσεγγίζει ουσιαστικά την αναλυτική λύση και παρέχει καλύτερη προσέγγιση από το PINN, για το συγκεκριμένο πρόβλημα. Για την μέθοδο Πεπερασμένων Διαφορών αναπτύχθηκε το πρόγραμμα [[62]]

5.1β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.2) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.2

N_0	N_b	N_u	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
400	500	2000	5.56402979e-03	1.64311041e-05	1:01	0:23
300	500	2000	5.26540773e-03	1.79740564e-05	1:06	0:23
200	500	2000	5.64024970e-03	2.47335847e-05	1:07	0:23
100	500	2000	6.25881832e-03	1.89268612e-05	1:06	0:23
50	500	2000	5.49996970e-03	6.42520308e-06	1:05	0:23
400	400	2000	6.42520308e-06	6.42520308e-06	1:06	0:23
400	300	2000	6.13923976e-03	1.50961878e-05	1:06	0:23
400	200	2000	7.96732400e-03	3.26428890e-05	1:07	0:23
400	100	2000	9.11326986e-03	4.06077888e-05	1:07	0:23
400	50	2000	1.00625996e-02	4.89951490e-05	1:05	0:23
400	10	2000	2.96355900e-03	5.30396937e-05	1:06	0:23
400	500	1800	5.31013822e-03	1.91373401e-05	1:06	0:23
400	500	1600	4.99213627e-03	1.56281621e-05	1:06	0:23
400	500	1400	5.17562218e-03	5.68286559e-05	1:07	0:23
400	500	1200	5.35534229e-03	1.91166673e-05	1:06	0:23
400	500	1000	5.45913028e-03	4.06110194e-05	1:05	0:23
400	500	800	6.32995833e-03	4.28095300e-05	1:05	0:23
400	500	600	6.93000713e-03	5.71549026e-05	1:05	0:22
400	500	400	6.11848105e-03	7.56416266e-05	1:05	0:22
400	500	200	5.88032007e-02	4.88279294e-03	1:05	0:24
300	400	1500	4.88284184e-03	5.34515930e-05	1:06	0:23
300	400	1000	5.17965807e-03	6.16368779e-05	1:07	0:24
300	400	500	6.20856369e-03	7.82193383e-05	1:07	0:23
200	300	1500	5.32763824e-03	1.54264053e-05	1:05	0:22
200	300	1000	4.35729744e-03	1.62357428e-05	1:06	0:23
200	300	500	6.85243309e-03	3.12263364e-05	1:05	0:23
100	200	1500	6.21527946e-03	3.46618763e-05	1:05	0:22
100	200	1000	5.88309206e-03	3.98258853e-05	1:07	0:23
100	200	500	6.27258606e-03	8.07745091e-05	1:06	0:23
50	100	1500	6.56537339e-03	1.44423766e-05	1:06	0:23
50	100	1000	7.25649996e-03	1.83046595e-05	1:07	0:23
50	100	500	6.66407309e-03	5.87401373e-05	1:06	0:23

Πίνακας 5.2: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε GPU μέσω CUDA

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης. Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 1 λεπτό, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε λίγο περισσότερο από 20 δευτερόλεπτα, γεγονός που υποδηλώνει περιορισμένη ευαισθησία του χρόνου εκ-

παίδευσης ως προς το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Αντίθετα, κατά την εκτέλεση των υπολογισμών στην CPU, ο χρόνος εκπαίδευσης εξαρτάται από το πλήθος και την σύνθεση των σημείων εκπαίδευσης, στο συγκεκριμένο πρόβλημα. Συγκεκριμένα, η βελτιστοποίηση με Adam απαιτεί από 1 έως περίπου 2 λεπτά, ενώ η L-BFGS κυμαίνεται από 17 μέχρι 30 δευτερόλεπτα.

Ως προς την τάξη της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam έχει ίδια τάξη μεγέθους 10^{-3} , όπως και στην GPU με χρήση CUDA. Ωστόσο, στη μέθοδο L-BFGS, στη CPU εμφανίζει μεγαλύτερη διακύμανση, η οποία κυμαίνεται από 10^{-4} έως και 10^{-6} , ενώ στην GPU μέσω CUDA οι τιμές συγκεντρώνονται κυρίως γύρω από την τάξη μεγέθους 10^{-5} , γεγονός που υποδηλώνει πιο ομοιόμορφη συμπεριφορά της διαδικασίας βελτιστοποίησης.

Καταλήγουμε στο συμπέρασμα ότι, η χρήση CPU είναι επαρκής για προβλήματα μικρού συνόλου δεδομένων, καθώς επιτυγχάνονται συγκρίσιμες τιμές της συνολικής Loss Function. Ωστόσο, καθώς αυξάνεται το πλήθος των σημείων εκπαίδευσης, ο χρόνος εκπαίδευσης στην CPU αυξάνεται αισθητά. Αντίθετα, η GPU με χρήση CUDA επιτρέπει ταχύτερη και πιο αποδοτική εκπαίδευση, με τον χρόνο να παραμένει σχεδόν σταθερός ακόμη και για μεγαλύτερα σύνολα δεδομένων, χωρίς να επηρεάζεται αρνητικά η σύγκλιση της συνάρτησης απώλειας. Για τον λόγο αυτό, στο συγκεκριμένο πρόβλημα, η GPU αποτελεί πιο κατάλληλη επιλογή για την εκπαίδευση των PINNs, ιδίως όταν έχουμε μεγάλο αριθμό σημείων εκπαίδευσης.

5.2 Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες

Στην Μερική Διαφορική Εξίσωση (5.1) εφαρμόζουμε δύο Αρχικές Συνθήκες (Initial Conditions), δύο Περιοδικές Συνοριακές Συνθήκες (Periodic Boundary Conditions).

Σε αντίθεση με την προηγούμενη ενότητα, ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN να είναι το μη συνεκτικό

$$x \in \Omega = [-L, \alpha] \cup [\beta, L], \quad -L < \alpha < \beta < L,$$

$$(5.7) \quad \begin{cases} u_{tt} - c^2 u_{xx} = 0, & x \in [-L, L], t \in [0, T], \\ u(x, 0) = A \sin\left(\frac{\pi}{L}x\right), & x \in [-L, \alpha] \cup [\beta, L], \\ u_t(x, 0) = -A \frac{\pi}{L} c \cos\left(\frac{\pi}{L}x\right), & x \in [-L, \alpha] \cup [\beta, L], \\ u(-L, t) = u(L, t), & t \in [0, T] \\ u_x(-L, t) = u_x(L, t), & t \in [0, T] \end{cases}$$

Για την (5.7) ορίσαμε τις ίδιες μεταβλητές (5.3).

Για την προσέγγιση της λύσης της (5.7) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [63]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

5.2α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Για τις παραμέτρους α και β επιλέχθηκαν συμμετρικές τιμές ως προς το κέντρο του διαστήματος $[-L, L]$, ώστε το αφαιρούμενο υποδιάστημα (α, β) να οδηγεί σε ομοιόμορφη διάσπαση του χωρίου ορισμού.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.7) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.3

$[-L, \alpha] \cup [\beta, L]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-2, -0.0] \cup [0.0, 2]$	5.09918761e-03	1.63952900e-05	1:43	0:28
$[-2, -0.1] \cup [0.1, 2]$	5.10148518e-03	1.52397943e-05	1:44	0:27
$[-2, -0.2] \cup [0.2, 2]$	3.84175568e-03	2.12366140e-05	1:46	0:28
$[-2, -0.3] \cup [0.3, 2]$	3.33505822e-03	1.75680361e-05	1:46	0:28
$[-2, -0.4] \cup [0.4, 2]$	4.32428624e-03	3.40151601e-05	1:45	0:27
$[-2, -0.5] \cup [0.5, 2]$	4.39499645e-03	3.94253511e-05	1:44	0:27
$[-2, -0.6] \cup [0.6, 2]$	5.48634818e-03	3.23345375e-05	1:44	0:27
$[-2, -0.7] \cup [0.7, 2]$	6.06722385e-03	2.34584277e-05	1:44	0:27
$[-2, -0.8] \cup [0.8, 2]$	6.67412998e-03	4.46843769e-05	1:45	0:27
$[-2, -0.9] \cup [0.9, 2]$	8.21236521e-03	3.18420643e-05	1:44	0:28
$[-2, -1.0] \cup [1.0, 2]$	1.00003108e-02	7.97780594e-05	1:44	0:28
$[-2, -1.1] \cup [1.1, 2]$	1.84360947e-02	1.45591679e-04	2:09	0:31
$[-2, -1.2] \cup [1.2, 2]$	1.90119669e-02	3.19866231e-04	2:01	0:32
$[-2, -1.3] \cup [1.3, 2]$	2.12570932e-02	2.38077977e-04	2:03	0:32
$[-2, -1.4] \cup [1.4, 2]$	2.58957855e-02	1.67240767e-04	1:58	0:32
$[-2, -1.5] \cup [1.5, 2]$	2.50932127e-02	2.44314026e-04	1:59	0:31

Πίνακας 5.3: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε CPU και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Από τον Πίνακα 5.3 μπορούν να εξαχθούν τα εξής συμπεράσματα

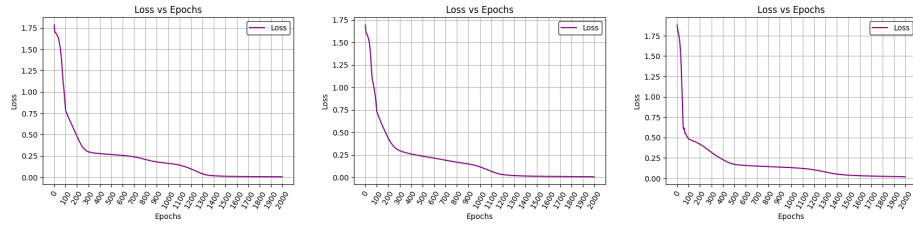
Από τα αποτελέσματα της συνολικής Loss Function, μέσω της βελτιστοποίησης Adam, παρατηρούμε ότι υπάρχει σταδιακή αύξηση της τάξης μεγέθους, από 10^{-3} σε 10^{-2} , γεγονός που υποδηλώνει ότι η μείωση των δεδομένων εκπαίδευσης, δηλαδή η αύξηση του υποδιαστήματος (α, β) , των Αρχικών Συνθηκών καθιστά το συγκεκριμένο πρόβλημα δυσκολότερο για το δίκτυο με μεγαλύτερο υπολογιστικό κόστος.

Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης

απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους που κυμαίνεται από 10^{-5} έως 10^{-4} , το οποίο υποδηλώνει ότι το δίκτυο αντιμετωπίζει την ίδια δυσκολία ως προς την μείωση των δεδομένων εκπαίδευσης. Αντίθετα, ο χρόνος εκπαίδευσης της μεθόδου L-BFGS παραμένει συγκριτικά πιο σταθερός, με μικρές μόνο διακυμάνσεις.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

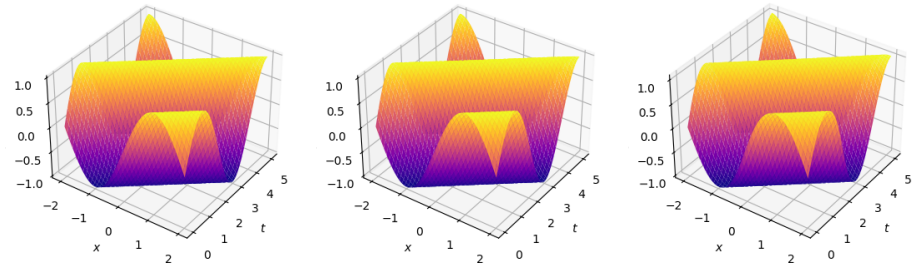
Με βάση τα παραπάνω, παίρνουμε τα γραφήματα 5.5, 5.6, 5.7:



(α') Loss με Α.Σ. στο $[-2, -0.1] \cup [0.1, 2]$ (β') Loss με Α.Σ. στο $[-2, -0.6] \cup [0.6, 2]$ (γ') Loss με Α.Σ. στο $[-2, -1.1] \cup [1.1, 2]$

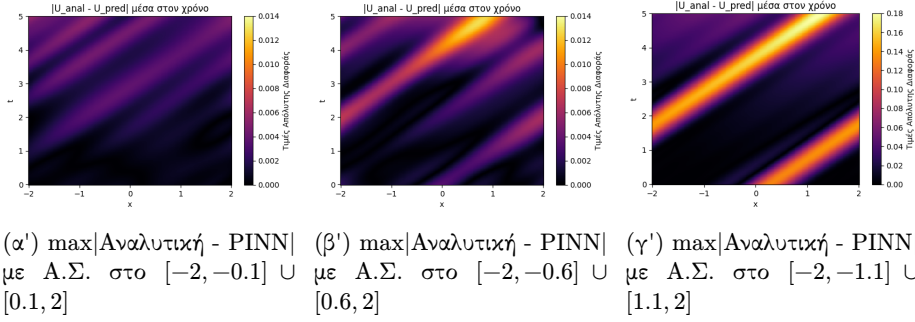
Σχήμα 5.5: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400,500,2000.

Στο Σχήμα (5.5) παρατηρούμε ότι, για διαφορετικές επιλογές του υποδιαστήματος (α, β) , το PINN εμφανίζει παρόμοια πορεία σύγκλισης της Loss Function κατά τη διάρκεια της εκπαίδευσης. Το γεγονός αυτό υποδηλώνει ότι το δίκτυο διατηρεί τη δυνατότητα να εκπαιδευτεί αποτελεσματικά ακόμη και όταν χρησιμοποιείται μόνο μέρος των δεδομένων των Αρχικών Συνθηκών. Η παρατήρηση αυτή επιβεβαιώνεται και από το Σχήμα (5.6).



(α') $\bar{u}_\theta(x, t)$ με Α.Σ. στο $[-2, -0.1] \cup [0.1, 2]$ (β') $\bar{u}_\theta(x, t)$ με Α.Σ. στο $[-2, -0.6] \cup [0.6, 2]$ (γ') $\bar{u}_\theta(x, t)$ με Α.Σ. στο $[-2, -1.1] \cup [1.1, 2]$

Σχήμα 5.6: Γραφήματα της Προσεγγιστικής λύσης PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400,500,2000.



Σχήμα 5.7: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400,500,2000.

Στο Σχήμα (5.7) παρατηρούμε ότι, ενώ η αύξηση του υποδιαστήματος (α, β) οδηγεί σε μεγαλύτερη απόκλιση μεταξύ της προσεγγιστικής και της αναλυτικής λύσης, η συνολική συμπεριφορά της προσεγγιστικής λύσης παραμένει ποιοτικά ικανοποιητική.

Το αποτέλεσμα αυτό υποδηλώνει ότι, η εκπαίδευση, ακόμη και με μερική χρήση του συνόλου εκπαίδευσης της Αρχικής Συνθήκης, καταφέρνει να γενικεύσει επαρκώς την φυσική του προβλήματος, ακόμη και όταν το σύνολο των σημείων εκπαίδευσης είναι περιορισμένο, για το συγκεκριμένο πρόβλημα.

5.2β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.7) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.4

$[-L, \alpha] \cup [\beta, L]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-2, -0.0] \cup [0.0, 2]$	5.56402979e-03	1.64311041e-05	1:07	0:23
$[-2, -0.1] \cup [0.1, 2]$	6.36431109e-03	1.89569419e-05	1:08	0:23
$[-2, -0.2] \cup [0.2, 2]$	4.90799733e-03	2.14131869e-05	1:07	0:23
$[-2, -0.3] \cup [0.3, 2]$	3.81341716e-03	3.60860613e-05	1:07	0:23
$[-2, -0.4] \cup [0.4, 2]$	3.57934600e-03	1.59413630e-05	1:07	0:23
$[-2, -0.5] \cup [0.5, 2]$	3.60538298e-03	3.45039662e-05	1:07	0:23
$[-2, -0.6] \cup [0.6, 2]$	4.79070749e-03	1.38301166e-05	1:08	0:23
$[-2, -0.7] \cup [0.7, 2]$	1.02577871e-02	3.80013895e-04	1:07	0:23
$[-2, -0.8] \cup [0.8, 2]$	1.08535066e-02	1.48150270e-04	1:06	0:23
$[-2, -0.9] \cup [0.9, 2]$	1.67709552e-02	3.97447002e-04	1:08	0:24
$[-2, -1.0] \cup [1.0, 2]$	2.12203022e-02	4.18092124e-04	1:09	0:23
$[-2, -1.1] \cup [1.1, 2]$	3.91827971e-02	4.62445198e-04	1:08	0:23
$[-2, -1.2] \cup [1.2, 2]$	4.92653474e-02	3.37798672e-04	1:07	0:23
$[-2, -1.3] \cup [1.3, 2]$	4.66767177e-02	2.21561408e-04	1:08	0:23
$[-2, -1.4] \cup [1.4, 2]$	5.15547618e-02	1.21410987e-04	1:07	0:23
$[-2, -1.5] \cup [1.5, 2]$	6.15334921e-02	7.41622760e-04	1:07	0:23

Πίνακας 5.4: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε GPU μέσω CUDA και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές του υποδιαστήματος (α, β) . Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 1 λεπτό, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε λίγο περισσότερο από 20 δευτερόλεπτα, γεγονός που υποδηλώνει περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το περιορισμένο σύνολο των δεδομένων εκπαίδευσης και στη δυσκολία του προβλήματος.

Αντίθετα, κατά την εκτέλεση των υπολογισμών στην CPU, ο χρόνος εκπαίδευσης αυξάνεται, καθώς αυξάνεται το υποδιάστημα (α, β) . Συγκεκριμένα, η μέθοδος Adam κυμαίνεται από περίπου 1:40 έως και 2 λεπτά, ενώ η μέθοδος L-BFGS κυμαίνεται από 27 μέχρι 32 δευτερόλεπτα. Το γεγονός αυτό υποδηλώνει ότι η εκπαίδευση στην CPU είναι πιο ευαίσθητη στη δυσκολία του προβλήματος.

Ως προς την τάξη μεγέθους της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam κυμαίνεται από 10^{-2} έως και 10^{-3} , αντίστοιχα με εκείνες που προκύπτουν στην GPU με χρήση CUDA. Επίσης, στη μέθοδο L-BFGS παρατηρούμε ότι στην CPU έχει τάξη μεγέθους που κυμαίνεται από 10^{-4} έως και 10^{-5} , αντίστοιχα πάλι με εκείνες που προκύπτουν στην GPU με χρήση CUDA, γεγονός που δείχνει ότι η ποιότητα της τελικής σύγκλισης παραμένει παρόμοια ανεξαρτήτως του υπολογιστικού μέσου.

Καταλήγουμε στο συμπέρασμα ότι η χρήση της GPU μέσω CUDA υπερτερεί ως προς τον χρόνο εκπαίδευσης, διατηρώντας συγκρίσιμη ακρίβεια με την CPU. Για τον λόγο αυτό, στο συγκεκριμένο πρόβλημα, η GPU αποτελεί πιο κατάλληλη επιλογή για την εκπαίδευση των PINNs, ιδίως όταν έχουμε αυξημένη δυσκολία και περιορισμένο σύνολο δεδομένων εκπαίδευσης.

5.3 Συνοριακές Συνθήκες Dirichlet

Στην Μερική Διαφορική Εξίσωση (5.1) εφαρμόζουμε δύο Αρχικές Συνθήκες (Initial Conditions), δύο Συνοριακές Συνθήκες Dirichlet (Dirichlet Boundary Conditions) και ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN.

$$(5.8) \quad \begin{cases} u_{tt} - c^2 u_{xx} = 0, & x \in [-L, L], t \in [0, T], \\ u(x, 0) = A \sin\left(\frac{\pi}{L}x\right), & x \in [-L, L], \\ u_t(x, 0) = 0, & x \in [-L, L], \\ u(-L, t) = 0, & t \in [0, T] \\ u(L, t) = 0, & t \in [0, T] \end{cases}$$

Για την (5.8) ορίσαμε τις ίδιες μεταβλητές (5.3).

Για την προσέγγιση της λύσης της (5.8) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [64]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

5.3α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.8) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.5

N_0	N_b	N_u	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
400	500	2000	9.68897063e-03	4.86276083e-04	1:59	0:29
300	500	2000	1.29958373e-02	9.76613024e-04	1:53	0:32
200	500	2000	2.05650926e-02	3.15056415e-03	1:49	0:28
100	500	2000	9.36890766e-03	7.18099531e-04	1:49	0:30
50	500	2000	2.13126764e-02	1.58186536e-03	1:48	0:29
400	400	2000	1.38241658e-02	1.00957963e-03	1:49	0:32
400	300	2000	2.31647119e-02	2.91209831e-03	1:49	0:29
400	200	2000	1.03489207e-02	8.89956136e-04	1:52	0:30
400	100	2000	8.32379237e-03	6.73355302e-04	1:51	0:29
400	50	2000	1.00182574e-02	1.13073271e-03	1:47	0:29
400	10	2000	4.54086289e-02	7.13758578e-04	1:46	0:30
400	500	1800	1.03117498e-02	8.17598775e-04	1:50	0:28
400	500	1600	6.88717747e-03	1.07669027e-03	1:45	0:28
400	500	1400	1.20746428e-02	3.55376484e-04	1:43	0:26
400	500	1200	4.50187847e-02	1.09571835e-03	1:31	0:25
400	500	1000	4.17517424e-02	8.02586728e-04	1:27	0:24
400	500	800	1.64853614e-02	1.47338898e-03	1:14	0:18
400	500	600	1.76839735e-02	1.06804655e-03	1:04	0:16
400	500	400	4.00375314e-02	1.58870220e-03	1:02	0:16
400	500	200	1.44917723e-02	1.59317569e-03	0:53	0:15
300	400	1500	8.49587098e-03	4.02965874e-04	1:34	0:26
300	400	1000	1.68272257e-02	1.16236962e-03	1:22	0:24
300	400	500	4.76407781e-02	1.65805558e-03	1:03	0:16
200	300	1500	6.24380559e-02	1.91201246e-03	1:34	0:25
200	300	1000	1.32273789e-02	1.78509031e-03	1:19	0:23
200	300	500	4.30935919e-02	1.30476861e-03	1:02	0:16
100	200	1500	2.86642164e-02	6.75892865e-04	1:35	0:26
100	200	1000	3.65088247e-02	5.15123829e-04	1:16	0:20
100	200	500	3.34808789e-02	2.38596019e-03	0:58	0:16
50	100	1500	9.03742015e-03	6.17707206e-04	1:30	0:24
50	100	1000	1.80169214e-02	1.02653366e-03	1:16	0:23
50	100	500	2.65356917e-02	1.73582137e-03	1:05	0:18

Πίνακας 5.5: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Συνοριακές Συνθήκες Dirichlet με εκπαίδευση σε CPU

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Από τον Πίνακα 5.5 μπορούν να εξαχθούν τα εξής συμπεράσματα

Τα αποτελέσματα της συνολικής Loss Function μέσω της Μεθόδου βελτιστοποίησης Adam παρατηρούμε ότι παραμένουν της ίδιας τάξης μεγέθους, 10^{-2} , για όλες τις εξεταζόμενες επιλογές σημείων εκπαίδευσης. Όμως, παρατηρούνται διακυμάνσεις που υποδηλώνουν ότι η σύγκλιση της Adam επηρεάζεται από τη σύνθεση του συνόλου εκπαίδευσης και όχι μόνο από το συνολικό πλήθος των σημείων.

Ως προς τον υπολογιστικό χρόνο της Adam, προκύπτει ότι η αύξηση του πλήθους των σημείων εκπαίδευσης οδηγεί γενικά σε μεγαλύτερους χρόνους εκπαίδευσης, με τον αριθμό των collocation points N_u να αποτελεί τον κυρίαρχο παράγοντα κόστους.

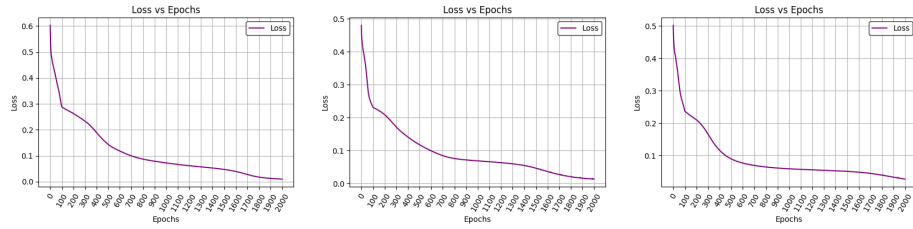
Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους που κυμαίνεται από 10^{-3} έως 10^{-4} .

Ο συνολικός χρόνος εκπαίδευσης της μεθόδου L-BFGS κυμαίνεται εντός ενός σχετικά στενού χρονικού διαστήματος, γεγονός που υποδηλώνει ότι επηρεάζεται σε μικρότερο βαθμό από το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Τα αποτελέσματα δείχνουν ότι η αύξηση του πλήθους των σημείων εκπαίδευσης δεν οδηγεί κατ' ανάγκη σε μείωση της συνολικής Loss Function, ενώ ταυτόχρονα μπορεί να αυξήσουν το υπολογιστικό κόστος σημαντικά, για το συγκεκριμένο πρόβλημα.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

Με βάση τα παραπάνω παίρνουμε τα γραφήματα 5.8, 5.9, 5.10 και 5.11:



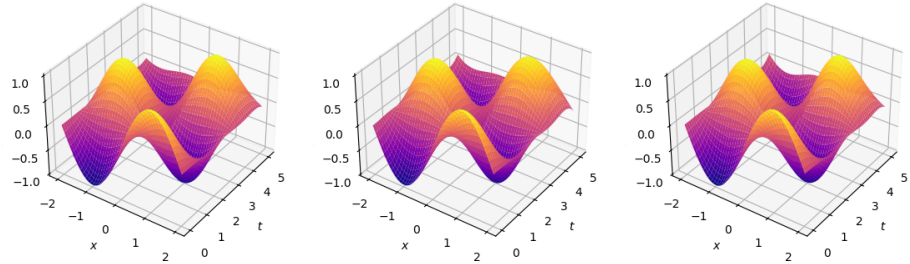
(α') Loss με 400,500,2000

(β') Loss με 200,300,1000

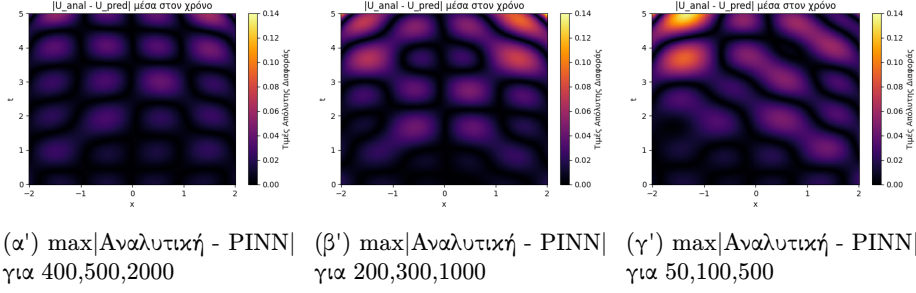
(γ') Loss με 50,100,500

Σχήμα 5.8: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (5.8) παρατηρούμε ότι, για τις διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης, η εξέλιξη της συνολικής συνάρτησης απώλειας ως προς τις εποχές παρουσιάζει παρόμοια συμπεριφορά σύγκλισης. Το γεγονός αυτό υποδηλώνει ότι το δίκτυο μπορεί να εκπαιδευτεί αποτελεσματικά χωρίς την ανάγκη πολύ μεγάλου αριθμού σημείων, διατηρώντας την ακρίβεια της προσέγγισης, όπως φαίνεται και στο Σχήμα (5.9).

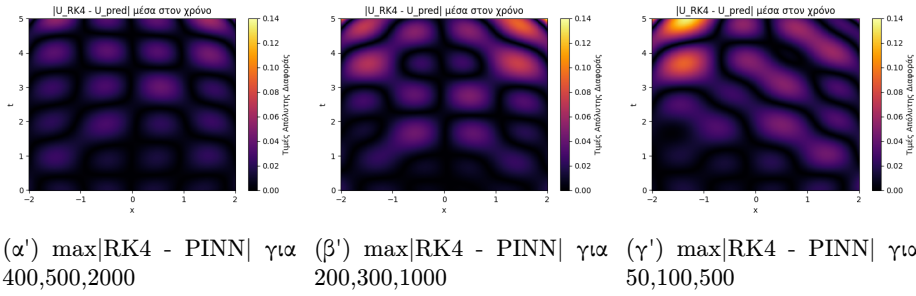
(α') $\bar{u}_\theta(x, t)$ για 400,500,2000(β') $\bar{u}_\theta(x, t)$ για 200,300,1000(γ') $\bar{u}_\theta(x, t)$ για 50,100,500

Σχήμα 5.9: Γραφήματα της προσέγγισης της λύσης με PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500



Σχήμα 5.10: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (5.10) παρατηρούμε ότι, ακόμη και για σχετικά μικρό αριθμό σημείων εκπαίδευσης, η προσεγγιστική λύση ακολουθεί ικανοποιητικά τη συμπεριφορά της αναλυτικής λύσης σε όλο τον εξεταζόμενο χωροχρόνο, με τις αποκλίσεις να παραμένουν αόρατες οπτικά.



Σχήμα 5.11: Μέγιστη Απόλυτη Διαφορά μεταξύ της Προσεγγιστικής Λύσης RK4 και της Προσεγγιστικής Λύσης PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (5.11) παρατηρούμε ότι τα γραφήματα της μέγιστης απόλυτης διαφοράς μεταξύ της μεθόδου Runge–Kutta 4 και του PINN και εκείνα της μέγιστης απόλυτης διαφοράς μεταξύ της αναλυτικής λύσης και του PINN είναι πρακτικά ταυτόσημα. Το αποτέλεσμα αυτό υποδηλώνει ότι η μέθοδος Runge–Kutta 4 προσεγγίζει ουσιαστικά την αναλυτική λύση και παρέχει καλύτερη προσέγγιση από το PINN για το συγκεκριμένο πρόβλημα. Για την μέθοδο Πεπερασμένων Διαφορών αναπτύχθηκε το πρόγραμμα [65]

5.3β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.8) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.6

N_0	N_b	N_u	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
400	500	2000	2.44856309e-02	1.48896815e-03	0:56	0:21
300	500	2000	2.49608308e-02	1.37311721e-03	0:56	0:21
200	500	2000	2.45153997e-02	1.55228877e-03	0:56	0:20
100	500	2000	2.35840697e-02	1.64495409e-03	0:56	0:20
50	500	2000	2.41596587e-02	1.48235483e-03	0:56	0:21
400	400	2000	2.44940259e-02	1.47882104e-03	0:56	0:20
400	300	2000	2.47220229e-02	1.42431376e-03	0:57	0:21
400	200	2000	2.60682311e-02	1.43035525e-03	0:56	0:21
400	100	2000	2.56665144e-02	1.33142539e-03	0:56	0:20
400	50	2000	2.31874511e-02	1.44927285e-03	0:55	0:20
400	10	2000	1.94841120e-02	1.26293697e-03	0:56	0:21
400	500	1800	2.40953136e-02	1.59617933e-03	0:56	0:20
400	500	1600	2.51005776e-02	1.56445405e-03	0:56	0:20
400	500	1400	2.40932703e-02	1.53662765e-03	0:55	0:21
400	500	1200	2.48704925e-02	1.55482325e-03	0:56	0:21
400	500	1000	2.43124049e-02	1.49167236e-03	0:56	0:21
400	500	800	2.23313142e-02	1.63572456e-03	0:55	0:21
400	500	600	2.28236243e-02	1.70287082e-03	0:55	0:20
400	500	400	3.17982323e-02	1.52392685e-03	0:55	0:21
400	500	200	3.01137045e-02	2.43048370e-03	0:55	0:21
300	400	1500	2.46666502e-02	1.52742164e-03	0:55	0:20
300	400	1000	2.47050934e-02	1.54399965e-03	0:56	0:21
300	400	500	2.70182453e-02	1.62178185e-03	0:56	0:21
200	300	1500	2.39559021e-02	1.53002341e-03	0:55	0:21
200	300	1000	2.38762517e-02	1.49790943e-03	0:56	0:21
200	300	500	2.63059363e-02	1.44271925e-03	0:56	0:21
100	200	1500	2.55485382e-02	1.62065146e-03	0:56	0:21
100	200	1000	2.61817425e-02	1.77336461e-03	0:56	0:21
100	200	500	2.79889852e-02	1.67110621e-03	0:55	0:21
50	100	1500	2.63936110e-02	1.40887871e-03	0:55	0:21
50	100	1000	2.69117001e-02	1.51133956e-03	0:57	0:21
50	100	500	2.94107273e-02	1.51388464e-03	0:56	0:21

Πίνακας 5.6: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Συνοριακές Συνθήκες Dirichlet με εκπαίδευση σε GPU μέσω CUDA

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης. Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 1 λεπτό, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε λίγο περισσότερο από 20 δευτερόλεπτα, γεγονός που υποδηλώνει περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Αντίθετα, κατά την εκτέλεση των υπολογισμών στην CPU, ο χρόνος εκπαίδευσης εξαρτάται από το πλήθος και την σύνθεση των σημείων εκπαίδευσης, στο συγκεκριμένο πρόβλημα. Συγκεκριμένα, η βελτιστοποίηση με Adam απαιτεί από 1 έως περίπου 2 λεπτά, ενώ η L-BFGS κυμαίνεται από 16 μέχρι 32 δευτερόλεπτα.

Ως προς την τάξη της απόκλισης της Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam έχει σταθερή απόκλιση της τάξης 10^{-2} , αντίστοιχα με εκείνες που προκύπτουν στην GPU με χρήση CUDA. Επίσης, στη μέθοδο L-BFGS παρατηρούμε ότι στην CPU έχει τάξη μεγέθους που κυμαίνεται από 10^{-3} έως και $10^{(-4)}$, ενώ στην GPU μέσω CUDA έχουμε σχεδόν σταθερή απόκλιση της τάξης 10^{-3} .

Καταλήγουμε στο συμπέρασμα ότι η χρήση της GPU μέσω CUDA υπερτερεί ως προς τον χρόνο εκπαίδευσης, διατηρώντας συγκρίσιμη ακρίβεια με την CPU. Για τον λόγο αυτό, στο συγκεκριμένο πρόβλημα, η GPU αποτελεί πιο κατάλληλη επιλογή για την εκπαίδευση των PINNs, ιδίως όταν έχουμε αυξημένη δυσκολία και περιορισμένο σύνολο δεδομένων εκπαίδευσης.

5.4 Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Συνοριακές Συνθήκες Dirichlet

Στην Μερική Διαφορική Εξίσωση (5.1) εφαρμόζουμε δύο Αρχικές Συνθήκες (Initial Conditions), δύο Συνοριακές Συνθήκες Dirichlet (Boundary Conditions).

Ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN να είναι το μη συνεκτικό ως προς τον χώρο

$$x \in \Omega = [-L, \alpha] \cup [\beta, L], \quad -L < \alpha < \beta < L,$$

$$(5.9) \quad \begin{cases} u_{tt} - c^2 u_{xx} = 0, & x \in [-L, L], t \in [0, T], \\ u(x, 0) = A \sin\left(\frac{\pi}{L}x\right), & x \in [-L, \alpha] \cup [\beta, L], \\ u_t(x, 0) = 0, & x \in [-L, \alpha] \cup [\beta, L], \\ u(-L, t) = 0, & t \in [0, T] \\ u(L, t) = 0, & t \in [0, T] \end{cases}$$

Για την (5.9) ορίσαμε τις ίδιες μεταβλητές (5.3).

Για την προσέγγιση της λύσης της (5.9) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [66]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

5.4α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Για τις παραμέτρους α και β επιλέχθηκαν συμμετρικές τιμές ως προς το κέντρο του διαστήματος $[-L, L]$, ώστε το αφαιρούμενο υποδιάστημα (α, β) να οδηγεί σε ομοιόμορφη διάσπαση του χωρίου ορισμού.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.9) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.7

$[-L, \alpha] \cup [\beta, L]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-2, -0.0] \cup [0.0, 2]$	9.68897063e-03	4.86276083e-04	1:51	0:28
$[-2, -0.1] \cup [0.1, 2]$	8.25070310e-03	4.56364127e-04	1:54	0:29
$[-2, -0.2] \cup [0.2, 2]$	6.03595842e-03	5.38339547e-04	1:54	0:30
$[-2, -0.3] \cup [0.3, 2]$	1.29796863e-02	6.16758887e-04	1:55	0:30
$[-2, -0.4] \cup [0.4, 2]$	1.91570818e-02	7.13086396e-04	1:52	0:31
$[-2, -0.5] \cup [0.5, 2]$	2.45940350e-02	8.36852589e-04	1:54	0:30
$[-2, -0.6] \cup [0.6, 2]$	3.27733196e-02	1.06082438e-03	1:55	0:30
$[-2, -0.7] \cup [0.7, 2]$	2.89859679e-02	1.98371592e-03	1:58	0:29
$[-2, -0.8] \cup [0.8, 2]$	2.04534661e-02	3.13283433e-03	1:53	0:30
$[-2, -0.9] \cup [0.9, 2]$	1.45325940e-02	2.63153412e-03	1:53	0:30
$[-2, -1.0] \cup [1.0, 2]$	1.59972534e-02	2.65857833e-03	1:56	0:30

Πίνακας 5.7: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Συνοριακές Συνθήκες Dirichlet με εκπαίδευση σε CPU και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Από τον Πίνακα 5.7 μπορούν να εξαχθούν τα εξής συμπεράσματα

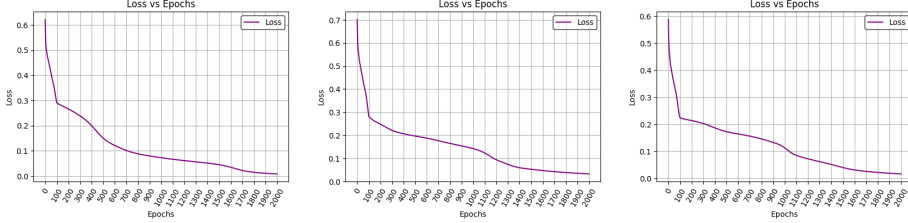
Από τα αποτελέσματα της συνολικής Loss Function, μέσω της βελτιστοποίησης Adam, παρατηρούμε ότι υπάρχει σταδιακή αύξηση της τάξης μεγέθους, από 10^{-3} σε 10^{-2} , γεγονός που υποδηλώνει ότι η μείωση των δεδομένων εκπαίδευσης, δηλαδή η αύξηση του υποδιαστήματος (α, β) , των Αρχικών Συνθηκών καθιστά το συγκεκριμένο πρόβλημα δυσκολότερο για το δίκτυο.

Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους που κυμαίνεται από 10^{-4} έως 10^{-3} , το οποίο υποδηλώνει ότι το δίκτυο αντιμετωπίζει την ίδια δυσκολία ως προς την μείωση των δεδομένων εκπαίδευσης.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με

την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

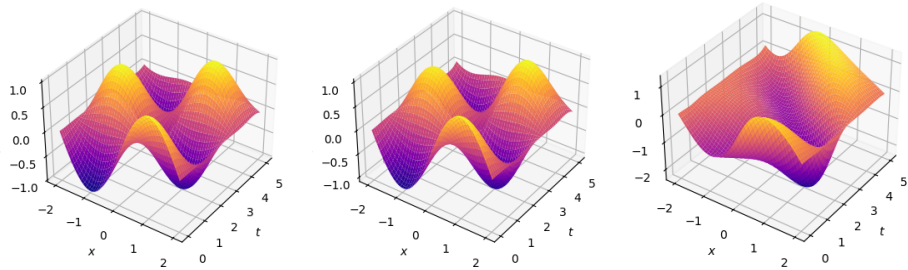
Με βάση τα παραπάνω παίρνουμε τα γραφήματα 5.12, 5.13, 5.14:



(α') Loss με Α.Σ. (β') Loss με Α.Σ. (γ') Loss με Α.Σ.
 $[-2, -0.1] \cup [0.1, 2]$ $[-2, -0.6] \cup [0.6, 2]$ $[-2, -1.0] \cup [1.0, 2]$

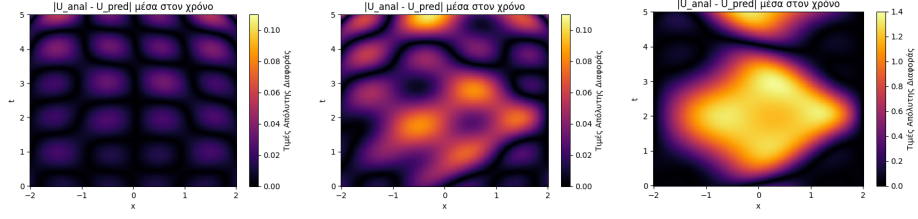
Σχήμα 5.12: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

Στο Σχήμα (5.12) παρατηρούμε ότι, για διαφορετικές επιλογές του υποδιαστήματος (α, β) , το PINN εμφανίζει παρόμοια πορεία σύγκλισης της Loss Function κατά τη διάρκεια της εκπαίδευσης. Όμως, όπως φαίνεται στο Σχήμα (5.13), η ελαχιστοποίηση της συνολικής Loss Function, από μόνη της, δεν αποτελεί ούτε μοναδική ούτε επαρκή συνθήκη για να εξασφαλιστεί ότι η εκπαίδευση έγινε προς την σωστή λύση.



(α') $\bar{u}_\theta(x, t)$ με Α.Σ. (β') $\bar{u}_\theta(x, t)$ με Α.Σ. (γ') $\bar{u}_\theta(x, t)$ με Α.Σ.
 $[-2, -0.1] \cup [0.1, 2]$ $[-2, -0.6] \cup [0.6, 2]$ $[-2, -1.0] \cup [1.0, 2]$

Σχήμα 5.13: Γραφήματα της προσέγγισης της λύσης με PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.



(α') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -0.1] \cup [0.1, 2]$ (β') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -0.6] \cup [0.6, 2]$ (γ') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -1.0] \cup [1.0, 2]$

Σχήμα 5.14: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

Στο Σχήμα (5.14) παρατηρούμε ότι υπάρχει ένα όριο στο πόσο μεγάλο μέρος μπορούμε να εξαιρέσουμε από το σύνολο εκπαίδευσης των Συνοριακών Συνθηκών, για το συγκεκριμένο πρόβλημα. Μέχρι ένα σημείο, η προσέγγιση της λύσης διατηρείται σε ικανοποιητικό επίπεδο ακρίβειας, ενώ πέρα από αυτό το όριο η ποιότητα της προσέγγισης υποβαθμίζεται σημαντικά.

5.4β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.9) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.8

$[-L, \alpha] \cup [\beta, L]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-2, -0.0] \cup [0.0, 2]$	2.44856309e-02	1.48896815e-03	0:58	0:21
$[-2, -0.1] \cup [0.1, 2]$	2.56364066e-02	1.48226856e-03	0:57	0:21
$[-2, -0.2] \cup [0.2, 2]$	2.70231385e-02	1.36855175e-03	0:57	0:21
$[-2, -0.3] \cup [0.3, 2]$	2.86370032e-02	1.36092445e-03	0:58	0:21
$[-2, -0.4] \cup [0.4, 2]$	2.86395103e-02	1.56132504e-03	0:57	0:21
$[-2, -0.5] \cup [0.5, 2]$	2.97430214e-02	1.59427081e-03	0:57	0:21
$[-2, -0.6] \cup [0.6, 2]$	3.08412258e-02	1.57432095e-03	0:58	0:21
$[-2, -0.7] \cup [0.7, 2]$	2.89902873e-02	1.75947964e-03	0:57	0:21
$[-2, -0.8] \cup [0.8, 2]$	2.76803840e-02	2.21690978e-03	0:58	0:21
$[-2, -0.9] \cup [0.9, 2]$	2.80373227e-02	1.79202168e-03	0:57	0:21
$[-2, -1.0] \cup [1.0, 2]$	2.72730514e-02	2.34934315e-03	0:57	0:21

Πίνακας 5.8: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Συνοριακές Συνθήκες Dirichlet με εκπαίδευση σε GPU μέσω CUDA και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Κατά την εκπαίδευση, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές του υποδιαστήματος, τόσο σε CPU, όσο και σε GPU μέσω CUDA. Συγκεκριμένα η CPU απαιτεί περίπου 2 λεπτά για την

βελτιστοποίηση με Adam και περίπου 30 δευτερόλεπτα για την βελτιστοποίηση με L-BFGS. Η GPU μέσω CUDA απαιτεί 1 λεπτό για την βελτιστοποίηση με Adam και περίπου 20 δευτερόλεπτα για την βελτιστοποίηση με L-BFGS.

Μπορούμε να συμπεράνουμε ότι, ο συνολικός χρόνος εκπαίδευσης τόσο στην CPU όσο και στην GPU μέσω CUDA δεν εξαρτάται σχεδόν καθόλου από το πλήθος των σημείων εκπαίδευσης, στο συγκεκριμένο πρόβλημα.

Ως προς την τάξη μεγέθους της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam κυμαίνεται από 10^{-2} έως και 10^{-3} , ενώ στην GPU μέσω CUDA έχει σταθερή τάξη μεγέθους 10^{-2} . Επίσης, στη μέθοδο L-BFGS παρατηρούμε ότι στην CPU έχει τάξη μεγέθους που κυμαίνεται από 10^{-3} έως και 10^{-4} , ενώ στην GPU μέσω CUDA έχει σταθερή τάξη 10^{-3} .

Καταλήγουμε στο συμπέρασμα ότι η χρήση της GPU μέσω CUDA υπερτερεί ως προς τον χρόνο εκπαίδευσης, διατηρώντας συγκρίσιμη ακρίβεια με την CPU. Για τον λόγο αυτό, στο συγκεκριμένο πρόβλημα, η GPU αποτελεί πιο κατάλληλη επιλογή για την εκπαίδευση των PINNs, ιδίως όταν έχουμε αυξημένη δυσκολία και περιορισμένο σύνολο δεδομένων εκπαίδευσης.

5.5 Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet

Στην Μερική Διαφορική Εξίσωση (5.1) εφαρμόζουμε δύο Αρχικές Συνθήκες (Initial Conditions), δύο Συνοριακές Συνθήκες Dirichlet (Boundary Conditions).

Σε αντίθεση με την προηγούμενη ενότητα, ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN να είναι το μη συνεκτικό ως προς τον χρόνο

$$t \in [0, \alpha] \cup [\beta, T], \quad 0 < \alpha < \beta < T,$$

$$(5.10) \quad \begin{cases} u_{tt} - c^2 u_{xx} = 0, & x \in [-L, L], t \in [0, T], \\ u(x, 0) = A \sin\left(\frac{\pi}{L}x\right), & x \in [-L, L], \\ u_t(x, 0) = 0, & x \in [-L, L], \\ u(-L, t) = 0, & t \in [0, \alpha] \cup [\beta, T] \\ u(L, t) = 0, & t \in [0, \alpha] \cup [\beta, T] \end{cases}$$

Για την (5.10) ορίσαμε τις ίδιες μεταβλητές (5.3).

Για την προσέγγιση της λύσης της (5.10) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [67]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

5.5α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Για τις παραμέτρους α και β επιλέχθηκαν συμμετρικές τιμές ως προς το κέντρο του διαστήματος $[0, T]$, ώστε το αφαιρούμενο υποδιάστημα (α, β) να οδηγεί σε ομοιόμορφη διάσπαση του χωρίου ορισμού.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.10) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.9

$[0, \alpha] \cup [\beta, T]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[0, 2.5] \cup [2.5, 5]$	9.68897063e-03	4.86276083e-04	1:44	0:31
$[0, 2.4] \cup [2.6, 5]$	9.58780758e-03	5.78686828e-04	1:45	0:19
$[0, 2.3] \cup [2.7, 5]$	8.89377948e-03	5.46641764e-04	1:51	0:19
$[0, 2.2] \cup [2.8, 5]$	9.21059772e-03	5.68258227e-04	1:47	0:18
$[0, 2.1] \cup [2.9, 5]$	9.46580991e-03	5.19330904e-04	1:12	0:18
$[0, 2.0] \cup [3.0, 5]$	1.01871099e-02	5.30202815e-04	1:12	0:22
$[0, 1.9] \cup [3.1, 5]$	1.07634068e-02	5.90765150e-04	1:11	0:20
$[0, 1.8] \cup [3.2, 5]$	1.09669436e-02	5.72827179e-04	1:07	0:16
$[0, 1.7] \cup [3.3, 5]$	1.06078461e-02	5.58257569e-04	1:07	0:17
$[0, 1.6] \cup [3.4, 5]$	9.79082100e-03	5.79964544e-04	1:08	0:18
$[0, 1.5] \cup [3.5, 5]$	9.09605809e-03	5.56839863e-04	1:13	0:20
$[0, 1.4] \cup [3.6, 5]$	8.60827882e-03	6.09142764e-04	1:19	0:19
$[0, 1.3] \cup [3.7, 5]$	9.86582227e-03	7.80350121e-04	1:19	0:21
$[0, 1.2] \cup [3.8, 5]$	1.26794819e-02	9.96027142e-04	1:17	0:20
$[0, 1.1] \cup [3.9, 5]$	1.36013972e-02	1.03282125e-03	1:12	0:21
$[0, 1.0] \cup [4.0, 5]$	1.16792228e-02	1.19597674e-03	1:13	0:18

Πίνακας 5.9: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet με εκπαίδευση σε CPU και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Από τον Πίνακα 5.9 μπορούν να εξαχθούν τα εξής συμπεράσματα

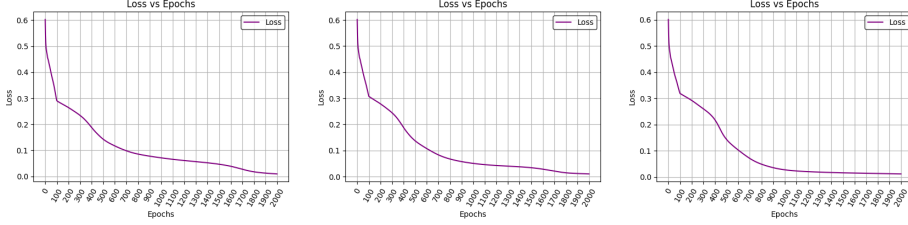
Από τα αποτελέσματα της συνολικής Loss Function, μέσω της βελτιστοποίησης Adam, παρατηρούμε ότι υπάρχει σταδιακή αύξηση της τάξης μεγέθους, από 10^{-3} σε 10^{-2} , γεγονός που υποδηλώνει ότι η μείωση των δεδομένων εκπαίδευσης, δηλαδή η αύξηση του υποδιαστήματος (α, β) , των Συνοριακών Συνθηκών καθιστά το συγκεκριμένο πρόβλημα δυσκολότερο για το δίκτυο με μεγαλύτερο υπολογιστικό κόστος.

Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους που κυμαίνεται από 10^{-4} έως 10^{-3} , το οποίο υποδηλώνει ότι το δίκτυο αντιμετωπίζει την ίδια δυσκολία ως προς την μείωση των δεδομένων εκπαίδευσης με επίσης μεγαλύτερο υπολογιστικό κόστος.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με

την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

Με βάση τα παραπάνω παίρνουμε τα γραφήματα 5.15, 5.16, 5.17:

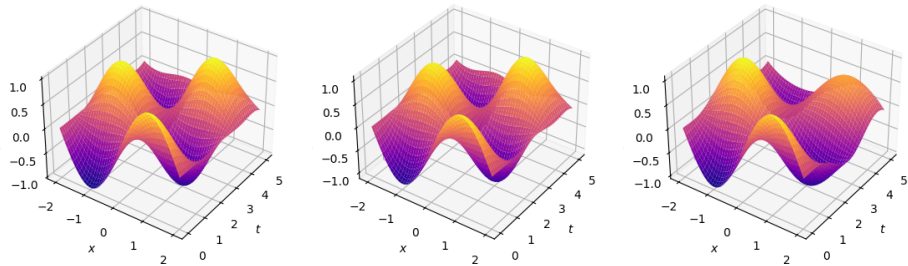


(α') Loss με $\Sigma.\Sigma.$ $[0, 2.4] \cup$ (β') Loss με $\Sigma.\Sigma.$ $[0, 1.7] \cup$ (γ') Loss με $\Sigma.\Sigma.$ $[0, 1.0] \cup$
 $[2.5, 5]$ $[3.3, 5]$ $[4.0, 5]$

Σχήμα 5.15: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

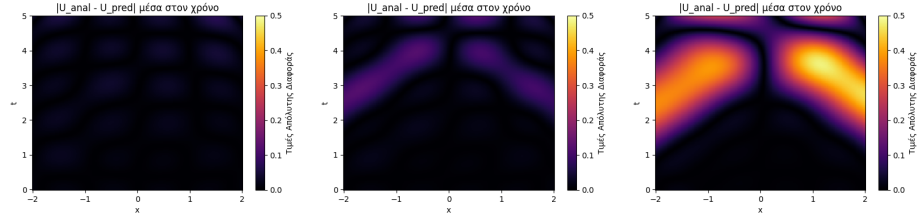
Στο Σχήμα (5.12) παρατηρούμε ότι, για μεγαλύτερες τιμές των α και β η συνολική Loss Function συγκλίνει ταχύτερα στην ελάχιστη τιμή της. Η συμπεριφορά αυτή υποδηλώνει ότι, καθώς αυξάνεται το υποδιάστημα (α, β και περιορίζεται το διαθέσιμο σύνολο εκπαίδευσης, ίσως το δίκτυο δεν μαθαίνει επαρκώς την φυσική του προβλήματος.

Πιο συγκεκριμένα, η ταχύτερη μείωση της loss δεν συνοδεύεται από αντίστοιχη βελτίωση της ποιότητας της προσεγγιστικής λύσης. Αντιθέτως, όπως φαίνεται και στο Σχήμα (5.16), για μεγάλες τιμές των α και β , η ακρίβεια της προσεγγιστικής λύσης μειώνεται αισθητά.



(α') $\bar{u}_\theta(x, t)$ με $\Sigma.\Sigma.$ $[0, 2.4] \cup$ (β') $\bar{u}_\theta(x, t)$ με $\Sigma.\Sigma.$ $[0, 1.7] \cup$ (γ') $\bar{u}_\theta(x, t)$ με $\Sigma.\Sigma.$ $[0, 1.0] \cup$
 $[2.5, 5]$ $[3.3, 5]$ $[4.0, 5]$

Σχήμα 5.16: Γραφήματα της προσέγγισης της λύσης με PINN με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.



(α') $\max|Αναλυτική - PINN|$ με $\Sigma.\Sigma. [0, 2.4] \cup [2.5, 5]$ (β') $\max|Αναλυτική - PINN|$ με $\Sigma.\Sigma. [0, 1.7] \cup [3.3, 5]$ (γ') $\max|Αναλυτική - PINN|$ με $\Sigma.\Sigma. [0, 1.0] \cup [4.0, 5]$

Σχήμα 5.17: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

Στο Σχήμα (5.17) παρατηρούμε ότι υπάρχει ένα όριο στο πόσο μεγάλο μέρος μπορούμε να εξαιρέσουμε από το σύνολο εκπαίδευσης των Συνοριακών Συνθηκών, για το συγκεκριμένο πρόβλημα. Μέχρι ένα σημείο, η προσέγγιση της λύσης διατηρείται σε ικανοποιητικό επίπεδο ακρίβειας, ενώ πέρα από αυτό το όριο η ποιότητα της προσέγγισης υποβαθμίζεται σημαντικά.

Επίσης, παρατηρούμε ότι το σφάλμα εμφανίζεται στις περιοχές των Συνοριακών Συνθηκών όπου δεν υπάρχει πληροφορία και διαδίδεται κατά μήκων των χαρακτηριστικών καμπυλών της εξίσωσης κύματος εντός του χωρίου. Η συμπεριφορά αυτή είναι συνεπής με τη θεωρία των υπερβολικών ΜΔΕ, σύμφωνα με την οποία οι διαταραχές και τα σφάλματα διαδίδονται κατά μήκος των χαρακτηριστικών καμπυλών.

5.5β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.10) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.10

$[0, \alpha] \cup [\beta, T]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[0, 2.5] \cup [2.5, 5]$	2.44856309e-02	1.48896815e-03	0:56	0:21
$[0, 2.4] \cup [2.6, 5]$	2.50813421e-02	1.51053374e-03	0:55	0:20
$[0, 2.3] \cup [2.7, 5]$	2.56735981e-02	1.54433318e-03	0:56	0:21
$[0, 2.2] \cup [2.8, 5]$	2.70866714e-02	1.71691435e-03	0:57	0:21
$[0, 2.1] \cup [2.9, 5]$	2.94078775e-02	1.77068787e-03	0:56	0:21
$[0, 2.0] \cup [3.0, 5]$	3.00042946e-02	1.87165069e-03	0:56	0:21
$[0, 1.9] \cup [3.1, 5]$	2.96860710e-02	1.87256280e-03	0:56	0:21
$[0, 1.8] \cup [3.2, 5]$	2.86112223e-02	1.83000462e-03	0:56	0:20
$[0, 1.7] \cup [3.3, 5]$	2.77646072e-02	1.76048675e-03	0:56	0:20
$[0, 1.6] \cup [3.4, 5]$	2.61319447e-02	1.88598852e-03	0:55	0:20
$[0, 1.5] \cup [3.5, 5]$	2.47068647e-02	2.13388004e-03	0:56	0:20
$[0, 1.4] \cup [3.6, 5]$	2.20699795e-02	2.33902410e-03	0:55	0:20
$[0, 1.3] \cup [3.7, 5]$	1.95084698e-02	2.43085902e-03	0:55	0:20
$[0, 1.2] \cup [3.8, 5]$	1.81370787e-02	2.58681295e-03	0:56	0:20
$[0, 1.1] \cup [3.9, 5]$	1.69670880e-02	2.26387475e-03	0:55	0:20
$[0, 1.0] \cup [4.0, 5]$	1.51036363e-02	1.91149651e-03	0:57	0:20

Πίνακας 5.10: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet με εκπαίδευση σε GPU μέσω CUDA και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές του υποδιαστήματος (α, β) . Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 1 λεπτό, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε λίγο περισσότερο από 20 δευτερόλεπτα, γεγονός που υποδηλώνει περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το περιορισμένο σύνολο των δεδομένων εκπαίδευσης και στη δυσκολία του προβλήματος.

Αντίθετα, κατά την εκτέλεση των υπολογισμών στην CPU, ο χρόνος εκπαίδευσης αυξάνεται, καθώς αυξάνεται το υποδιάστημα (α, β) . Συγκεκριμένα, η μέθοδος Adam κυμαίνεται από περίπου 1:40 έως και 2 λεπτά, ενώ η μέθοδος L-BFGS κυμαίνεται από 16 μέχρι 31 δευτερόλεπτα. Το γεγονός αυτό υποδηλώνει ότι η εκπαίδευση στην CPU είναι πιο ευαίσθητη στη δυσκολία του προβλήματος.

Ως προς την τάξη της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam έχει τάξη μεγέθους που κυμαίνεται από 10^{-2} έως και 10^{-3} και στη μέθοδο L-BFGS κυμαίνεται από 10^{-3} έως και $10^(-4)$. Στην GPU με χρήση CUDA, η μέθοδος Adam έχει σταθερή τάξη μεγέθους 10^{-2} και η μέθοδος L-BFGS οι τιμές συγκεντρώνονται κυρίως γύρω από την τάξη μεγέθους 10^{-3} , τα οποία υποδηλώνουν πιο ομοιόμορφη συμπεριφορά της διαδικασίας βελτιστοποίησης.

Καταλήγουμε στο συμπέρασμα ότι η χρήση της GPU μέσω CUDA υπερτερεί ως προς τον χρόνο εκπαίδευσης, διατηρώντας συγκρίσιμη ακρίβεια με την CPU. Για τον λόγο αυτό, στο συγκεκριμένο πρόβλημα, η GPU αποτελεί πιο κατάλληλη επιλογή για την εκπαίδευση των PINNs, ιδίως όταν έχουμε αυξημένη δυσκολία και περιορισμένο σύνολο δεδομένων εκπαίδευσης

5.6 Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet

Στην Μερική Διαφορική Εξίσωση (5.1) εφαρμόζουμε δύο Αρχικές Συνθήκες (Initial Conditions), δύο Συνοριακές Συνθήκες Dirichlet (Boundary Conditions).

Σε αντίθεση με την προηγούμενη ενότητα, ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN να είναι το μη συνεκτικό ως προς τον χώρο και τον χρόνο

$$x \in \Omega = [-L, \alpha_1] \cup [\beta_1, L], \quad -L < \alpha_1 < \beta_1 < L,$$

$$t \in [0, \alpha_2] \cup [\beta_2, T], \quad 0 < \alpha_2 < \beta_2 < T,$$

$$(5.11) \quad \begin{cases} u_{tt} - c^2 u_{xx} = 0, & x \in [-L, L], t \in [0, T], \\ u(x, 0) = A \sin\left(\frac{\pi}{L}x\right), & x \in [-L, \alpha_1] \cup [\beta_1, L], \\ u_t(x, 0) = 0, & x \in [-L, \alpha_1] \cup [\beta_1, L], \\ u(-L, t) = 0, & t \in [[0, \alpha_2] \cup [\beta_2, T] \\ u(L, t) = 0, & t \in [0, \alpha_2] \cup [\beta_2, T] \end{cases}$$

Για την (5.11) ορίσαμε τις ίδιες μεταβλητές (5.3).

Για την προσέγγιση της λύσης της (5.11) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [68]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

5.6α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Για τις παραμέτρους α_1 , α_2 , β_1 και β_2 επιλέχθηκαν συμμετρικές τιμές ως προς το κέντρο των διαστημάτων $(-L, L)$ και $[0, T]$, ώστε τα αφαιρούμενα υποδιαστήματα (α_1, β_1) και (α_2, β_2) να οδηγούν σε ομοιόμορφη διάσπαση του χωρίου ορισμού.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.11) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.11

5.6 ΚΑΤΑ ΤΜΗΜΑΤΑ ΟΡΙΣΜΕΝΕΣ ΑΡΧΙΚΕΣ ΣΥΝΘΗΚΕΣ ΚΑΙ ΚΑΤΑ ΤΜΗΜΑΤΑ
ΟΡΙΣΜΕΝΕΣ ΣΥΝΟΡΙΑΚΕΣ ΣΥΝΘΗΚΕΣ DIRICHLET · 61

$[-L, \alpha_1] \cup [\beta_1, L]$	$[0, \alpha_2] \cup [\beta_2, T]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-2, -0.0] \cup [0.0, 2]$	$[0, 2.5] \cup [2.5, 5]$	9.68897063e-03	4.86276083e-04	1:44	0:28
$[-2, -0.1] \cup [0.1, 2]$	$[0, 2.4] \cup [2.6, 5]$	9.06492025e-03	4.48508101e-04	1:50	0:28
$[-2, -0.2] \cup [0.2, 2]$	$[0, 2.3] \cup [2.7, 5]$	6.83165481e-03	5.12327417e-04	1:47	0:28
$[-2, -0.3] \cup [0.3, 2]$	$[0, 2.2] \cup [2.8, 5]$	1.53000820e-02	6.10669667e-04	1:46	0:27
$[-2, -0.4] \cup [0.4, 2]$	$[0, 2.1] \cup [2.9, 5]$	2.99940109e-02	1.47155486e-03	1:46	0:27
$[-2, -0.5] \cup [0.5, 2]$	$[0, 2.0] \cup [3.0, 5]$	2.68534701e-02	3.44150187e-03	1:45	0:28
$[-2, -0.6] \cup [0.6, 2]$	$[0, 1.9] \cup [3.1, 5]$	2.37083081e-02	3.51442327e-03	1:44	0:28
$[-2, -0.7] \cup [0.7, 2]$	$[0, 1.8] \cup [3.2, 5]$	1.93158537e-02	4.86279372e-03	1:45	0:28
$[-2, -0.8] \cup [0.8, 2]$	$[0, 1.7] \cup [3.3, 5]$	1.45546179e-02	3.97034362e-03	1:46	0:27
$[-2, -0.9] \cup [0.9, 2]$	$[0, 1.6] \cup [3.4, 5]$	1.12635270e-02	3.11177038e-03	1:43	0:28
$[-2, -1.0] \cup [1.0, 2]$	$[0, 1.5] \cup [3.5, 5]$	9.33856517e-03	2.70897592e-03	1:43	0:27

Πίνακας 5.11: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet με εκπαίδευση σε CPU και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

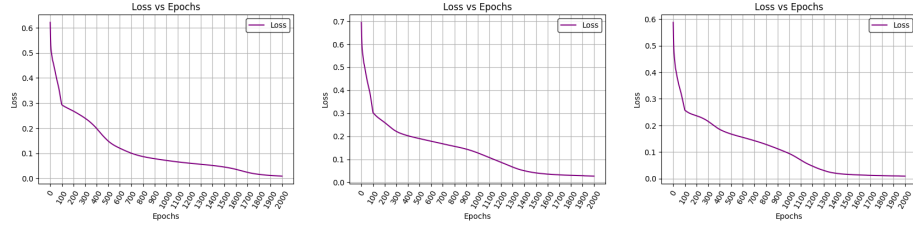
Από τον Πίνακα 5.11 μπορούν να εξαχθούν τα εξής συμπεράσματα

Τα αποτελέσματα της συνολικής Loss Function μέσω της Μεθόδου βελτιστοποίησης Adam παρατηρούμε ότι παραμένουν ουσιαστικά της ίδιας τάξης μεγέθους 10^{-2} , γεγονός που υποδηλώνει ότι η μείωση των δεδομένων εκπαίδευσης, δηλαδή η αύξηση των υποδιαστημάτων (α_1, β_1) των Αρχικών Συνθηκών και (α_2, β_2) των Συνοριακών Συνθηκών, καθιστά το συγκεκριμένο πρόβλημα εξαιρετικά δύσκολο.

Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους που κυμαίνεται από 10^{-4} έως 10^{-3} , το οποίο υποδηλώνει ότι το δίκτυο αντιμετωπίζει την ίδια δυσκολία ως προς την μείωση των δεδομένων εκπαίδευσης.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

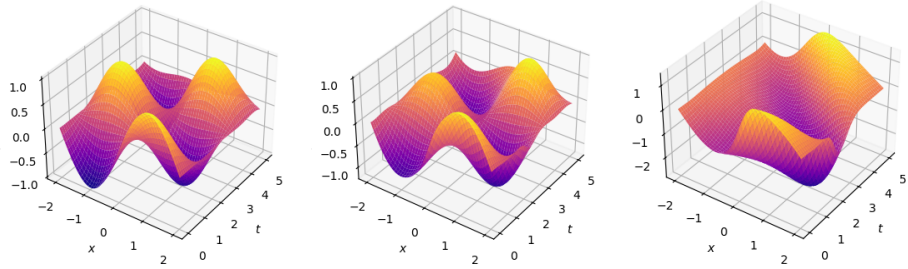
Με βάση τα παραπάνω παίρνουμε τα γραφήματα 5.18, 5.19, 5.20:



(α') Loss με Α.Σ. $[-2, -0.1] \cup [0.1, 2]$ και με Σ.Σ. $[0, 2.4] \cup [2.6, 5]$ (β') Loss με Α.Σ. $[-2, -0.5] \cup [0.5, 2]$ και με Σ.Σ. $[0, 2.0] \cup [3.0, 5]$ (γ') Loss με Α.Σ. $[-2, -1.0] \cup [1.0, 2]$ και με Σ.Σ. $[0, 1.5] \cup [3.5, 5]$

Σχήμα 5.18: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam με κατά Τμήματα Ορισμένες Αρχικές Συνθληκές και με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

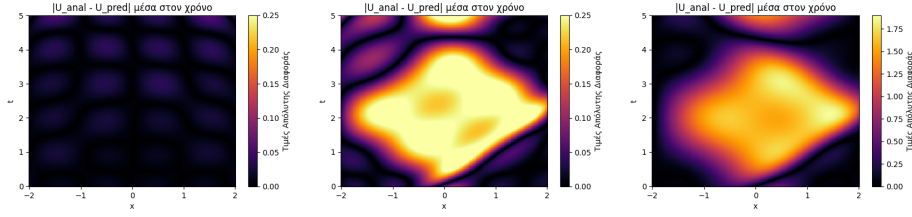
Στο Σχήμα (5.18) παρατηρούμε ότι, για διαφορετικές επιλογές των υποδιαστημάτων (α_1, β_1) και (α_2, β_2) , το PINN εμφανίζει παρόμοια πορεία σύγκλισης της Loss Function κατά τη διάρκεια της εκπαίδευσης. Όμως, όπως φαίνεται στο Σχήμα (5.19), η ελαχιστοποίηση της συνολικής Loss Function, από μόνη της, δεν αποτελεί ούτε μοναδική ούτε επαρκή συνθήκη για να εξασφαλιστεί ότι η εκπαίδευση έγινε προς την σωστή λύση.



(α') $\bar{u}_\theta(x, t)$ με Α.Σ. $[-2, -0.1] \cup [0.1, 2]$ και με Σ.Σ. $[0, 2.4] \cup [2.6, 5]$ (β') $\bar{u}_\theta(x, t)$ με Α.Σ. $[-2, -0.5] \cup [0.5, 2]$ και με Σ.Σ. $[0, 2.0] \cup [3.0, 5]$ (γ') $\bar{u}_\theta(x, t)$ με Α.Σ. $[-2, -1.0] \cup [1.0, 2]$ και με Σ.Σ. $[0, 1.5] \cup [3.5, 5]$

Σχήμα 5.19: Γραφήματα της προσέγγισης της λύσης με PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθληκές και με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

5.6 ΚΑΤΑ ΤΜΗΜΑΤΑ ΟΡΙΣΜΕΝΕΣ ΑΡΧΙΚΕΣ ΣΥΝΘΗΚΕΣ ΚΑΙ ΚΑΤΑ ΤΜΗΜΑΤΑ ΟΡΙΣΜΕΝΕΣ ΣΥΝΟΡΙΑΚΕΣ ΣΥΝΘΗΚΕΣ DIRICHLET · 63



(α') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -0.1] \cup [0.1, 2]$ και με Σ.Σ. $[0, 2.4] \cup [2.6, 5]$ (β') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -0.5] \cup [0.5, 2]$ και με Σ.Σ. $[0, 2.0] \cup [3.0, 5]$ (γ') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -1.0] \cup [1.0, 2]$ και με Σ.Σ. $[0, 1.5] \cup [3.5, 5]$

Σχήμα 5.20: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

Στο Σχήμα (5.20) παρατηρούμε ότι, όσο περιορίζεται το πλήθος των σημείων εκπαίδευσης, ταυτόχρονα από τις Αρχικές και Συνοριακές Συνθήκες το δίκτυο δυσκολεύεται να γενικεύσει την φυσική του προβλήματος.

5.6β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.11) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.12

$[-L, \alpha_1] \cup [\beta_1, L]$	$[0, \alpha_2] \cup [\beta_2, T]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-2, -0.0] \cup [0.0, 2]$	$[0.2, 5] \cup [2.5, 5]$	2.44856309e-02	1.48896815e-03	0:57	0:21
$[-2, -0.1] \cup [0.1, 2]$	$[0.2, 4] \cup [2.6, 5]$	2.87564117e-02	1.45925314e-03	0:58	0:21
$[-2, -0.2] \cup [0.2, 2]$	$[0.2, 3] \cup [2.7, 5]$	2.91415565e-02	1.49823644e-03	0:57	0:21
$[-2, -0.3] \cup [0.3, 2]$	$[0.2, 2] \cup [2.8, 5]$	3.14826332e-02	1.71542820e-03	0:57	0:21
$[-2, -0.4] \cup [0.4, 2]$	$[0.2, 1] \cup [2.9, 5]$	3.26750316e-02	1.88479072e-03	0:57	0:20
$[-2, -0.5] \cup [0.5, 2]$	$[0.2, 0] \cup [3.0, 5]$	3.36062014e-02	2.11706269e-03	0:56	0:20
$[-2, -0.6] \cup [0.6, 2]$	$[0.1, 9] \cup [3.1, 5]$	3.30712050e-02	2.48357886e-03	0:57	0:21
$[-2, -0.7] \cup [0.7, 2]$	$[0.1, 8] \cup [3.2, 5]$	3.01508456e-02	2.18433491e-03	0:57	0:21
$[-2, -0.8] \cup [0.8, 2]$	$[0.1, 7] \cup [3.3, 5]$	2.93342546e-02	2.92569515e-03	0:57	0:21
$[-2, -0.9] \cup [0.9, 2]$	$[0.1, 6] \cup [3.4, 5]$	2.71083824e-02	3.99535429e-03	0:58	0:21
$[-2, -1.0] \cup [1.0, 2]$	$[0.1, 5] \cup [3.5, 5]$	2.40543112e-02	3.37423431e-03	0:58	0:21

Πίνακας 5.12: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet με εκπαίδευση σε GPU μέσω CUDA και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές των υποδιαστημάτων (α_1, β_1) και (α_2, β_2) . Συγκεκριμένα, η μέθοδος Adam απαιτεί

περίπου 1 λεπτό, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε λίγο περισσότερο από 20 δευτερόλεπτα.

Όμοια, κατά την εκτέλεση των υπολογισμών στην CPU, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις. Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 2 λεπτά, ενώ η μέθοδος L-BFGS λίγο λιγότερο από 30 δευτερόλεπτα.

Τα παραπάνω δείχνουν ότι τόσο η χρήση CPU, όσο και η χρήση GPU μέσω CUDA, παρουσιάζουν περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το περιορισμένο σύνολο των δεδομένων εκπαίδευσης και στη δυσκολία του προβλήματος.

Ως προς την τάξη της απόκλισης της Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam κυμαίνεται από 10^{-2} έως και 10^{-3} , ενώ στην GPU με χρήση CUDA έχει σταθερή τάξη μεγέθους 10^{-2} . Επίσης, στη μέθοδο L-BFGS παρατηρούμε ότι στην CPU έχει τάξη μεγέθους που κυμαίνεται από 10^{-3} έως και $10^{(-4)}$, ενώ στην GPU μέσω CUDA έχουμε σχεδόν σταθερή τάξη μεγέθους 10^{-3} .

Καταλήγουμε στο συμπέρασμα ότι η χρήση της GPU μέσω CUDA υπερτερεί ως προς τον χρόνο εκπαίδευσης, διατηρώντας συγκρίσιμη ακρίβεια με την CPU. Για τον λόγο αυτό, στο συγκεκριμένο πρόβλημα, η GPU αποτελεί πιο κατάλληλη επιλογή για την εκπαίδευση των PINNs, ιδίως όταν έχουμε αυξημένη δυσκολία και περιορισμένο σύνολο δεδομένων εκπαίδευσης.

5.7 Συνοριακές Συνθήκες Neumann

Στην Μερική Διαφορική Εξίσωση (5.1) εφαρμόζουμε δύο Αρχικές Συνθήκες (Initial Conditions), δύο Συνοριακές Συνθήκες Neumann (Neumann Boundary Conditions) και ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN.

$$(5.12) \quad \begin{cases} u_{tt} - c^2 u_{xx} = 0, & x \in [-L, L], t \in [0, T], \\ u(x, 0) = A \cos\left(\frac{\pi}{L}x\right), & x \in [-L, L], \\ u_t(x, 0) = -A \frac{\pi}{L} c \cos\left(\frac{\pi}{L}x\right), & x \in [-L, L], \\ u_x(-L, t) = 0, & t \in [0, T] \\ u_x(L, t) = 0, & t \in [0, T] \end{cases}$$

Για την (5.12) ορίσαμε τις ίδιες μεταβλητές (5.3).

Για την προσέγγιση της λύσης της (5.12) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [69]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

5.7α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.12) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.13

N_0	N_b	N_u	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
400	500	2000	6.98457006e-03	1.74223562e-04	1:41	0:26
300	500	2000	1.00350091e-02	2.63118534e-04	1:38	0:26
200	500	2000	3.42225051e-03	1.93546133e-04	1:38	0:26
100	500	2000	2.90841935e-03	1.93094413e-04	1:39	0:27
50	500	2000	2.38975883e-02	3.92448623e-04	1:38	0:26
400	400	2000	9.80733242e-03	2.75927712e-04	1:40	0:26
400	300	2000	2.20031175e-03	1.18393269e-04	1:35	0:26
400	200	2000	2.20862264e-03	1.40724296e-04	1:36	0:26
400	100	2000	1.34390695e-02	4.13578382e-04	1:34	0:25
400	50	2000	7.40299094e-03	2.30543985e-04	1:33	0:25
400	10	2000	1.14702191e-02	1.82733842e-04	1:36	0:25
400	500	1800	1.17031271e-02	2.97342689e-04	1:34	0:25
400	500	1600	4.91290772e-03	3.01454216e-04	1:27	0:24
400	500	1400	5.64272469e-03	2.45716947e-04	1:22	0:22
400	500	1200	2.89714034e-03	9.28647860e-05	1:18	0:21
400	500	1000	1.49676548e-02	1.63097022e-04	1:12	0:19
400	500	800	4.82953526e-03	3.17456201e-04	1:04	0:20
400	500	600	3.80329974e-02	5.70460805e-04	0:59	0:15
400	500	400	1.47588309e-02	2.93831807e-04	0:54	0:14
400	500	200	1.05414856e-02	3.73090123e-04	0:50	0:13
300	400	1500	7.63462251e-03	1.65736623e-04	1:24	0:22
300	400	1000	4.81059216e-03	2.32561040e-04	1:11	0:18
300	400	500	2.53474116e-02	4.13616159e-04	0:55	0:15
200	300	1500	1.26822582e-02	3.69078800e-04	1:22	0:22
200	300	1000	9.45383590e-03	4.51239437e-04	1:09	0:18
200	300	500	4.27541211e-02	6.08145259e-04	0:54	0:14
100	200	1500	1.84416473e-02	2.61760462e-04	1:21	0:22
100	200	1000	1.05356202e-02	9.57405791e-05	1:06	0:18
100	200	500	1.64906010e-02	5.95689984e-04	0:52	0:13
50	100	1500	9.15895309e-03	2.32491846e-04	1:17	0:21
50	100	1000	4.53382321e-02	3.05888650e-04	1:04	0:17
50	100	500	4.31020632e-02	3.90670088e-04	0:51	0:13

Πίνακας 5.13: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Συνοριακές Συνθήκες Neumann με εκπαίδευση σε CPU

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Από τον Πίνακα 5.13 μπορούν να εξαχθούν τα εξής συμπεράσματα

Τα αποτελέσματα της συνολικής Loss Function μέσω της Μεθόδου βελτιστοποίησης Adam παρατηρούμε ότι οι τάξεις μεγέθους κυμαίνονται από 10^{-2} έως και 10^{-3} , για όλες τις εξεταζόμενες επιλογές σημείων εκπαίδευσης. Όμως, παρατηρούνται διακυμάνσεις που υποδηλώνουν ότι η σύγκλιση της Adam επηρεάζεται από τη σύνθεση του συνόλου εκπαίδευσης και όχι μόνο από το συνολικό πλήθος των σημείων.

Ως προς τον υπολογιστικό χρόνο της Adam, προκύπτει ότι η αύξηση του πλήθους των σημείων εκπαίδευσης οδηγεί γενικά σε μεγαλύτερους χρόνους εκπαίδευσης, με τον αριθμό των collocation points N_u να αποτελεί τον κυρίαρχο παράγοντα κόστους.

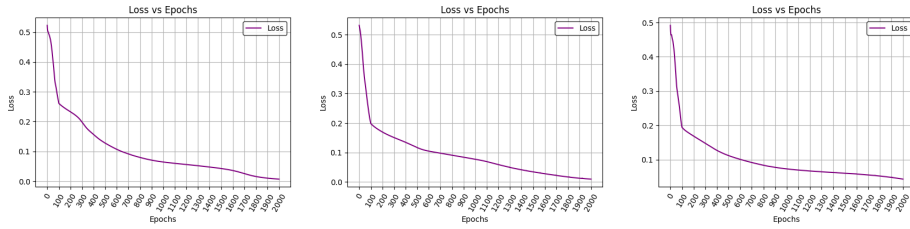
Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν ίδια τάξη μεγέθους, 10^{-4} .

Ο συνολικός χρόνος εκπαίδευσης της μεθόδου L-BFGS κυμαίνεται εντός ενός σχετικά στενού χρονικού διαστήματος, γεγονός που υποδηλώνει ότι επηρεάζεται σε μικρότερο βαθμό από το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Τα αποτελέσματα δείχνουν ότι η αύξηση του πλήθους των σημείων εκπαίδευσης δεν οδηγεί κατ' ανάγκη σε μείωση της συνολικής Loss Function, ενώ ταυτόχρονα μπορεί να αυξήσουν το υπολογιστικό κόστος σημαντικά, για το συγκεκριμένο πρόβλημα.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

Με βάση τα παραπάνω παίρνουμε τα γραφήματα 5.21, 5.22, 5.23 και 5.24:



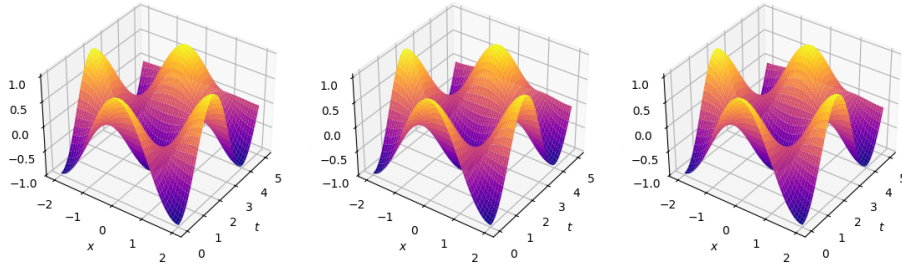
(α') Loss με 400,500,2000

(β') Loss με 200,300,1000

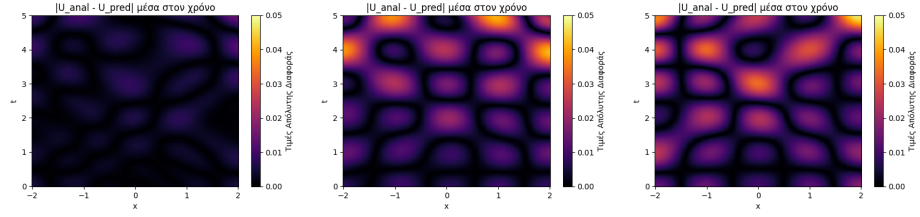
(γ') Loss με 50,100,500

Σχήμα 5.21: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (5.21) παρατηρούμε ότι, για τις διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης, η εξέλιξη της συνολικής συνάρτησης απώλειας ως προς τις εποχές παρουσιάζει παρόμοια συμπεριφορά σύγκλισης. Το γεγονός αυτό υποδηλώνει ότι το δίκτυο μπορεί να εκπαιδευτεί αποτελεσματικά χωρίς την ανάγκη πολύ μεγάλου αριθμού σημείων, διατηρώντας την ακρίβεια της προσέγγισης, όπως φαίνεται και στο Σχήμα (5.22).

(α') $\bar{u}_\theta(x, t)$ για 400,500,2000(β') $\bar{u}_\theta(x, t)$ για 200,300,1000(γ') $\bar{u}_\theta(x, t)$ για 50,100,500

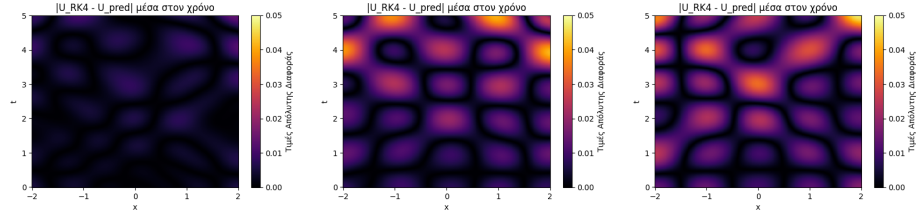
Σχήμα 5.22: Γραφήματα της προσέγγισης της λύσης με PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500



(α') $\max|Αναλυτική - PINN|$ για 400,500,2000 (β') $\max|Αναλυτική - PINN|$ για 200,300,1000 (γ') $\max|Αναλυτική - PINN|$ για 50,100,500

Σχήμα 5.23: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (5.23) παρατηρούμε ότι, ακόμη και για σχετικά μικρό αριθμό σημείων εκπαίδευσης, η προσεγγιστική λύση ακολουθεί ικανοποιητικά τη συμπεριφορά της αναλυτικής λύσης σε όλο τον εξεταζόμενο χωροχρόνο, με τις αποκλίσεις να παραμένουν αόρατες οπτικά.



(α') $\max|RK4 - PINN|$ για 400,500,2000 (β') $\max|RK4 - PINN|$ για 200,300,1000 (γ') $\max|RK4 - PINN|$ για 50,100,500

Σχήμα 5.24: Μέγιστη Απόλυτη Διαφορά μεταξύ της Προσεγγιστικής Λύσης RK4 και της Προσεγγιστικής Λύσης PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (5.24) παρατηρούμε ότι τα γραφήματα της μέγιστης απόλυτης διαφοράς μεταξύ της μεθόδου Runge–Kutta 4 και του PINN και εκείνα της μέγιστης απόλυτης διαφοράς μεταξύ της αναλυτικής λύσης και του PINN είναι πρακτικά ταυτόσημα. Το αποτέλεσμα αυτό υποδηλώνει ότι η μέθοδος Runge–Kutta 4 προσεγγίζει ουσιαστικά την αναλυτική λύση και παρέχει καλύτερη προσέγγιση από το PINN για το συγκεκριμένο πρόβλημα. Για την μέθοδο Πεπερασμένων Διαφορών αναπτύχθηκε το πρόγραμμα [70]]

5.7β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.12) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.14

N_0	N_b	N_u	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
400	500	2000	4.28482406e-02	1.07611518e-03	1:06	0:23
300	500	2000	4.01274562e-02	1.00856787e-03	1:06	0:24
200	500	2000	4.47437428e-02	1.01694954e-03	1:06	0:24
100	500	2000	4.68303002e-02	1.02386170e-03	1:05	0:23
50	500	2000	4.13187854e-02	5.74182719e-04	1:05	0:23
400	400	2000	4.13339287e-02	1.02055701e-03	1:06	0:24
400	300	2000	4.34756726e-02	1.06491463e-03	1:06	0:23
400	200	2000	4.62174900e-02	1.09128398e-03	1:06	0:23
400	100	2000	4.70943078e-02	1.17238564e-03	1:07	0:25
400	50	2000	4.84037064e-02	1.18533184e-03	1:06	0:24
400	10	2000	2.48014908e-02	4.48631967e-04	1:05	0:23
400	500	1800	3.57976668e-02	1.11702201e-03	1:06	0:23
400	500	1600	3.80482264e-02	1.17295049e-03	1:05	0:23
400	500	1400	3.66927609e-02	9.97438096e-04	1:04	0:24
400	500	1200	3.56457606e-02	1.24247279e-03	1:06	0:24
400	500	1000	4.32214253e-02	1.02588546e-03	1:06	0:24
400	500	800	5.12723066e-02	8.93713150e-04	1:06	0:24
400	500	600	4.75727953e-02	1.09728926e-03	1:06	0:23
400	500	400	4.85267155e-02	8.55918333e-04	1:05	0:24
400	500	200	4.19162102e-02	1.39101094e-03	1:06	0:24
300	400	1500	3.22782770e-02	1.04594219e-03	1:06	0:24
300	400	1000	3.75891328e-02	1.03637564e-03	1:07	0:24
300	400	500	4.45895195e-02	1.21615164e-03	1:06	0:24
200	300	1500	4.25201841e-02	9.86763858e-04	1:05	0:24
200	300	1000	3.76531333e-02	1.11049355e-03	1:07	0:24
200	300	500	3.77686881e-02	8.51074583e-04	1:06	0:23
100	200	1500	5.01840860e-02	1.43260416e-03	1:06	0:24
100	200	1000	4.39028889e-02	1.21168594e-03	1:06	0:23
100	200	500	3.83547284e-02	6.67726446e-04	1:05	0:24
50	100	1500	4.26461883e-02	6.99495431e-04	1:05	0:24
50	100	1000	3.84761244e-02	5.61491936e-04	1:07	0:23
50	100	500	2.64298934e-02	5.27170079e-04	1:05	0:24

Πίνακας 5.14: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Συνοριακές Συνθήκες Neumann με εκπαίδευση σε GPU μέσω CUDA

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης. Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 1 λεπτό, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε λίγο περισσότερο από 20 δευτερόλεπτα, γεγονός που υποδηλώνει περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Αντίθετα, κατά την εκτέλεση των υπολογισμών στην CPU, ο χρόνος εκπαίδευσης εξαρτάται από το πλήθος και την σύνθεση των σημείων εκπαίδευσης, στο συγκεκριμένο πρόβλημα. Συγκεκριμένα, η βελτιστοποίηση με Adam απαιτεί από περίπου 1:40 έως περίπου 2 λεπτά, ενώ η L-BFGS κυμαίνεται από 13 μέχρι 26 δευτερόλεπτα.

Ως προς την τάξη της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam τάξη μεγέθους που κυμαίνεται από 10^{-2} έως και 10^{-3} , ενώ στην μέθοδο L-BFGS οι τιμές συγκεντρώνονται κυρίως γύρω από την τάξη μεγέθους 10^{-4} .

Στην GPU μέσω CUDA τόσο στην μέθοδο Adam όσο και στην μέθοδο L-BFGS παρατηρείται σταθερότητα ως προς τις τάξεις μεγέθους, με 10^{-2} και 10^{-3} αντίστοιχα, γεγονός που υποδηλώνει πιο ομοιόμορφη συμπεριφορά της διαδικασίας βελτιστοποίησης.

Καταλήγουμε στο συμπέρασμα ότι, η χρήση CPU είναι επαρκής για προβλήματα μικρού συνόλου δεδομένων, καθώς επιτυγχάνονται συγκρίσιμες τιμές της συνολικής Loss Function. Ωστόσο, καθώς αυξάνεται το πλήθος των σημείων εκπαίδευσης, ο χρόνος εκπαίδευσης στην CPU αυξάνεται αισθητά. Αντίθετα, η GPU με χρήση CUDA επιτρέπει ταχύτερη και πιο αποδοτική εκπαίδευση, με τον χρόνο να παραμένει σχεδόν σταθερός ακόμη και για μεγαλύτερα σύνολα δεδομένων, χωρίς να επηρεάζεται αρνητικά η σύγκλιση της συνάρτησης απώλειας. Για τον λόγο αυτό, στο συγκεκριμένο πρόβλημα, η GPU αποτελεί πιο κατάλληλη επιλογή για την εκπαίδευση των PINNs, ιδίως όταν έχουμε μεγάλο αριθμό σημείων εκπαίδευσης.

5.8 Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Συνοριακές Συνθήκες Neumann

Στην Μερική Διαφορική Εξίσωση (5.1) εφαρμόζουμε δύο Αρχικές Συνθήκες (Initial Conditions), δύο Συνοριακές Συνθήκες Neumann (Neumann Boundary Conditions).

Ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN να είναι το μη συνεκτικό ως προς τον χώρο

$$x \in \Omega = [-L, \alpha] \cup [\beta, L], \quad -L < \alpha < \beta < L,$$

$$(5.13) \quad \begin{cases} u_{tt} - c^2 u_{xx} = 0, & x \in [-L, L], t \in [0, T], \\ u(x, 0) = A \cos\left(\frac{\pi}{L}x\right), & x \in [-L, \alpha] \cup [\beta, L], \\ u_t(x, 0) = 0, & x \in [-L, \alpha] \cup [\beta, L], \\ u_x(-L, t) = 0, & t \in [0, T] \\ u_x(L, t) = 0, & t \in [0, T] \end{cases}$$

Για την (5.13) ορίσαμε τις ίδιες μεταβλητές (5.3).

Για την προσέγγιση της λύσης της (5.9) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [71]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

5.8α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Για τις παραμέτρους α και β επιλέχθηκαν συμμετρικές τιμές ως προς το κέντρο του διαστήματος $[-L, L]$, ώστε το αφαιρούμενο υποδιάστημα (α, β) να οδηγεί σε μοιόμορφη διάσπαση του χωρίου ορισμού.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.13) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.15

$[-L, \alpha] \cup [\beta, L]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-2, -0.0] \cup [0.0, 2]$	6.98457006e-03	1.74223562e-04	1:49	0:27
$[-2, -0.1] \cup [0.1, 2]$	1.88153647e-02	3.04723711e-04	1:44	0:28
$[-2, -0.2] \cup [0.2, 2]$	1.58952419e-02	1.17518292e-04	1:46	0:27
$[-2, -0.3] \cup [0.3, 2]$	4.04588170e-02	3.30122915e-04	1:44	0:27
$[-2, -0.4] \cup [0.4, 2]$	3.94476354e-02	2.02716299e-04	1:43	0:27
$[-2, -0.5] \cup [0.5, 2]$	4.46210839e-02	4.99310903e-04	1:47	0:27
$[-2, -0.6] \cup [0.6, 2]$	5.05735762e-02	1.25195575e-03	1:45	0:27
$[-2, -0.7] \cup [0.7, 2]$	2.59714965e-02	1.97997404e-04	1:43	0:27
$[-2, -0.8] \cup [0.8, 2]$	3.29439417e-02	2.25901604e-04	1:44	0:27
$[-2, -0.9] \cup [0.9, 2]$	3.06956545e-02	5.97886159e-04	1:44	0:26
$[-2, -1.0] \cup [1.0, 2]$	2.76120603e-02	5.25231997e-04	1:44	0:26

Πίνακας 5.15: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Συνοριακές Συνθήκες Neumann με εκπαίδευση σε CPU και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

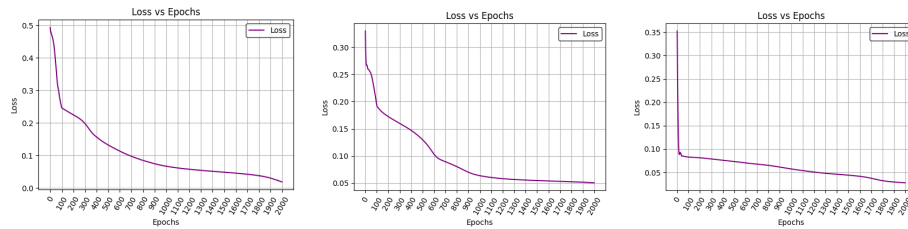
Από τον Πίνακα 5.15 μπορούν να εξαχθούν τα εξής συμπεράσματα

Από τα αποτελέσματα της συνολικής Loss Function, μέσω της βελτιστοποίησης Adam, παρατηρούμε ότι υπάρχει σταδιακή αύξηση της τάξης μεγέθους, από 10^{-3} σε 10^{-2} , γεγονός που υποδηλώνει ότι η μείωση των δεδομένων εκπαίδευσης, δηλαδή η αύξηση του υποδιαστήματος (α, β) , των Αρχικών Συνθηκών καθιστά το συγκεκριμένο πρόβλημα δυσκολότερο για το δίκτυο.

Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους 10^{-4} , το οποίο υποδηλώνει ότι το δίκτυο αντιμετωπίζει επαρκώς την δυσκολία ως προς την μείωση των δεδομένων εκπαίδευσης. Αντίθετα, ο χρόνος εκπαίδευσης της μεθόδου L-BFGS παραμένει συγκριτικά πιο σταθερός, με μικρές μόνο διακυμάνσεις.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

Με βάση τα παραπάνω παίρνουμε τα γραφήματα 5.25, 5.26, 5.27:

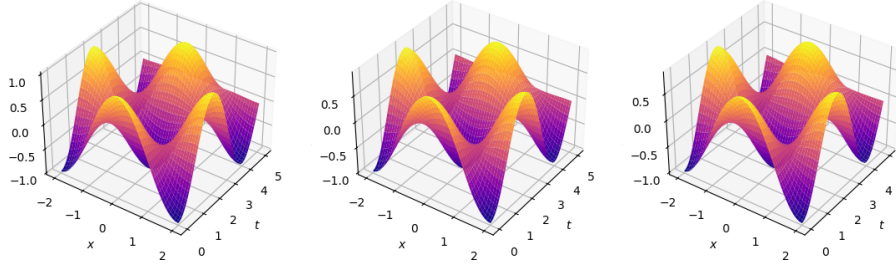


(α') Loss με Α.Σ. (β') Loss με Α.Σ. (γ') Loss με Α.Σ.
 $[-2, -0.1] \cup [0.1, 2]$ $[-2, -0.6] \cup [0.6, 2]$ $[-2, -1.0] \cup [1.0, 2]$

Σχήμα 5.25: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

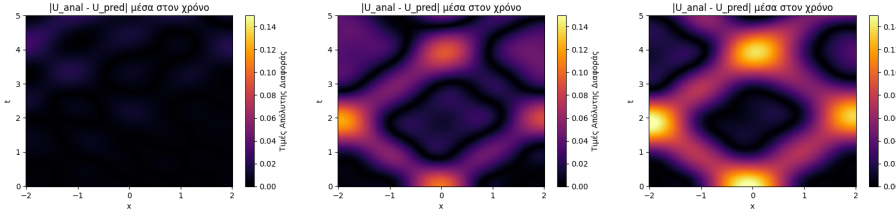
Στο Σχήμα (5.25) παρατηρούμε ότι, για μεγαλύτερες τιμές των α και β η συνολική Loss Function συγκλίνει ταχύτερα στην ελάχιστη τιμή της. Η συμπεριφορά αυτή υποδηλώνει ότι, καθώς αυξάνεται το υποδιάστημα (α, β και περιορίζεται το διαθέσιμο σύνολο εκπαίδευσης, το δίκτυο ίσως δεν μαθαίνει επαρκώς την φυσική του προβλήματος.

Πιο συγκεκριμένα, η ταχύτερη μείωση της loss δεν συνοδεύεται από αντίστοιχη βελτίωση της ποιότητας της προσεγγιστικής λύσης. Αντιθέτως, όπως φαίνεται και στο Σχήμα (5.26), για μεγάλες τιμές των α και β , η ακρίβεια της προσεγγιστικής λύσης μειώνεται αισθητά.



(α') $\bar{u}_\theta(x, t)$ με Α.Σ. $[-2, -0.1] \cup [0.1, 2]$ (β') $\bar{u}_\theta(x, t)$ με Α.Σ. $[-2, -0.6] \cup [0.6, 2]$ (γ') $\bar{u}_\theta(x, t)$ με Α.Σ. $[-2, -1.0] \cup [1.0, 2]$

Σχήμα 5.26: Γραφήματα της προσέγγισης της λύσης με PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.



(α') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -0.1] \cup [0.1, 2]$ (β') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -0.6] \cup [0.6, 2]$ (γ') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -1.0] \cup [1.0, 2]$

Σχήμα 5.27: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

Στο Σχήμα (5.27) παρατηρούμε ότι υπάρχει ένα όριο στο πόσο μεγάλο μέρος μπορούμε να εξαιρέσουμε από το σύνολο εκπαίδευσης των Συνοριακών Συνθηκών, για το συγκεκριμένο πρόβλημα. Μέχρι ένα σημείο, η προσέγγιση της λύσης διατηρείται σε ικανοποιητικό επίπεδο ακρίβειας, ενώ πέρα από αυτό το όριο η ποιότητα της προσέγγισης υποβαθμίζεται σημαντικά.

Παρατηρούμε ότι το σφάλμα εμφανίζεται αρχικά στην περιοχή της Αρχικής Συνθήκης, όπου η πληροφορία δεν είναι διαθέσιμη και διαδίδεται κατά μήκων των χαρακτηριστικών καμπυλών της εξίσωσης κύματος μέσα στο χωρίο. Έπειτα, όταν φτάνει στα χωρικά σύνορα, ανακλάται και συνεχίζει να εξελίσσεται στο πεδίο. Η συμπεριφορά αυτή είναι συμβατή με τη φύση των υπερβολικών ΜΔΕ.

5.8β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.13) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.16

$[-L, \alpha] \cup [\beta, L]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-2, -0.0] \cup [0.0, 2]$	4.28482406e-02	1.07611518e-03	1:06	0:23
$[-2, -0.1] \cup [0.1, 2]$	4.29153517e-02	1.02367019e-03	1:07	0:24
$[-2, -0.2] \cup [0.2, 2]$	3.12174298e-02	8.50698503e-04	1:08	0:24
$[-2, -0.3] \cup [0.3, 2]$	3.84810902e-02	8.79824511e-04	1:07	0:23
$[-2, -0.4] \cup [0.4, 2]$	5.57724200e-02	1.69184036e-03	1:07	0:24
$[-2, -0.5] \cup [0.5, 2]$	4.02489826e-02	6.11176132e-04	1:07	0:23
$[-2, -0.6] \cup [0.6, 2]$	2.57051941e-02	5.03080781e-04	1:07	0:23
$[-2, -0.7] \cup [0.7, 2]$	3.12444158e-02	3.92450602e-04	1:07	0:23
$[-2, -0.8] \cup [0.8, 2]$	3.32710072e-02	3.45226465e-04	1:08	0:23
$[-2, -0.9] \cup [0.9, 2]$	3.40956040e-02	6.46916626e-04	1:07	0:23
$[-2, -1.0] \cup [1.0, 2]$	3.68997678e-02	8.11621780e-04	1:07	0:23

Πίνακας 5.16: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Συνοριακές Συνθήκες Neumann με εκπαίδευση σε GPU μέσω CUDA και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές του υποδιαστήματος (α, β) . Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 1 λεπτό, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε λίγο περισσότερο από 20 δευτερόλεπτα.

Όμοια, στην CPU, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές του υποδιαστήματος (α, β) . Συγκεκριμένα, η βελτιστοποίηση με Adam απαιτεί λιγότερο από 2 λεπτά, ενώ η L-BFGS λίγο λιγότερο από 30 δευτερόλεπτα.

Τόσο με χρήση CPU, όσο και με χρήση GPU μέσω CUDA, παρατηρούμε περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το περιορισμένο σύνολο των δεδομένων εκπαίδευσης και στη δυσκολία του προβλήματος.

Ως προς την τάξη της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam τάξει μεγέθους που κυμαίνεται από 10^{-2} έως και 10^{-3} , ενώ στην μέθοδο L-BFGS οι τιμές συγκεντρώνονται κυρίως γύρω από την τάξη μεγέθους 10^{-4} .

Στην GPU μέσω CUDA τόσο στην μέθοδο Adam όσο και στην μέθοδο L-BFGS παρατηρείται σταθερότητα ως προς τις τάξεις μεγέθους, με 10^{-2} και 10^{-3} αντίστοιχα, γεγονός που υποδηλώνει πιο ομοιόμορφη συμπεριφορά της διαδικασίας βελτιστοποίησης.

Συνολικά, καταλήγουμε στο συμπέρασμα ότι η CPU μπορεί, σε ορισμένες περιπτώσεις, να επιτύχει χαμηλότερες τελικές τιμές Loss, με κόστος όμως αυξημένου χρόνου εκπαίδευσης. Αντίθετα, η GPU με χρήση CUDA προσφέρει σημαντική επιτάχυνση και σταθερότητα στον χρόνο εκπαίδευσης, επιτυγχάνοντας συγκρίσιμες τάξεις απόκλισης, γεγονός που την καθιστά ιδιαίτερα αποδοτική για μεγαλύτερα σύνολα δεδομένων στο συγκεκριμένο πρόβλημα.

5.9 Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Neumann

Στην Μερική Διαφορική Εξίσωση (5.1) εφαρμόζουμε δύο Αρχικές Συνθήκες (Initial Conditions), δύο Συνοριακές Συνθήκες Neumann (Neumann Boundary

Conditions).

Σε αντίθεση με την προηγούμενη ενότητα, ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN να είναι το μη συνεκτικό ως προς τον χρόνο

$$t \in [0, \alpha] \cup [\beta, T], \quad 0 < \alpha < \beta < T,$$

$$(5.14) \quad \begin{cases} u_{tt} - c^2 u_{xx} = 0, & x \in [-L, L], t \in [0, T], \\ u(x, 0) = A \cos\left(\frac{\pi}{L}x\right), & x \in [-L, L], \\ u_t(x, 0) = 0, & x \in [-L, L], \\ u_x(-L, t) = 0, & t \in [0, \alpha] \cup [\beta, T] \\ u_x(L, t) = 0, & ; t \in [0, \alpha] \cup [\beta, T] \end{cases}$$

Για την (5.14) ορίσαμε τις ίδιες μεταβλητές (5.3).

Για την προσέγγιση της λύσης της (5.14) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [72]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

5.9α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Για τις παραμέτρους α και β επιλέχθηκαν συμμετρικές τιμές ως προς το κέντρο του διαστήματος $[0, T]$, ώστε το αφαιρούμενο υποδιάστημα (α, β) να οδηγεί σε ομοιόμορφη διάσπαση του χωρίου ορισμού.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.10) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.17

$[0, \alpha] \cup [\beta, T]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[0, 2.5] \cup [2.5, 5]$	6.98457006e-03	1.74223562e-04	1:50	0:29
$[0, 2.4] \cup [2.6, 5]$	7.31440261e-03	1.65125908e-04	1:53	0:31
$[0, 2.3] \cup [2.7, 5]$	8.83689802e-03	1.60480660e-04	1:56	0:29
$[0, 2.2] \cup [2.8, 5]$	1.24273598e-02	1.60491414e-04	1:57	0:32
$[0, 2.1] \cup [2.9, 5]$	1.71499643e-02	1.45497572e-04	1:57	0:30
$[0, 2.0] \cup [3.0, 5]$	2.27288809e-02	1.48281673e-04	1:54	0:29
$[0, 1.9] \cup [3.1, 5]$	3.08975857e-02	1.53578876e-04	1:53	0:30
$[0, 1.8] \cup [3.2, 5]$	4.35937829e-02	2.68986623e-04	1:51	0:30
$[0, 1.7] \cup [3.3, 5]$	3.96093875e-02	5.48232289e-04	1:54	0:29
$[0, 1.6] \cup [3.4, 5]$	3.48140746e-02	5.54415572e-04	1:51	0:29
$[0, 1.5] \cup [3.5, 5]$	3.11712064e-02	9.67046712e-04	1:54	0:29
$[0, 1.4] \cup [3.6, 5]$	2.94328295e-02	1.29943248e-03	1:52	0:29
$[0, 1.3] \cup [3.7, 5]$	2.58431695e-02	1.95341557e-03	1:54	0:30
$[0, 1.2] \cup [3.8, 5]$	2.17188597e-02	1.28904311e-03	1:55	0:30
$[0, 1.1] \cup [3.9, 5]$	2.00256277e-02	1.82822801e-03	1:51	0:30
$[0, 1.0] \cup [4.0, 5]$	1.49191543e-02	4.08099359e-03	1:51	0:29

Πίνακας 5.17: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Neumann με εκπαίδευση σε CPU και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Από τον Πίνακα 5.17 μπορούν να εξαχθούν τα εξής συμπεράσματα

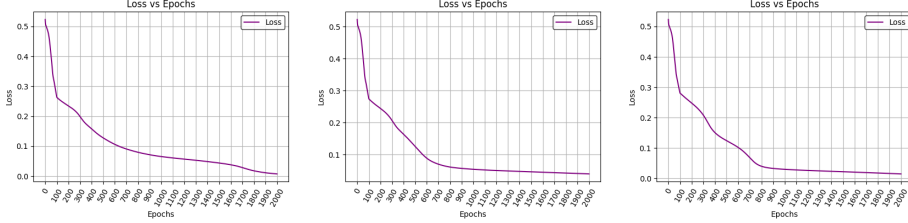
Από τα αποτελέσματα της συνολικής Loss Function, μέσω της βελτιστοποίησης Adam, παρατηρούμε ότι υπάρχει σταδιακή αύξηση της τάξης μεγέθους, από 10^{-3} σε 10^{-2} , γεγονός που υποδηλώνει ότι η μείωση των δεδομένων εκπαίδευσης, δηλαδή η αύξηση του υποδιαστήματος (α, β) , των Συνοριακών Συνθηκών καθιστά το συγκεκριμένο πρόβλημα δυσκολότερο για το δίκτυο με μεγαλύτερο υπολογιστικό κόστος.

Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους που κυμαίνεται από 10^{-4} έως 10^{-3} , το οποίο υποδηλώνει ότι το δίκτυο αντιμετωπίζει την ίδια δυσκολία ως προς την μείωση των δεδομένων εκπαίδευσης. Αντίθετα, ο χρόνος εκπαίδευσης της μεθόδου L-BFGS παραμένει συγκριτικά πιο σταθερός, με μικρές μόνο διακυμάνσεις.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με

την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

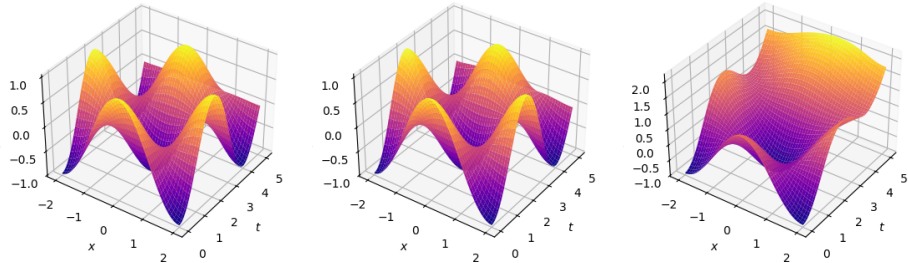
Με βάση τα παραπάνω παίρνουμε τα γραφήματα 5.28, 5.29, 5.30:



(α') Loss με $\Sigma. \Sigma.$ $[0, 2.4] \cup$ (β') Loss με $\Sigma. \Sigma.$ $[0, 1.7] \cup$ (γ') Loss με $\Sigma. \Sigma.$ $[0, 1.0] \cup$
 $[2.5, 5]$ $[3.3, 5]$ $[4.0, 5]$

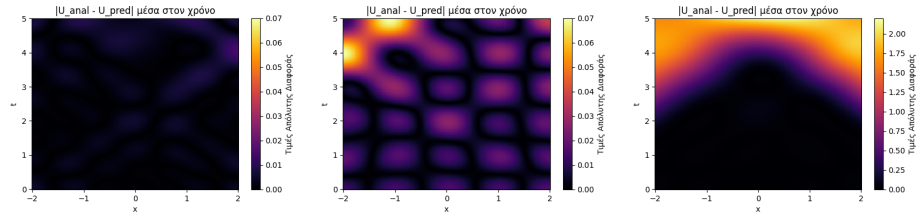
Σχήμα 5.28: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Neumann. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

Στο Σχήμα (5.25) παρατηρούμε ότι, για διαφορετικές επιλογές του υποδιαστήματος (α, β) , το PINN εμφανίζει παρόμοια πορεία σύγκλισης της Loss Function κατά τη διάρκεια της εκπαίδευσης. Όμως, όπως φαίνεται στο Σχήμα (5.26), η ελαχιστοποίηση της συνολικής Loss Function, από μόνη της, δεν αποτελεί ούτε μοναδική ούτε επαρκή συνθήκη για να εξασφαλιστεί ότι η εκπαίδευση έγινε προς την σωστή λύση.



(α') $\bar{u}_\theta(x, t)$ με $\Sigma. \Sigma.$ $[0, 2.4] \cup$ (β') $\bar{u}_\theta(x, t)$ με $\Sigma. \Sigma.$ $[0, 1.7] \cup$ (γ') $\bar{u}_\theta(x, t)$ με $\Sigma. \Sigma.$ $[0, 1.0] \cup$
 $[2.5, 5]$ $[3.3, 5]$ $[4.0, 5]$

Σχήμα 5.29: Γραφήματα της προσέγγισης της λύσης με PINN με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Neumann. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.



(α') $\max|Αναλυτική - PINN|$ με Σ.Σ. $[0, 2.4] \cup [2.5, 5]$ (β') $\max|Αναλυτική - PINN|$ με Σ.Σ. $[0, 1.7] \cup [3.3, 5]$ (γ') $\max|Αναλυτική - PINN|$ με Σ.Σ. $[0, 1.0] \cup [4.0, 5]$

Σχήμα 5.30: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Neumann. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

Στο Σχήμα (5.30) παρατηρούμε ότι υπάρχει ένα όριο στο πόσο μεγάλο μέρος μπορούμε να εξαιρέσουμε από το σύνολο εκπαίδευσης των Συνοριακών Συνθηκών, για το συγκεκριμένο πρόβλημα. Μέχρι ένα σημείο, η προσέγγιση της λύσης διατηρείται σε ικανοποιητικό επίπεδο ακρίβειας, ενώ πέρα από αυτό το όριο η ποιότητα της προσέγγισης υποβαθμίζεται σημαντικά.

5.9β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.14) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.18

$[0, \alpha] \cup [\beta, T]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[0, 2.5] \cup [2.5, 5]$	4.28482406e-02	1.07611518e-03	1:08	0:24
$[0, 2.4] \cup [2.6, 5]$	4.27463874e-02	1.14477496e-03	1:08	0:24
$[0, 2.3] \cup [2.7, 5]$	4.23103347e-02	1.09001691e-03	1:07	0:24
$[0, 2.2] \cup [2.8, 5]$	4.24348228e-02	1.02450443e-03	1:07	0:24
$[0, 2.1] \cup [2.9, 5]$	4.51534726e-02	9.74925410e-04	1:07	0:24
$[0, 2.0] \cup [3.0, 5]$	4.90196310e-02	9.12898686e-04	1:08	0:24
$[0, 1.9] \cup [3.1, 5]$	4.97441888e-02	9.90069588e-04	1:07	0:24
$[0, 1.8] \cup [3.2, 5]$	4.54315394e-02	1.26775447e-03	1:07	0:24
$[0, 1.7] \cup [3.3, 5]$	4.33102362e-02	1.25025678e-03	1:07	0:23
$[0, 1.6] \cup [3.4, 5]$	3.98892239e-02	1.38213509e-03	1:20	0:24
$[0, 1.5] \cup [3.5, 5]$	3.69833149e-02	1.72209158e-03	1:08	0:24
$[0, 1.4] \cup [3.6, 5]$	3.11040133e-02	2.91411788e-03	1:08	0:24
$[0, 1.3] \cup [3.7, 5]$	2.52481773e-02	4.14686929e-03	1:08	0:24
$[0, 1.2] \cup [3.8, 5]$	2.22011227e-02	4.57064668e-03	1:08	0:24
$[0, 1.1] \cup [3.9, 5]$	1.99082922e-02	4.49892320e-03	1:08	0:24
$[0, 1.0] \cup [4.0, 5]$	1.61761101e-02	3.17590311e-03	1:06	0:23

Πίνακας 5.18: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Neumann με εκπαίδευση σε GPU μέσω CUBA και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης. Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 1 λεπτό, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε λίγο περισσότερο από 20 δευτερόλεπτα, γεγονός που υποδηλώνει περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Όμοια, στην CPU, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές του υποδιαστήματος (α, β) . Συγκεκριμένα, η βελτιστοποίηση με Adam απαιτεί λιγότερο από 2 λεπτά, ενώ η L-BFGS λίγο περίπου 30 δευτερόλεπτα.

Τόσο με χρήση CPU, όσο και με χρήση GPU μέσω CUDA, παρατηρούμε περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το περιορισμένο σύνολο των δεδομένων εκπαίδευσης και στη δυσκολία του προβλήματος.

Ως προς την τάξη της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam τάξη μεγέθους που κυμαίνεται από 10^{-2} έως και 10^{-3} , ενώ στην μέθοδο L-BFGS οι τιμές συγκεντρώνονται κυρίως γύρω από την τάξη μεγέθους 10^{-4} .

Στην GPU μέσω CUDA τόσο στην μέθοδο Adam όσο και στην μέθοδο L-BFGS παρατηρείται σταθερότητα ως προς τις τάξεις μεγέθους, με 10^{-2} και 10^{-3} αντίστοιχα, γεγονός που υποδηλώνει πιο ομοιόμορφη συμπεριφορά της διαδικασίας βελτιστοποίησης.

Συνολικά, καταλήγουμε στο συμπέρασμα ότι η CPU μπορεί, σε ορισμένες περιπτώσεις, να επιτύχει χαμηλότερες τελικές τιμές Loss, με κόστος όμως αυξημένου χρόνου εκπαίδευσης. Αντίθετα, η GPU με χρήση CUDA προσφέρει σημαντική επιτάχυνση και σταθερότητα στον χρόνο εκπαίδευσης, επιτυγχάνοντας συγκρίσιμες τάξεις απόκλισης, γεγονός που την καθιστά ιδιαίτερα αποδοτική για μεγαλύτερα σύνολα δεδομένων στο συγκεκριμένο πρόβλημα.

5.10 Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Dirichlet

Στην Μερική Διαφορική Εξίσωση (5.1) εφαρμόζουμε δύο Αρχικές Συνθήκες (Initial Conditions), δύο Συνοριακές Συνθήκες Dirichlet (Boundary Conditions).

Σε αντίθεση με την προηγούμενη ενότητα, ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN να είναι το μη συνεκτικό ως προς τον χώρο και τον χρόνο

$$x \in \Omega = [-L, \alpha_1] \cup [\beta_1, L], \quad -L < \alpha_1 < \beta_1 < L,$$

$$t \in [0, \alpha_2] \cup [\beta_2, T], \quad 0 < \alpha_2 < \beta_2 < T,$$

$$(5.15) \quad \begin{cases} u_{tt} - c^2 u_{xx} = 0, & x \in [-L, L], t \in [0, T], \\ u(x, 0) = A \cos\left(\frac{\pi}{L}x\right), & x \in [-L, \alpha_1] \cup [\beta_1, L], \\ u_t(x, 0) = 0, & x \in [-L, \alpha_1] \cup [\beta_1, L], \\ u_x(-L, t) = 0, & t \in [0, \alpha] \cup [\beta, T] \\ u_x(L, t) = 0, & t \in [0, \alpha] \cup [\beta, T] \end{cases}$$

Για την (5.15) ορίσαμε τις ίδιες μεταβλητές (5.3).

Για την προσέγγιση της λύσης της (5.15) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [73]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

5.10α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Για τις παραμέτρους α_1 , α_2 , β_1 και β_2 επιλέχθηκαν συμμετρικές τιμές ως προς το κέντρο των διαστημάτων $(-L, L)$ και $[0, T]$, ώστε τα αφαιρούμενα υποδιαστήματα (α_1, β_1) και (α_2, β_2) να οδηγούν σε ομοιόμορφη διάσπαση του χωρίου ορισμού.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.11) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.19

$[-L, \alpha_1] \cup [\beta_1, L]$	$[0, \alpha_2] \cup [\beta_2, T]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-2, -0.0] \cup [0.0, 2]$	$[0.2, 5] \cup [2.5, 5]$	6.98457006e-03	1.74223562e-04	1:38	0:28
$[-2, -0.1] \cup [0.1, 2]$	$[0.2, 4] \cup [2.6, 5]$	2.38830149e-02	2.90317927e-04	1:37	0:26
$[-2, -0.2] \cup [0.2, 2]$	$[0.2, 3] \cup [2.7, 5]$	2.50937250e-02	1.70120431e-04	1:40	0:26
$[-2, -0.3] \cup [0.3, 2]$	$[0.2, 2] \cup [2.8, 5]$	5.08970581e-02	3.09328083e-04	1:42	0:28
$[-2, -0.4] \cup [0.4, 2]$	$[0.2, 1] \cup [2.9, 5]$	5.22635505e-02	2.53869948e-04	1:42	0:26
$[-2, -0.5] \cup [0.5, 2]$	$[0.2, 0] \cup [3.0, 5]$	4.72046323e-02	1.04526768e-03	1:42	0:28
$[-2, -0.6] \cup [0.6, 2]$	$[0.1, 9] \cup [3.1, 5]$	4.75716740e-02	9.20686754e-04	1:38	0:26
$[-2, -0.7] \cup [0.7, 2]$	$[0.1, 8] \cup [3.2, 5]$	3.37043740e-02	2.22792209e-04	1:40	0:26
$[-2, -0.8] \cup [0.8, 2]$	$[0.1, 7] \cup [3.3, 5]$	3.71858180e-02	5.03305288e-04	1:37	0:25
$[-2, -0.9] \cup [0.9, 2]$	$[0.1, 6] \cup [3.4, 5]$	3.02412342e-02	1.00827171e-03	1:38	0:25
$[-2, -1.0] \cup [1.0, 2]$	$[0.1, 5] \cup [3.5, 5]$	2.48197652e-02	8.65676731e-04	1:39	0:26

Πίνακας 5.19: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Nuemann με εκπαίδευση σε CPU και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Από τον Πίνακα 5.19 μπορούν να εξαχθούν τα εξής συμπεράσματα

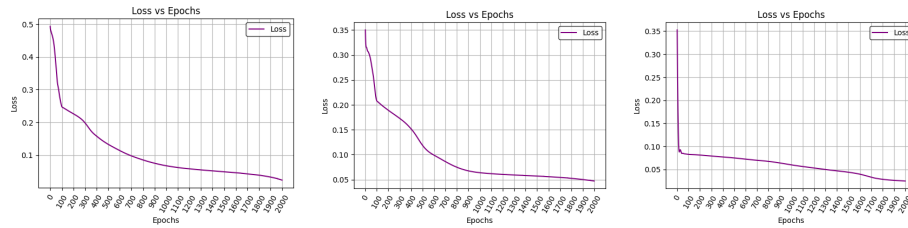
Τα αποτελέσματα της συνολικής Loss Function μέσω της Μεθόδου βελτιστοποίησης Adam παρατηρούμε ότι παραμένουν ουσιαστικά της ίδιας τάξης μεγέθους 10^{-2} , γεγονός που υποδηλώνει ότι η μείωση των δεδομένων εκπαίδευσης, δηλαδή

η αύξηση των υποδιαστημάτων (α_1, β_1) των Αρχικών Συνθηκών και (α_2, β_2) των Συνοριακών Συνθηκών, καθιστά το συγκεκριμένο πρόβλημα εξαιρετικά δύσκολο.

Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους 10^{-4} , το οποίο υποδηλώνει ότι το δίκτυο αντιμετωπίζει επαρκώς την δυσκολία ως προς την μείωση των δεδομένων εκπαίδευσης.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

Με βάση τα παραπάνω παίρνουμε τα γραφήματα 5.31, 5.32, 5.33:

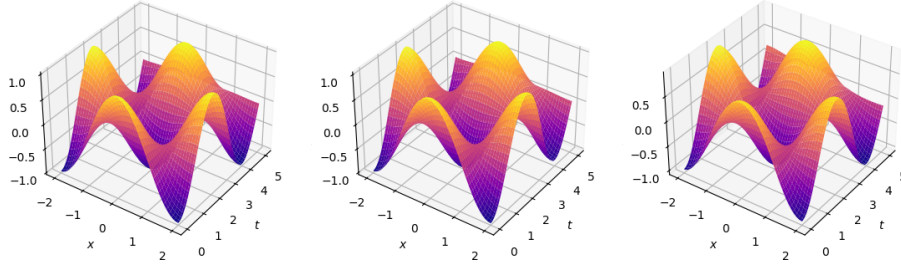


(α') Loss με Α.Σ. $[-2, -0.1] \cup [0.1, 2]$ και με Σ.Σ. $[0, 2.4] \cup [2.6, 5]$ (β') Loss με Α.Σ. $[-2, -0.5] \cup [0.5, 2]$ και με Σ.Σ. $[0, 2.0] \cup [3.0, 5]$ (γ') Loss με Α.Σ. $[-2, -1.0] \cup [1.0, 2]$ και με Σ.Σ. $[0, 1.5] \cup [3.5, 5]$

Σχήμα 5.31: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam με κατά Τμήματα Ορισμένες Αρχικές Συνθηκές και με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Neumann. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

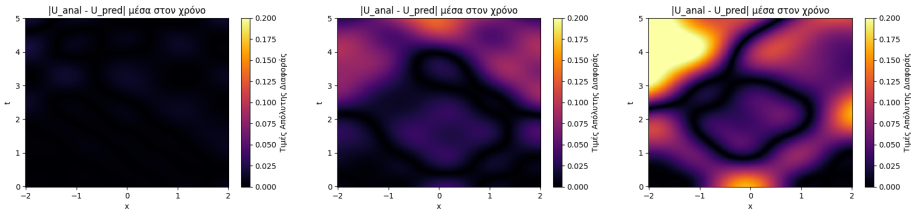
Στο Σχήμα (5.31) παρατηρούμε ότι, όσο αυξάνονται τα υποδιαστήματα (α_1, β_1) και (α_2, β_2) , δηλαδή όσο περιορίζεται το πλήθος των σημείων της εκπαίδευσης, τόσο η συνολική Loss Function αυξάνεται, με εξαίρεση το τρίτο γράφημα, όπου η συνολική Loss Function συγκλίνει ταχύτερα στην ελάχιστη τιμή της. Αυτό υποδηλώνει ότι ίσως το δίκτυο δεν μαθαίνει επαρκώς την φυσική του προβλήματος, το οποίο μπορεί να επαληθευτεί στο Σχήμα (5.32).

5.10 ΚΑΤΑ ΤΜΗΜΑΤΑ ΟΡΙΣΜΕΝΕΣ ΑΡΧΙΚΕΣ ΣΥΝΘΗΚΕΣ ΚΑΙ ΚΑΤΑ ΤΜΗΜΑΤΑ ΟΡΙΣΜΕΝΕΣ ΣΥΝΟΡΙΑΚΕΣ ΣΥΝΘΗΚΕΣ DIRICHLET · 83



(α') $\bar{u}_\theta(x, t)$ με Α.Σ. $[-2, -0.1] \cup [0.1, 2]$ και με Σ.Σ. $[0, 2.4] \cup [2.6, 5]$ (β') $\bar{u}_\theta(x, t)$ με Α.Σ. $[-2, -0.5] \cup [0.5, 2]$ και με Σ.Σ. $[0, 2.0] \cup [3.0, 5]$ (γ') $\bar{u}_\theta(x, t)$ με Α.Σ. $[-2, -1.0] \cup [1.0, 2]$ και με Σ.Σ. $[0, 1.5] \cup [3.5, 5]$

Σχήμα 5.32: Γραφήματα της προσέγγισης της λύσης με PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθληκες και με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Neumann. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.



(α') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -0.1] \cup [0.1, 2]$ και με Σ.Σ. $[0, 2.4] \cup [2.6, 5]$ (β') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -0.5] \cup [0.5, 2]$ και με Σ.Σ. $[0, 2.0] \cup [3.0, 5]$ (γ') $\max|Αναλυτική - PINN|$ με Α.Σ. $[-2, -1.0] \cup [1.0, 2]$ και με Σ.Σ. $[0, 1.5] \cup [3.5, 5]$

Σχήμα 5.33: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθληκες και με κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Neumann. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

Στο Σχήμα (5.33) παρατηρούμε ότι υπάρχει ένα όριο στο πόσο μεγάλο μέρος μπορούμε να εξαιρέσουμε από το σύνολο εκπαίδευσης των Αρχικών και Συνοριακών Συνθηκών ταυτόχρονα, για το συγκεκριμένο πρόβλημα. Μέχρι ένα σημείο, η προσέγγιση της λύσης διατηρείται σε ικανοποιητικό επίπεδο ακρίβειας, ενώ πέρα από αυτό το όριο η ποιότητα της προσέγγισης υποβαθμίζεται σημαντικά, χωρίς όμως να χάνει τόσο έντονα την γεωμετρία του.

5.10β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (5.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια.

Με βάση τα (5.4, 5.5, 5.6) προσεγγίζουμε την (5.14) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 5.20

$[-L, \alpha_1] \cup [\beta_1, L]$	$[0, \alpha_2] \cup [\beta_2, T]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-2, -0.0] \cup [0.0, 2]$	$[0, 2.5] \cup [2.5, 5]$	4.28482406e-02	1.07611518e-03	1:07	0:25
$[-2, -0.1] \cup [0.1, 2]$	$[0, 2.4] \cup [2.6, 5]$	4.44692522e-02	1.02044200e-03	1:09	0:24
$[-2, -0.2] \cup [0.2, 2]$	$[0, 2.3] \cup [2.7, 5]$	3.62705961e-02	9.06979432e-04	1:09	0:24
$[-2, -0.3] \cup [0.3, 2]$	$[0, 2.2] \cup [2.8, 5]$	4.35738824e-02	9.42911080e-04	1:08	0:24
$[-2, -0.4] \cup [0.4, 2]$	$[0, 2.1] \cup [2.9, 5]$	5.99691868e-02	1.42855477e-03	1:09	0:24
$[-2, -0.5] \cup [0.5, 2]$	$[0, 2.0] \cup [3.0, 5]$	4.59680595e-02	4.91285988e-04	1:08	0:24
$[-2, -0.6] \cup [0.6, 2]$	$[0, 1.9] \cup [3.1, 5]$	4.37186509e-02	5.51955076e-04	1:09	0:24
$[-2, -0.7] \cup [0.7, 2]$	$[0, 1.8] \cup [3.2, 5]$	4.15018760e-02	3.15840094e-04	1:08	0:23
$[-2, -0.8] \cup [0.8, 2]$	$[0, 1.7] \cup [3.3, 5]$	3.82729918e-02	2.98686035e-04	1:08	0:24
$[-2, -0.9] \cup [0.9, 2]$	$[0, 1.6] \cup [3.4, 5]$	3.28415260e-02	4.70621279e-04	1:09	0:24
$[-2, -1.0] \cup [1.0, 2]$	$[0, 1.5] \cup [3.5, 5]$	3.21133658e-02	5.14245708e-04	1:10	0:24

Πίνακας 5.20: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες Nuemann με εκπαίδευση σε GPU μέσω CUDA και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης. Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 1 λεπτό, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε περίπου 25 δευτερόλεπτα, γεγονός που υποδηλώνει περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Όμοια, στην CPU, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές του υποδιαστήματος (α, β) . Συγκεκριμένα, η βελτιστοποίηση με Adam απαιτεί λιγότερο από 2 λεπτά, ενώ η L-BFGS λίγο περίπου 30 δευτερόλεπτα.

Τόσο με χρήση CPU, όσο και με χρήση GPU μέσω CUDA, παρατηρούμε περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το περιορισμένο σύνολο των δεδομένων εκπαίδευσης και στη δυσκολία του προβλήματος.

Ως προς την τάξη της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam έχει σταθερή τάξη μεγέθους 10^{-2} , ενώ στην μέθοδο L-BFGS οι τιμές συγκεντρώνονται κυρίως γύρω από την τάξη μεγέθους 10^{-4} .

Στην GPU μέσω CUDA τόσο στην μέθοδο Adam όσο και στην μέθοδο L-BFGS παρατηρείται σταθερότητα ως προς τις τάξεις μεγέθους, με 10^{-2} και 10^{-3} αντίστοιχα, γεγονός που υποδηλώνει πιο ομοιόμορφη συμπεριφορά της διαδικασίας βελτιστοποίησης.

Συνολικά, καταλήγουμε στο συμπέρασμα ότι η CPU μπορεί, σε ορισμένες περιπτώσεις, να επιτύχει χαμηλότερες τελικές τιμές Loss, με κόστος όμως αυξημένου χρόνου εκπαίδευσης. Αντίθετα, η GPU με χρήση CUDA προσφέρει σημαντική επιτάχυνση και σταθερότητα στον χρόνο εκπαίδευσης, επιτυγχάνοντας συγκρίσιμες τάξεις απόκλισης, γεγονός που την καθιστά ιδιαίτερα αποδοτική για μεγαλύτερα σύνολα δεδομένων στο συγκεκριμένο πρόβλημα.

Κεφάλαιο 6

Εξίσωση Korteweg - de Vries (KdV)

Ορίζουμε την (μη γραμμική) Μερική Διαφορική Εξίσωση KdV που ορίζεται ως εξής

$$(6.1) \quad u_t - \alpha u u_x + \beta u_{xxx} = 0, x \in \Omega, t \in [0, T]$$

Το σύνολο $\Omega \subset \mathbb{R}^N$ είναι ένα ανοιχτό και φραγμένο σύνολο και $T \in \mathbb{R}^+$ το μέγιστο του χρόνου.

Η αναλυτική μορφή (6.1) διατυπώθηκε το 1895 από τους Diederik Johannes Korteweg (Ντίντερικ Γιοχάνες Κόρτεβεχ 1848 - 1941) και Gustav de Vries (Γκούσταφ ντε Φρις 1866 - 1934). [74]

Η εξίσωση KdV αποτελεί θεμελιώδες μοντέλο στη θεωρία των μη γραμμικών κυμάτων, καθώς περιγράφει την εξέλιξη των Σολιτονίων (soliton waves). Τα σολιτόνια συνιστούν ειδική κατηγορία μη γραμμικών κυμάτων που χαρακτηρίζονται από, τη διατήρηση του σχήματος τους καθώς διαδίδονται, τη διάδοση με σταθερή ταχύτητα και την αλληλεπίδραση με άλλα σολιτόνια χωρίς μεταβολή της μορφής τους. Το φαινόμενο αυτό είχε παρατηρηθεί για πρώτη φορά το 1834 από τον John Scott Russell (Τζον Σκοτ Ράσελ 1808 - 1882) κατά την παρατήρηση και μελέτη κυμάτων σε ρηχά κανάλια νερού. [75]

Σκοπός είναι να προσεγγίσουμε την πραγματική λύση $u(x, t)$ με ακρόντως μικρή απώλεια μέσω Physics-Informed Neural Networks.

6.1 Περιοδικές Συνοριακές Συνθήκες

Στην Μερική Διαφορική Εξίσωση (6.1) εφαρμόζουμε μία Αρχική Συνθήκη (Initial Condition), τρεις Περιοδικές Συνοριακές Συνθήκες (Periodic Boundary Conditions) και ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN.

$$(6.2) \quad \begin{cases} u_t - \alpha u u_x + \beta u_{xxx} = 0, & x \in [-L, L], \quad t \in [0, T], \\ u(x, 0) = \frac{c}{2} \operatorname{sech}^2\left(\frac{\sqrt{c}}{2}x\right), & x \in [-L, L], \\ u(-L, t) = u(L, t), & t \in [0, T] \\ u_x(-L, t) = u_x(L, t), & t \in [0, T] \\ u_{xx}(-L, t) = u_{xx}(L, t), & t \in [0, T] \end{cases}$$

Για την (6.2) ορίζουμε

$$(6.3) \quad \alpha = p = 6, \beta = q = 1, T = 5, L = 12, c = 1$$

Για την προσέγγιση της λύσης της (6.2) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [76]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

6.1α' Εκπαίδευση σε CPU

Ορίζουμε τον Ρυθμό Μάθησης ώστε, σε αντίθεση με την Εξίσωση Κύματος, η ποσότητα 10^{-4} να εφαρμόζεται στις πρώτες 1000 εποχές

$$(6.4) \quad \text{Ρυθμός Μάθησης} = \begin{cases} 5 \times 10^{-4}, & \text{Εποχές} < 1000, \\ 1 \times 10^{-4}, & 1000 \leq \text{Εποχές} < 3000 \end{cases}$$

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια με την εξίσωση κύματος.

Με βάση τα (5.4, 6.4, 5.6) προσεγγίζουμε την (6.2) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 6.1

N_0	N_b	N_u	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$	$Epochs_{Adam}$
400	500	2000	9.99536132e-06	1.76529397e-06	2:19	0:05	892
300	500	2000	9.98220730e-06	1.94681456e-06	2:05	0:07	747
200	500	2000	9.98239739e-06	9.96172912e-06	2:09	0:00	787
100	500	2000	3.81073296e-05	1.72622117e-06	2:57	0:21	2000
50	500	2000	9.99742042e-06	1.50278902e-06	2:21	0:08	1624
400	400	2000	9.99797157e-06	9.97282223e-06	1:01	0:00	727
400	300	2000	9.99288477e-06	9.97227016e-06	1:07	0:00	802
400	200	2000	3.70576672e-05	1.25971269e-06	2:46	0:27	2000
400	100	2000	1.63067143e-05	1.71630404e-06	2:42	0:09	2000
400	50	2000	1.58237563e-05	1.76070455e-06	2:40	0:11	2000
400	10	2000	3.02122880e-05	2.31156423e-06	2:37	0:07	2000
400	500	1800	1.76421581e-05	1.55613429e-06	2:47	0:09	2000
400	500	1600	4.94323431e-05	1.91691902e-06	2:36	0:10	2000
400	500	1400	1.27586582e-05	1.31318620e-06	2:26	0:07	2000
400	500	1200	1.18978487e-05	1.79673202e-06	2:20	0:06	2000
400	500	1000	1.70038566e-05	1.43419300e-06	2:07	0:07	2000
400	500	800	1.16412621e-05	8.02780676e-07	2:01	0:11	2000
400	500	600	9.99722033e-06	2.97420092e-06	1:18	0:03	1453
400	500	400	9.99933582e-06	1.89856826e-06	1:09	0:03	1409
400	500	200	1.12532480e-05	1.50403957e-06	1:29	0:05	2000
300	400	1500	1.41586625e-05	9.28656334e-07	2:30	0:11	2000
300	400	1000	9.87724616e-06	1.49914581e-06	1:01	0:07	990
300	400	500	9.99840540e-06	1.45160254e-06	1:17	0:04	1544
200	300	1500	6.36188270e-05	1.02280319e-06	2:27	0:13	2000
200	300	1000	1.05900053e-05	2.17185698e-06	2:01	0:04	2000
200	300	500	1.01053320e-05	2.80115546e-06	1:35	0:03	2000
100	200	1500	1.24936178e-05	1.53236510e-06	2:22	0:09	2000
100	200	1000	9.96664494e-06	1.41340183e-06	0:38	0:00	649
100	200	500	9.99628264e-06	9.99240547e-06	0:47	0:00	1020
50	100	1500	2.14352367e-05	1.20730681e-06	2:19	0:10	2000
50	100	1000	1.90481769e-05	1.68646500e-06	1:56	0:06	2000
50	100	500	9.98640280e-06	9.96434846e-06	0:43	0:00	999

Πίνακας 6.1: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ KdV με Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε CPU

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Σε ορισμένες περιπτώσεις, η εκπαίδευση ολοκληρώθηκε πριν τη μέγιστη τιμή των 2000 εποχών λόγω ικανοποιητικής σύγκλισης. Το $Epochs_{Adam}$ αντιστοιχεί στο πλήθος των εποχών της εκπαίδευσης.

Από τον Πίνακα 6.1 μπορούν να εξαχθούν τα εξής συμπεράσματα

Τα αποτελέσματα της συνολικής Loss Function μέσω της Μεθόδου βελτιστοποίησης Adam παρατηρούμε ότι παραμένουν της ίδιας τάξης μεγέθους, 10^{-5} , για όλες τις εξεταζόμενες επιλογές σημείων εκπαίδευσης. Όμως, παρατηρούνται διακυμάνσεις που υποδηλώνουν ότι η σύγκλιση της Adam επηρεάζεται από τη σύνθεση του συνόλου εκπαίδευσης και όχι μόνο από το συνολικό πλήθος των σημείων.

Ως προς τον υπολογιστικό χρόνο της Adam, προκύπτει ότι η αύξηση του πλήθους των σημείων εκπαίδευσης οδηγεί γενικά σε μεγαλύτερους χρόνους εκπαίδευσης, με τον αριθμό των collocation points N_u να αποτελεί τον κυρίαρχο παράγοντα κόστους.

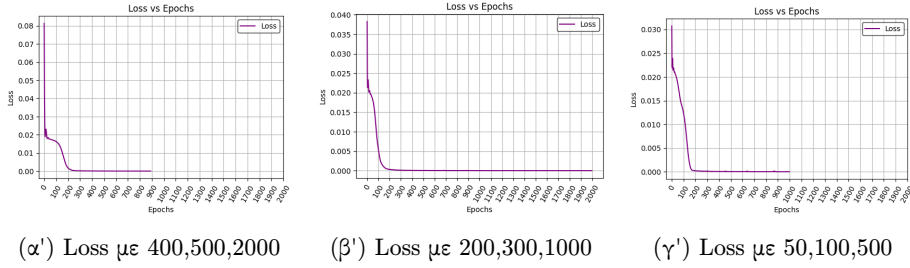
Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS παραμένουν της ίδιας τάξης μεγέθους, 10^{-6} .

Ο συνολικός χρόνος εκπαίδευσης της μεθόδου L-BFGS κυμαίνεται εντός ενός σχετικά στενού χρονικού διαστήματος, γεγονός που υποδηλώνει ότι επηρεάζεται σε μικρότερο βαθμό από το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Τα αποτελέσματα δείχνουν ότι η αύξηση του πλήθους των σημείων εκπαίδευσης δεν οδηγεί κατ' ανάγκη σε μείωση της συνολικής Loss Function, ενώ ταυτόχρονα μπορεί να αυξήσουν το υπολογιστικό κόστος σημαντικά, για το συγκεκριμένο πρόβλημα.

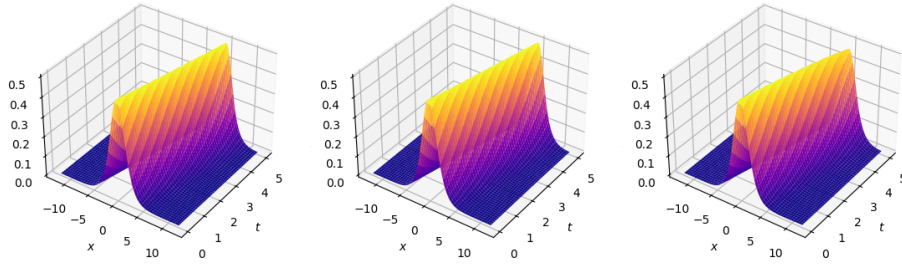
Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

Με βάση τα παραπάνω, παίρνουμε τα γραφήματα 6.1, 6.2, 6.3 και 6.4:

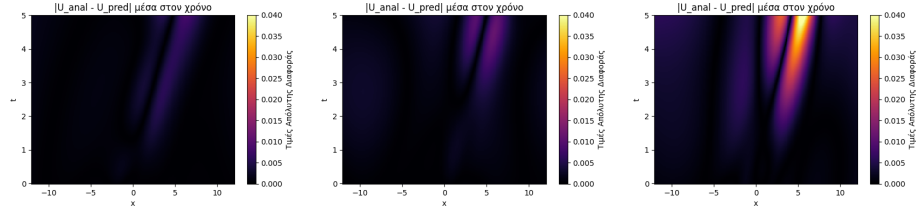


Σχήμα 6.1: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (6.1) παρατηρούμε ότι, για τις διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης, η εξέλιξη της συνολικής συνάρτησης απώλειας ως προς τις εποχές παρουσιάζει παρόμοια συμπεριφορά σύγκλισης. Το γεγονός αυτό υποδηλώνει ότι το δίκτυο μπορεί να εκπαιδευτεί αποτελεσματικά χωρίς την ανάγκη πολύ μεγάλου αριθμού σημείων, διατηρώντας την ακρίβεια της προσέγγισης, όπως φαίνεται και στο Σχήμα (6.2).



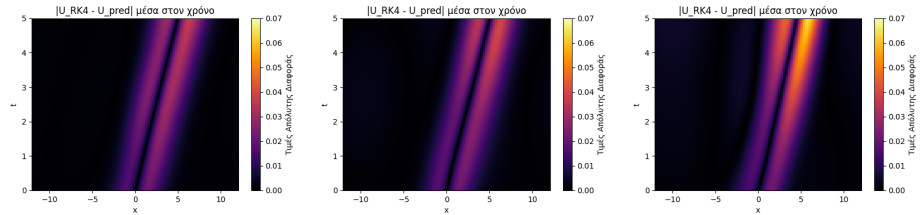
Σχήμα 6.2: Γραφήματα της προσέγγισης της λύσης με PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500



(α') $\max|\text{Αναλυτική} - \text{PINN}|$ για 400,500,2000 (β') $\max|\text{Αναλυτική} - \text{PINN}|$ για 200,300,1000 (γ') $\max|\text{Αναλυτική} - \text{PINN}|$ για 50,100,500

Σχήμα 6.3: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (6.3) παρατηρούμε ότι, ακόμη και για σχετικά μικρό αριθμό σημείων εκπαίδευσης, η προσεγγιστική λύση ακολουθεί ικανοποιητικά τη συμπεριφορά της αναλυτικής λύσης σε όλο τον εξεταζόμενο χωροχρόνο, με τις αποκλίσεις να παραμένουν αόρατες οπτικά.



(α') $\max|\text{RK4} - \text{PINN}|$ για 400,500,2000 (β') $\max|\text{RK4} - \text{PINN}|$ για 200,300,1000 (γ') $\max|\text{RK4} - \text{PINN}|$ για 50,100,500

Σχήμα 6.4: Μέγιστη Απόλυτη Διαφορά μεταξύ της Προσεγγιστικής Λύσης RK4 και της Προσεγγιστικής Λύσης PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (6.4) παρατηρούμε ότι, τα γραφήματα της μέγιστης απόλυτης διαφοράς μεταξύ της μεθόδου Runge–Kutta 4 και του PINN και εκείνα της μέγιστης απόλυτης διαφοράς μεταξύ της αναλυτικής λύσης και του PINN έχουν διαφορά μεταξύ τους. Παρατηρούμε ότι, για το συγκεκριμένο πρόβλημα, το PINN έχει καλύτερα προσεγγίσει την αναλυτική λύση.

Για την μέθοδο Πεπερασμένων Διαφορών αναπτύχθηκε το πρόγραμμα [[77]]

6.1β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (6.4) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια. Επίσης, οι μεταβλητές της εξίσωσης (6.3) παρέμειναν ίδιες.

Με βάση τα (5.4, 6.4, 5.6) προσεγγίζουμε την (6.2) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 6.2

N_0	N_b	N_u	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$	$Epochs_{Adam}$
400	500	2000	6.17642727e-05	1.44918397e-06	1:45	0:10	2000
300	500	2000	6.12943986e-05	9.84050530e-07	1:44	0:12	2000
200	500	2000	6.06935573e-05	5.38849781e-06	1:44	0:05	2000
100	500	2000	5.49956130e-05	1.15639989e-06	1:43	0:12	2000
50	500	2000	5.12220795e-05	1.24150631e-06	1:42	0:10	2000
400	400	2000	6.17712576e-05	1.38615906e-06	1:42	0:09	2000
400	300	2000	6.09460476e-05	3.29602472e-06	1:42	0:07	2000
400	200	2000	5.95642377e-05	4.77874482e-06	1:43	0:05	2000
400	100	2000	5.97031831e-05	3.49250331e-06	1:43	0:07	2000
400	50	2000	6.07501970e-05	2.93162134e-06	1:42	0:07	2000
400	10	2000	6.27764603e-05	1.14821785e-06	1:42	0:11	2000
400	500	1800	6.15149838e-05	1.06330140e-06	1:42	0:11	2000
400	500	1600	6.15149838e-05	1.06330140e-06	1:42	0:11	2000
400	500	1400	5.93308032e-05	2.75772322e-06	1:42	0:08	2000
400	500	1200	5.93308032e-05	2.75772322e-06	1:42	0:07	2000
400	500	1000	6.13428419e-05	1.38424741e-06	1:41	0:10	2000
400	500	800	6.08749797e-05	1.73866556e-06	1:42	0:09	2000
400	500	600	6.41577353e-05	1.32544562e-06	1:43	0:11	2000
400	500	400	6.26311739e-05	1.92483685e-06	1:43	0:08	2000
400	500	200	6.68917564e-05	1.65340953e-06	1:44	0:10	2000
300	400	1500	5.97255057e-05	9.89027853e-07	1:40	0:11	2000
300	400	1000	6.12618242e-05	2.87419221e-06	1:41	0:07	2000
300	400	500	6.43678359e-05	1.50350638e-06	1:45	0:10	2000
200	300	1500	5.84449408e-05	3.22053370e-06	1:42	0:06	2000
200	300	1000	6.01814827e-05	2.87363832e-06	1:42	0:07	2000
200	300	500	6.29660863e-05	2.43447266e-06	1:44	0:08	2000
100	200	1500	5.19324385e-05	1.14539273e-06	1:43	0:10	2000
100	200	1000	5.48243697e-05	1.01151534e-06	1:45	0:11	2000
100	200	500	5.74039514e-05	1.47396884e-06	1:43	0:09	2000
50	100	1500	4.88263249e-05	1.08849360e-06	1:43	0:10	2000
50	100	1000	5.16293403e-05	1.13445935e-06	1:42	0:11	2000
50	100	500	5.36310872e-05	1.09558437e-06	1:43	0:12	2000

Πίνακας 6.2: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ KdV με Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε GPU

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης. Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 2 λεπτά, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε λίγο περισσότερο από 10 δευτερόλεπτα, γεγονός που υποδηλώνει περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Αντίθετα, κατά την εκτέλεση των υπολογισμών στην CPU, ο χρόνος εκπαίδευσης εξαρτάται από το πλήθος και την σύνθεση των σημείων εκπαίδευσης, στο συγκεκριμένο πρόβλημα. Συγκεκριμένα, η βελτιστοποίηση με Adam απαιτεί από 1 έως περίπου 3 λεπτά, ενώ η L-BFGS κυμαίνεται από 0 μέχρι περίπου 10 δευτερόλεπτα και σε δύο περιπτώσεις ξεπέρασε τα 20 δευτερόλεπτα.

Ως προς την τάξη της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam έχει ίδια τάξη μεγέθους 10^{-5} , όπως και στην GPU με χρήση CUDA. Επίσης, στη μέθοδο L-BFGS, στη CPU έχει τάξη μεγέθους 10^{-6} , όπως και στην GPU με χρήση CUDA, γεγονός που υποδηλώνει ομοιόμορφη συμπεριφορά της διαδικασίας βελτιστοποίησης και στις δύο περιπτώσεις.

Καταλήγουμε στο συμπέρασμα ότι, η χρήση CPU είναι επαρκής για προβλήματα μικρού συνόλου δεδομένων, καθώς επιτυγχάνονται συγκρίσιμες τιμές της συνολικής Loss Function. Ωστόσο, καθώς αυξάνεται το πλήθος των σημείων εκπαίδευσης, ο χρόνος εκπαίδευσης στην CPU αυξάνεται αισθητά. Αντίθετα, η GPU με χρήση CUDA επιτρέπει ταχύτερη και πιο αποδοτική εκπαίδευση, με τον χρόνο να παραμένει σχεδόν σταθερός ακόμη και για μεγαλύτερα σύνολα δεδομένων, χωρίς να επηρεάζεται αρνητικά η σύγκλιση της συνάρτησης απώλειας. Για τον λόγο αυτό, στο συγκεκριμένο πρόβλημα, η GPU αποτελεί πιο κατάλληλη επιλογή για την εκπαίδευση των PINNs, ιδίως όταν έχουμε μεγάλο αριθμό σημείων εκπαίδευσης.

6.2 Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες

Στην Μερική Διαφορική Εξίσωση (6.1) εφαρμόζουμε μία Αρχική Συνθήκη (Initial Condition), τρεις Περιοδικές Συνοριακές Συνθήκες (Periodic Boundary Conditions).

Σε αντίθεση με την προηγούμενη ενότητα, ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN να είναι το μη συνεκτικό

$$x \in \Omega = [-L, \alpha] \cup [\beta, L], \quad -L < \alpha < \beta < L,$$

$$(6.5) \quad \begin{cases} u_t - \alpha u u_x + \beta u_{xxx} = 0, & x \in [-L, L], \quad t \in [0, T], \\ u(x, 0) = \frac{c}{2} \operatorname{sech}^2\left(\frac{\sqrt{c}}{2}x\right), & x \in [-L, \alpha] \cup [\beta, L], \\ u(-L, t) = u(L, t), & t \in [0, T] \\ u_x(-L, t) = u_x(L, t), & t \in [0, T] \\ u_{xx}(-L, t) = u_{xx}(L, t), & t \in [0, T] \end{cases}$$

Για την (6.5) ορίσαμε τις ίδιες μεταβλητές (5.3).

Για την προσέγγιση της λύσης της (6.5) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [78]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

6.2α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (6.4) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια. Επίσης, οι μεταβλητές τις εξίσωσης (6.3) παρέμειναν ίδιες.

Για τις παραμέτρους α και β επιλέχθηκαν συμμετρικές τιμές ως προς το κέντρο του διαστήματος $[-L, L]$, ώστε το αφαιρούμενο υποδιάστημα (α, β) να οδηγεί σε ομοιόμορφη διάσπαση του χωρίου ορισμού.

Με βάση τα (5.4, 6.4, 5.6) προσεγγίζουμε την (5.7) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 6.3

$[-L, \alpha] \cup [\beta, L]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$	$Epochs_{Adam}$
$[-12, -0.0] \cup [0.0, 12]$	9.99536132e-06	1.76529397e-06	1:29	0:05	892
$[-12, -0.5] \cup [0.5, 12]$	9.99461099e-06	1.30601222e-06	1:21	0:09	908
$[-12, -1.0] \cup [1.0, 12]$	9.98416726e-06	9.96623203e-06	1:27	0:00	950
$[-12, -1.5] \cup [1.5, 12]$	9.99767053e-06	9.99232361e-06	1:33	0:00	1017
$[-12, -2.0] \cup [2.0, 12]$	1.64022167e-05	1.92617949e-06	3:03	0:08	2000
$[-12, -2.5] \cup [2.5, 12]$	3.01367236e-05	4.34287222e-06	3:04	0:06	2000
$[-12, -3.0] \cup [3.0, 12]$	3.42601634e-05	1.54527675e-06	3:03	0:15	2000
$[-12, -3.5] \cup [3.5, 12]$	5.09396887e-05	1.54155850e-06	3:00	0:20	2000
$[-12, -4.0] \cup [4.0, 12]$	3.42009116e-05	2.18542073e-06	3:02	0:25	2000
$[-12, -4.5] \cup [4.5, 12]$	1.45261984e-05	1.45238482e-05	2:59	0:00	2000
$[-12, -5.0] \cup [5.0, 12]$	9.99912845e-06	4.13797216e-06	0:44	0:09	477
$[-12, -5.5] \cup [5.5, 12]$	9.98855558e-06	1.90206572e-06	0:17	0:00	183
$[-12, -6.0] \cup [6.0, 12]$	9.96290510e-06	9.27386168e-07	0:12	0:04	139
$[-12, -6.5] \cup [6.5, 12]$	9.98135602e-06	4.67016577e-07	0:11	0:04	123

Πίνακας 6.3: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ KdV με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε CPU και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Από τον Πίνακα 6.3 μπορούν να εξαχθούν τα εξής συμπεράσματα

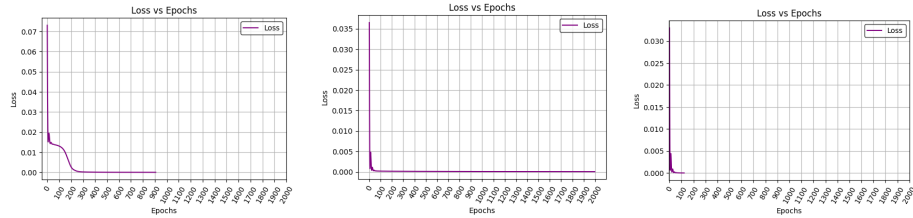
Από τα αποτελέσματα της συνολικής Loss Function, μέσω της βελτιστοποίησης Adam, παρατηρούμε ότι υπάρχει σταδιακή αύξηση της τάξης μεγέθους, από 10^{-6} σε 10^{-5} , γεγονός που υποδηλώνει ότι η μείωση των δεδομένων εκπαίδευσης, δηλαδή η αύξηση του υποδιαστήματος (α, β) , των Αρχικών Συνθηκών καθιστά το συγκεκριμένο πρόβλημα δυσκολότερο για το δίκτυο. Επιπλέον, όταν εφαρμόζουμε

τις Αρχικές Συνθήκες στο διάστημα $[-12, -5] \cup [5, 12]$ το δίκτυο αδυνατεί να προσεγγίσει την αναλυτική λύση.

Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας, αλλά όχι αισθητά. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους που κυμαίνεται από 10^{-6} έως 10^{-5} , το οποίο υποδηλώνει ότι το δίκτυο αντιμετωπίζει την ίδια δυσκολία ως προς την μείωση των δεδομένων εκπαίδευσης.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

Με βάση τα παραπάνω, παίρνουμε τα γραφήματα 6.5, 6.6, 6.7:

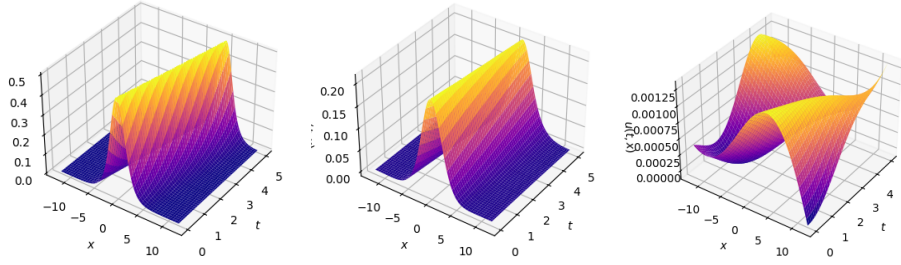


(α') Loss με Α.Σ. στο $[-12, -0.5] \cup [0.5, 12]$ (β') Loss με Α.Σ. στο $[-12, -3.5] \cup [3.5, 12]$ (γ') Loss με Α.Σ. στο $[-12, -6.5] \cup [6.5, 12]$

Σχήμα 6.5: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400,500,2000.

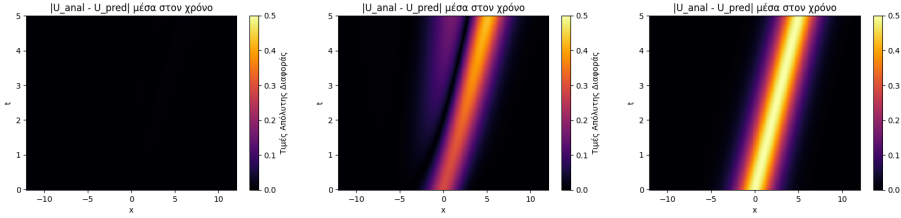
Στο Σχήμα (6.5) παρατηρούμε ότι, για διαφορετικές επιλογές του υποδιαστήματος (α, β) , το PINN εμφανίζει διαφορετικές πορείες σύγκλισης της Loss Function κατά τη διάρκεια της εκπαίδευσης. Στο πρώτο γράφημα τερματίζει νωρίτερα καθώς έχει επαρκή σύγκλιση, στο δεύτερο χρειάζεται παραπάνω εποχές για την απαραίτητη σύγκλιση ενώ στο τρίτο αν και τερματίζει νωρίτερα, αδυνατεί να προσεγγίσει την αναλυτική λύση, όπως φαίνεται στο Σχήμα (6.6).

Η ελαχιστοποίηση της συνολικής Loss Function, από μόνη της, δεν αποτελεί ούτε μοναδική ούτε επαρκή συνθήκη για να εξασφαλιστεί ότι η εκπαίδευση έγινε προς την σωστή λύση.



(α') $\bar{u}_\theta(x, t)$ με Α.Σ. στο $[-12, -0.5] \cup [0.5, 12]$ (β') $\bar{u}_\theta(x, t)$ με Α.Σ. στο $[-12, -3.5] \cup [3.5, 12]$ (γ') $\bar{u}_\theta(x, t)$ με Α.Σ. στο $[-12, -6.5] \cup [6.5, 12]$

Σχήμα 6.6: Γραφήματα της Προσεγγιστικής λύσης PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400,500,2000.



(α') $\max|Αναλυτική - PINN|$ με Α.Σ. στο $[-12, -0.5] \cup [0.5, 12]$ (β') $\max|Αναλυτική - PINN|$ με Α.Σ. στο $[-12, -3.5] \cup [3.5, 12]$ (γ') $\max|Αναλυτική - PINN|$ με Α.Σ. στο $[-12, -6.5] \cup [6.5, 12]$

Σχήμα 6.7: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400,500,2000.

Στο Σχήμα (6.7) παρατηρούμε ότι υπάρχει ένα όριο στο πόσο μεγάλο μέρος μπορούμε να εξαιρέσουμε από το σύνολο εκπαίδευσης των Συνοριακών Συνθηκών, για το συγκεκριμένο πρόβλημα. Μέχρι ένα σημείο, η προσέγγιση της λύσης διατηρείται σε ικανοποιητικό επίπεδο ακρίβειας, ενώ πέρα από αυτό το όριο η ποιότητα της προσέγγισης υποβαθμίζεται σημαντικά.

6.2β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (6.4) και τα βάρη της ολικής Συνάρτησης Απώλειας (5.6) παρέμειναν ίδια. Επίσης, οι μεταβλητές της εξίσωσης (6.3) παρέμειναν ίδιες.

Με βάση τα (5.4, 6.4, 5.6) προσεγγίζουμε την (6.5) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 6.4

$[-L, \alpha] \cup [\beta, L]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$	$Epochs_{Adam}$
$[-12, -0.0] \cup [0.0, 12]$	1.71626707e-05	2.46428135e-06	1:44	0:04	2000
$[-12, -0.5] \cup [0.5, 12]$	1.44954211e-05	1.51962558e-06	1:43	0:07	2000
$[-12, -1.0] \cup [1.0, 12]$	1.74804300e-05	1.73944773e-06	1:43	0:06	2000
$[-12, -1.5] \cup [1.5, 12]$	3.41599480e-05	2.05390302e-06	1:42	0:05	2000
$[-12, -2.0] \cup [2.0, 12]$	6.19914354e-05	4.49123536e-06	1:42	0:06	2000
$[-12, -2.5] \cup [2.5, 12]$	6.70803493e-05	1.15502758e-06	1:44	0:14	2000
$[-12, -3.0] \cup [3.0, 12]$	6.34126336e-05	1.94072641e-06	1:43	0:15	2000
$[-12, -3.5] \cup [3.5, 12]$	4.79558657e-05	2.99913631e-06	1:42	0:12	2000
$[-12, -4.0] \cup [4.0, 12]$	2.71270601e-05	2.28096337e-06	1:42	0:16	2000
$[-12, -4.5] \cup [4.5, 12]$	9.99993790e-06	2.14877900e-06	1:12	0:10	2000
$[-12, -5.0] \cup [5.0, 12]$	9.99911845e-06	2.94335905e-06	0:28	0:04	557

Πίνακας 6.4: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ KdV με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε GPU και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης. Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 1 λεπτά, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε λίγο περισσότερο από 10 δευτερόλεπτα τις περισσότερες φορές, γεγονός που υποδηλώνει περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Αντίθετα, κατά την εκτέλεση των υπολογισμών στην CPU, ο χρόνος εκπαίδευσης εξαρτάται από το πλήθος και την σύνθεση των σημείων εκπαίδευσης, στο συγκεκριμένο πρόβλημα. Συγκεκριμένα, η βελτιστοποίηση με Adam απαιτεί από περίπου 1:20 έως και 3 λεπτά, ενώ η L-BFGS κυμαίνεται από 0 μέχρι 25 δευτερόλεπτα.

Ως προς την τάξη της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam τάξη μεγέθους είναι 10^{-5} , ενώ στην μέθοδο L-BFGS οι τιμές συγκεντρώνονται κυρίως γύρω από την τάξη μεγέθους 10^{-6} .

Στην GPU μέσω CUDA τόσο στην μέθοδο Adam όσο και στην μέθοδο L-BFGS παρατηρείται σταθερότητα ως προς τις τάξεις μεγέθους, με 10^{-5} και 10^{-6} αντίστοιχα, γεγονός που υποδηλώνει πιο ομοιόμορφη συμπεριφορά της διαδικασίας βελτιστοποίησης.

Καταλήγουμε στο συμπέρασμα ότι, η χρήση CPU είναι επαρκής για προβλήματα μικρού συνόλου δεδομένων, καθώς επιτυγχάνονται συγκρίσιμες τιμές της συνολικής Loss Function. Ωστόσο, καθώς αυξάνεται το πλήθος των σημείων εκπαίδευσης, ο χρόνος εκπαίδευσης στην CPU αυξάνεται αισθητά. Αντίθετα, η GPU με χρήση CUDA επιτρέπει ταχύτερη και πιο αποδοτική εκπαίδευση, με τον χρόνο να παραμένει σχεδόν σταθερός ακόμη και για μεγαλύτερα σύνολα δεδομένων, χωρίς να επηρεάζεται αρνητικά η σύγκλιση της συνάρτησης απώλειας. Για τον λόγο αυτό, στο συγκεκριμένο πρόβλημα, η GPU αποτελεί πιο κατάλληλη επιλογή για την εκπαίδευση των PINNs, ιδίως όταν έχουμε μεγάλο αριθμό σημείων εκπαίδευσης.

Κεφάλαιο 7

Εξίσωση Non-Linear Schrödinger (NLS)

Ορίζουμε την (μη γραμμική) Μερική Διαφορική Εξίσωση KdV που ορίζεται ως εξής

$$(7.1) \quad i \psi_t + p \psi_{xx} + q |\psi|^2 \psi = 0, x \in \Omega, t \in [0, T]$$

Το σύνολο $\Omega \subset \mathbb{R}^N$ είναι ένα ανοιχτό και φραγμένο σύνολο και $T \in \mathbb{R}^+$ το μέγιστο του χρόνου.

Η γραμμική μορφή της εξίσωσης Schrödinger, για $q = 0$, διατυπώθηκε το 1926 από τον Erwin Rudolf Josef Alexander Schrödinger (Έρουν Σρούντιγκερ 1887 - 1961). Η μη γραμμική μορφή της (7.1) καθιερώθηκε από τους Vladimir E. Zakharov (Βλαντιμίρ Ζάχαροβ 1939 - 2023) το 1968 και Akira Hasegawa (Ακίρα Χασεγκάουα 1934 - 2025) και Frederick Drach Tappert (Φρίντερικ Τάπετ 1940 - 2002) το 1973, ενώ σημαντική ήταν και η συμβολή των Tosiya Taniuti (Τοσίγια Τανιούτι) και Nobuo Yajima (Νομπούο Γιαζίμα) το 1969. [79], [80], [81], [82], [83]

Η εξίσωση NLS αποτελεί, όπως και η NLS, θεμελιώδες μοντέλο στη θεωρία των μη γραμμικών κυμάτων, καθώς περιγράφει την εξέλιξη φακέλων κυμάτων (Wave Envelopes) σε μη γραμμικά και διασπαιρετικά (Dispersion) μέσα, δείχνοντας πώς αυτά τα κύματα μπορούν να διατηρήσουν το σχήμα τους καθώς κινούνται.

Σκοπός είναι να προσεγγίσουμε την πραγματική λύση $\psi(x, t)$ με ακριβή απώλεια μέσω Physics-Informed Neural Networks.

7.1 Περιοδικές Συνοριακές Συνθήκες

Στην Μερική Διαφορική Εξίσωση (7.1) εφαρμόζουμε μία Αρχική Συνθήκη (Initial Condition), δύο Περιοδικές Συνοριακές Συνθήκες (Periodic Boundary Conditions) και ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN.

$$(7.2) \quad \begin{cases} i \psi_t + p \psi_{xx} + q |\psi|^2 \psi = 0, & x \in [-L, L], t \in [0, T], \\ \psi(x, 0) = A \operatorname{sech}(\psi_0 x) e^{i c x}, & x \in [-L, L], \\ \psi(-L, t) = \psi(L, t), & t \in [0, T] \\ \psi_x(-L, t) = \psi_x(L, t), & t \in [0, T] \end{cases}$$

Για την (6.2) ορίσαμε τις μεταβλητές ως εξής

$$(7.3) \quad p = 0.5, q = 1, T = 5, L = 20, c = 0.5, A = 1$$

Για την προσέγγιση της λύσης της (7.2) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [84]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

7.1α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) παρέμεινε ίδια με την εξίσωση κύματος.

Τα βάρη της ολικής Συνάρτησης Απώλειας και ο Ρυθμός Μάθησης ορίστηκαν ως εξής

$$(7.4) \quad \text{Τα βάρη της ολικής Συνάρτησης Απώλειας} \quad \begin{cases} w_{psi} = 100, \\ w_b = 1, \\ w_0 = 10 \end{cases}$$

$$(7.5) \quad \text{Ρυθμός Μάθησης} = \begin{cases} 1 \times 10^{-3}, & \text{Εποχές} < 1000, \\ 1 \times 10^{-4}, & 1000 \leq \text{Εποχές} < 3000 \end{cases}$$

Με βάση τα (5.4, 7.5, 7.4) προσεγγίζουμε την (7.2) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 7.1

N_0	N_b	N_u	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
400	500	2000	2.27393140e-03	1.64322060e-04	2:18	0:36
300	500	2000	1.98073033e-03	1.72585758e-04	2:21	0:37
200	500	2000	1.74683658e-03	1.38616437e-04	2:21	0:37
100	500	2000	1.53363347e-02	3.63474392e-04	2:05	0:35
50	500	2000	2.84920656e-03	2.12717641e-04	2:28	0:36
400	400	2000	2.11582682e-03	1.77770853e-04	2:19	0:37
400	300	2000	1.74748630e-03	1.39053649e-04	2:15	0:34
400	200	2000	1.62866339e-02	2.89134739e-04	2:15	0:33
400	100	2000	3.95906810e-03	2.98866362e-04	2:17	0:34
400	50	2000	7.90615566e-03	3.73484014e-04	2:10	0:34
400	10	2000	3.67938611e-03	3.37242614e-04	2:09	0:33
400	500	1800	3.82966571e-03	2.64313334e-04	2:16	0:36
400	500	1600	7.06134550e-03	3.95908253e-04	2:05	0:32
400	500	1400	2.14783894e-03	1.40389457e-04	1:55	0:29
400	500	1200	1.64861128e-01	1.02145481e-03	1:48	0:26
400	500	1000	2.84381560e-03	2.45868810e-04	1:38	0:25
400	500	800	3.15537001e-03	2.17197332e-04	1:30	0:23
400	500	600	2.44074780e-03	1.30506582e-04	1:23	0:20
400	500	400	1.15125859e-03	1.07299842e-04	1:16	0:19
400	500	200	5.91407996e-04	8.09696430e-05	1:08	0:17
300	400	1500	2.02532113e-03	1.66697209e-04	1:54	0:31
300	400	1000	1.89197576e-03	1.51515094e-04	1:35	0:24
300	400	500	1.62161654e-03	1.36691815e-04	1:16	0:20
200	300	1500	1.46771595e-03	1.07016211e-04	1:52	0:28
200	300	1000	3.80333001e-03	3.00812855e-04	1:34	0:26
200	300	500	3.28521128e-03	1.53900866e-04	1:14	0:19
100	200	1500	9.56515521e-02	1.06500322e-03	1:48	0:27
100	200	1000	2.52227718e-03	1.32462417e-04	1:31	0:23
100	200	500	1.52260577e-03	1.00334029e-04	1:11	0:18
50	100	1500	1.41535292e-03	9.78328608e-05	1:47	0:28
50	100	1000	1.41293956e-02	9.84773622e-04	1:28	0:22
50	100	500	4.33199434e-03	2.41193498e-04	1:11	0:17

Πίνακας 7.1: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ NLS με Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε CPU

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Από τον Πίνακα 7.1 μπορούν να εξαχθούν τα εξής συμπεράσματα

Τα αποτελέσματα της συνολικής Loss Function μέσω της Μεθόδου βελτιστοποίησης Adam παρατηρούμε ότι παραμένουν της ίδιας τάξης μεγέθους, 10^{-3} , για όλες τις εξεταζόμενες επιλογές σημείων εκπαίδευσης. Όμως, παρατηρούνται διακυμάνσεις που υποδηλώνουν ότι η σύγκλιση της Adam επηρεάζεται από τη σύνθεση του συνόλου εκπαίδευσης και όχι μόνο από το συνολικό πλήθος των σημείων.

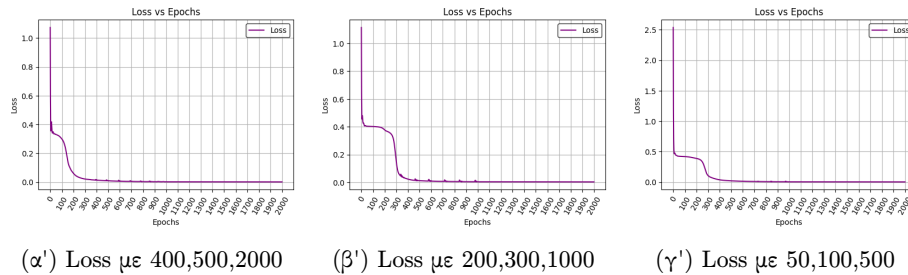
Ως προς τον υπολογιστικό χρόνο της Adam, προκύπτει ότι η αύξηση του πλήθους των σημείων εκπαίδευσης οδηγεί γενικά σε μεγαλύτερους χρόνους εκπαίδευσης, με τον αριθμό των collocation points N_u να αποτελεί τον κυρίαρχο παράγοντα κόστους.

Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους 10^{-4} .

Τα αποτελέσματα δείχνουν ότι η αύξηση του πλήθους των σημείων εκπαίδευσης δεν οδηγεί κατ' ανάγκη σε μείωση της συνολικής Loss Function, ενώ ταυτόχρονα μπορεί να αυξήσουν το υπολογιστικό κόστος σημαντικά, για το συγκεκριμένο πρόβλημα.

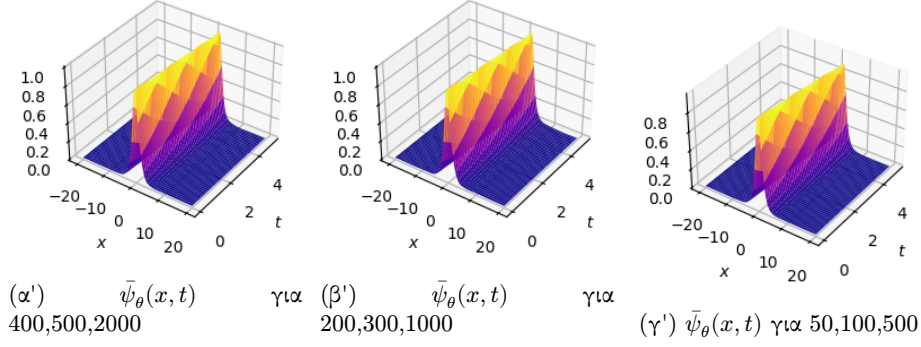
Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

Με βάση τα παραπάνω, παίρνουμε τα γραφήματα 7.1, 7.2, 7.3 και 7.4:

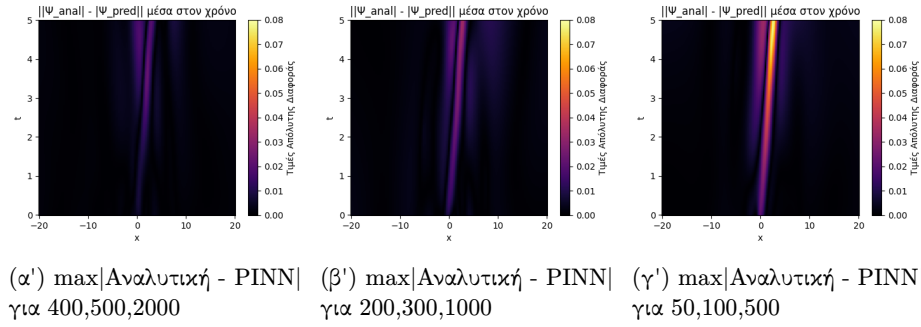


Σχήμα 7.1: Διαγράμματα της συνολικής Συνάρτησης Απώλειας μετά από Adam. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (7.1) παρατηρούμε ότι, για τις διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης, η εξέλιξη της συνολικής συνάρτησης απώλειας ως προς τις εποχές παρουσιάζει παρόμοια συμπεριφορά σύγκλισης. Το γεγονός αυτό υποδηλώνει ότι το δίκτυο μπορεί να εκπαιδευτεί αποτελεσματικά χωρίς την ανάγκη πολύ μεγάλου αριθμού σημείων, διατηρώντας μια σχετικά καλή προσέγγισης, όπως φαίνεται και στο Σχήμα (7.2).

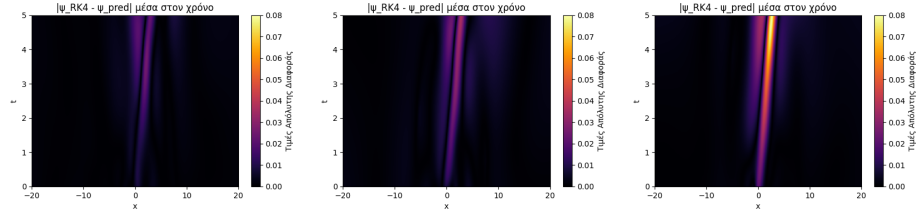


Σχήμα 7.2: Γραφήματα της προσέγγισης της λύσης με PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500



Σχήμα 7.3: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (5.3) παρατηρούμε ότι, όσο λιγότερο πλήθος εκπαίδευσης θεωρούμε, τόσο η συμπεριφορά της προσεγγιστικής λύσης αποκλίνει από την αναλυτική λύση.



(α') $\max|\text{RK4} - \text{PINN}|$ για 400,500,2000 (β') $\max|\text{RK4} - \text{PINN}|$ για 200,300,1000 (γ') $\max|\text{RK4} - \text{PINN}|$ για 50,100,500

Σχήμα 7.4: Μέγιστη Απόλυτη Διαφορά μεταξύ της Προσεγγιστικής Λύσης RK4 και της Προσεγγιστικής Λύσης PINN. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u στα αριστερά 400,500,2000, στην μέση 200,300,1000 και στα δεξιά 50,100,500

Στο Σχήμα (7.4) παρατηρούμε ότι, τα γραφήματα της μέγιστης απόλυτης διαφοράς μεταξύ της μεθόδου Runge–Kutta 4 και του PINN και εκείνα της μέγιστης απόλυτης διαφοράς μεταξύ της αναλυτικής λύσης και του PINN είναι πρακτικά ταυτόσημα. Το αποτέλεσμα αυτό υποδηλώνει ότι η μέθοδος Runge–Kutta 4 προσεγγίζει ουσιαστικά την αναλυτική λύση και παρέχει καλύτερη προσέγγιση από το PINN, για το συγκεκριμένο πρόβλημα.

Για την μέθοδο Πεπερασμένων Διαφορών αναπτύχθηκε το πρόγραμμα [[85]]

7.1β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (7.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (7.4) παρέμειναν ίδια.

Με βάση τα (5.4, 7.5, 7.4) προσεγγίζουμε την (7.2) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 7.2

N_0	N_b	N_u	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
400	500	2000	2.73256493e-03	1.98757654e-04	1:26	0:26
300	500	2000	2.66400306e-03	1.81128285e-04	1:26	0:25
200	500	2000	2.67714192e-03	1.68505605e-04	1:26	0:25
100	500	2000	2.97663035e-03	2.20219634e-04	1:26	0:26
50	500	2000	3.02442443e-03	1.93819462e-04	1:26	0:26
400	400	2000	2.75348080e-03	1.99052767e-04	1:26	0:26
400	300	2000	2.73413141e-03	2.02733208e-04	1:26	0:25
400	200	2000	2.76891235e-03	1.88842809e-04	1:26	0:26
400	100	2000	2.78151361e-03	1.93708780e-04	1:26	0:25
400	50	2000	2.74100853e-03	1.99720947e-04	1:25	0:26
400	10	2000	2.80545373e-03	1.99349626e-04	1:26	0:26
400	500	1800	2.70119915e-03	1.81001567e-04	1:26	0:26
400	500	1600	2.63468199e-03	1.93317930e-04	1:26	0:26
400	500	1400	2.79980781e-03	1.94847089e-04	1:24	0:25
400	500	1200	2.76471348e-03	1.68465078e-04	1:25	0:26
400	500	1000	2.39427830e-03	1.50946260e-04	1:23	0:25
400	500	800	2.40478152e-03	1.69910112e-04	1:23	0:25
400	500	600	2.25692987e-03	1.32754838e-04	1:25	0:26
400	500	400	2.05701101e-03	1.39547294e-04	1:24	0:25
400	500	200	9.93398018e-04	1.59090894e-04	1:26	0:25
300	400	1500	2.67405482e-03	1.68120168e-04	1:26	0:26
300	400	1000	2.39869626e-03	1.41601864e-04	1:24	0:25
300	400	500	2.10673758e-03	1.49130719e-04	1:25	0:26
200	300	1500	2.66710436e-03	1.61040924e-04	1:24	0:25
200	300	1000	2.32542912e-03	1.28813568e-04	1:25	0:26
200	300	500	2.10384163e-03	1.36862043e-04	1:25	0:26
100	200	1500	2.92377826e-03	1.99609232e-04	1:25	0:26
100	200	1000	2.74918135e-03	2.09665988e-04	1:26	0:26
100	200	500	2.25520949e-03	1.78892602e-04	1:25	0:26
50	100	1500	2.90098554e-03	2.08727288e-04	1:25	0:26
50	100	1000	2.60032364e-03	1.41357086e-04	1:25	0:26
50	100	500	2.16589496e-03	1.20624609e-04	1:25	0:25

Πίνακας 7.2: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ NLS με Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε GPU

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές πλήθους σημείων εκπαίδευσης. Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 1:30 λεπτά, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε περίπου 25 δευτερόλεπτα, γεγονός που υποδηλώνει περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Αντίθετα, κατά την εκτέλεση των υπολογισμών στην CPU, ο χρόνος εκπαίδευσης εξαρτάται από το πλήθος και την σύνθεση των σημείων εκπαίδευσης, στο συγκεκριμένο πρόβλημα. Συγκεκριμένα, η βελτιστοποίηση με Adam απαιτεί από 1 έως περίπου 2 λεπτά, ενώ η L-BFGS κυμαίνεται από 17 μέχρι περίπου 37 δευτερόλεπτα.

Ως προς την τάξη της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam έχει ίδια τάξη μεγέθους 10^{-3} , όπως και στην GPU με χρήση CUDA. Επίσης, στη μέθοδο L-BFGS, στη CPU έχει τάξη μεγέθους 10^{-4} , όπως και στην GPU με χρήση CUDA, γεγονός που υποδηλώνει ομοιόμορφη συμπεριφορά της διαδικασίας βελτιστοποίησης και στις δύο περιπτώσεις.

Καταλήγουμε στο συμπέρασμα ότι, η χρήση CPU είναι επαρκής για προβλήματα μικρού συνόλου δεδομένων, καθώς επιτυγχάνονται συγκρίσιμες τιμές της συνολικής Loss Function. Ωστόσο, καθώς αυξάνεται το πλήθος των σημείων εκπαίδευσης, ο χρόνος εκπαίδευσης στην CPU αυξάνεται αισθητά. Αντίθετα, η GPU με χρήση CUDA επιτρέπει ταχύτερη και πιο αποδοτική εκπαίδευση, με τον χρόνο να παραμένει σχεδόν σταθερός ακόμη και για μεγαλύτερα σύνολα δεδομένων, χωρίς να επηρεάζεται αρνητικά η σύγκλιση της συνάρτησης απώλειας. Για τον λόγο αυτό, στο συγκεκριμένο πρόβλημα, η GPU αποτελεί πιο κατάλληλη επιλογή για την εκπαίδευση των PINNs, ιδίως όταν έχουμε μεγάλο αριθμό σημείων εκπαίδευσης.

7.2 Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες

Στην Μερική Διαφορική Εξίσωση (7.1) εφαρμόζουμε μία Αρχική Συνθήκη (Initial Condition), δύο Περιοδικές Συνοριακές Συνθήκες (Periodic Boundary Conditions).

Σε αντίθεση με την προηγούμενη ενότητα, ορίζουμε το χωρίο στο οποίο εκπαιδεύεται το PINN να είναι το μη συνεκτικό ως προς τον χρόνο

$$x \in \Omega = [-L, \alpha] \cup [\beta, L], \quad -L < \alpha < \beta < L,$$

$$(7.6) \quad \begin{cases} i\psi_t + p\psi_{xx} + q|\psi|^2\psi = 0, & x \in [-L, L], t \in [0, T], \\ \psi(x, 0) = A \operatorname{sech}(\psi_0 x) e^{icx}, & x \in [-L, \alpha] \cup [\beta, L], \\ \psi(-L, t) = \psi(L, t), & t \in [0, T] \\ \psi_x(-L, t) = \psi_x(L, t), & t \in [0, T] \end{cases}$$

Για την (7.6) ορίσαμε τις ίδιες μεταβλητές (7.3).

Για την προσέγγιση της λύσης της (7.6) μέσω PINNs, αναπτύχθηκε το πρόγραμμα [86]. Τα αποτελέσματα της εκπαίδευσης και οι αντίστοιχοι πίνακες προέκυψαν από την εκτέλεση του συγκεκριμένου κώδικα.

7.2α' Εκπαίδευση σε CPU

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (7.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (7.4) παρέμειναν ίδια.

Για τις παραμέτρους α και β επιλέχθηκαν συμμετρικές τιμές ως προς το κέντρο του διαστήματος $[-L, L]$, ώστε το αφαιρούμενο υποδιάστημα (α, β) να οδηγεί σε ομοιόμορφη διάσπαση του χωρίου ορισμού.

Με βάση τα (5.4, 7.5, 7.4) προσεγγίζουμε την (7.6) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 7.3

$[-L, \alpha] \cup [\beta, L]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-20, -0.0] \cup [0.0, 20]$	2.27829115e-03	1.62075681e-04	1:26	0:21
$[-20, -0.5] \cup [0.5, 20]$	2.07295781e-03	1.44060585e-04	1:26	0:22
$[-20, -1.0] \cup [1.0, 20]$	2.18461314e-03	2.67024472e-04	1:28	0:21
$[-20, -1.5] \cup [1.5, 20]$	3.58770555e-03	1.89763494e-04	1:27	0:23
$[-20, -2.0] \cup [2.0, 20]$	1.49910888e-02	4.79323440e-04	1:25	0:21
$[-20, -2.5] \cup [2.5, 20]$	7.53989769e-03	4.12171852e-04	1:27	0:22
$[-20, -3.0] \cup [3.0, 20]$	2.42206571e-03	4.77655383e-04	1:26	0:22

Πίνακας 7.3: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε CPU και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Καθώς οι χρόνοι εκπαίδευσης υπολογίστηκαν στο περιβάλλον Google Colab, ενδέχεται να παρουσιάζουν μικρές αποκλίσεις ανά εκτέλεση.

Τα N_0 , N_b , και N_u αντιστοιχούν στο σύνολο των σημείων εκπαίδευσης για τις Αρχικές Συνθήκες, για τις Περιοδικές Συνοριακές Συνθήκες και για το εσωτερικό του χωρίου.

Το $Loss_{Adam}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης Adam.

Το $Loss_{LBFGS}$ αναφέρεται στο τελευταίο σφάλμα της εκπαίδευσης μέσω της Μεθόδου Βελτιστοποίησης L-BFGS.

Τα $Time_{Adam}$ και $Time_{LBFGS}$ αντιστοιχούν στον συνολικό χρόνο εκπαίδευσης των μεθόδων αντίστοιχα.

Από τον Πίνακα 7.3 μπορούν να εξαχθούν τα εξής συμπεράσματα

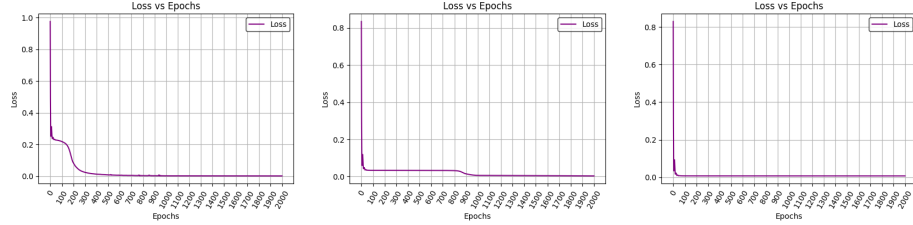
Από τα αποτελέσματα της συνολικής Loss Function, μέσω της βελτιστοποίησης Adam, παρατηρούμε ότι υπάρχει σταδιακή αύξηση της τάξης μεγέθους, από 10^{-3} σε 10^{-2} , γεγονός που υποδηλώνει ότι η μείωση των δεδομένων εκπαίδευσης, δηλαδή η αύξηση του υποδιαστήματος (α, β) , των Αρχικών Συνθηκών καθιστά το συγκεκριμένο πρόβλημα δυσκολότερο για το δίκτυο. Συγκεκριμένα, από το διάστημα $[-20, -2.5] \cup [2.5, 20]$ το δίκτυο αδυνατεί να προσεγγίσει την αναλυτική λύση, παρά του ότι η Loss Function μειώνεται.

Μετά την ολοκλήρωση της εκπαίδευσης με Adam, η εφαρμογή της Μεθόδου Βελτιστοποίησης L-BFGS οδηγεί σε περαιτέρω μείωση της συνολικής συνάρτησης απώλειας. Τα αποτελέσματα της συνολικής Loss Function ύστερα από την εφαρμογή της μεθόδου L-BFGS έχουν τάξης μεγέθους 10^{-4} .

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι ο συνδυασμός αρχικής εκπαίδευσης με την Μέθοδο Βελτιστοποίησης Adam και τελικής βελτιστοποίησης με

την Μέθοδο Βελτιστοποίησης L-BFGS αποτελεί αποδοτική στρατηγική για την εκπαίδευση PINNs σε προβλήματα ΜΔΕ, καθώς έχουμε αισθητή μείωση της συνολικής Loss Function.

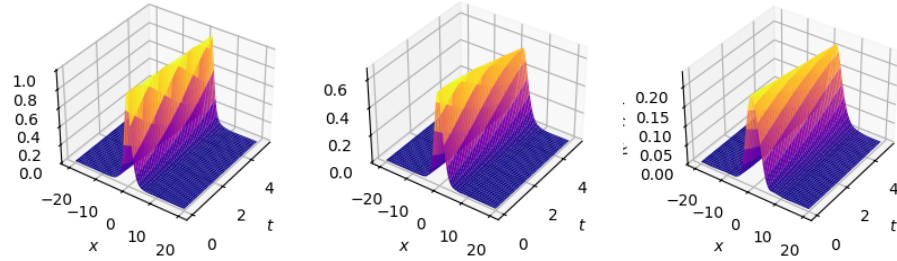
Με βάση τα παραπάνω, παίρνουμε τα γραφήματα 7.5, 7.6, 7.7:



(α') Loss με Α.Σ. στο $[-20, -0.5] \cup [0.5, 20]$ (β') Loss με Α.Σ. στο $[-20, -1.5] \cup [1.5, 20]$ (γ') Loss με Α.Σ. στο $[-20, -2.5] \cup [2.5, 20]$

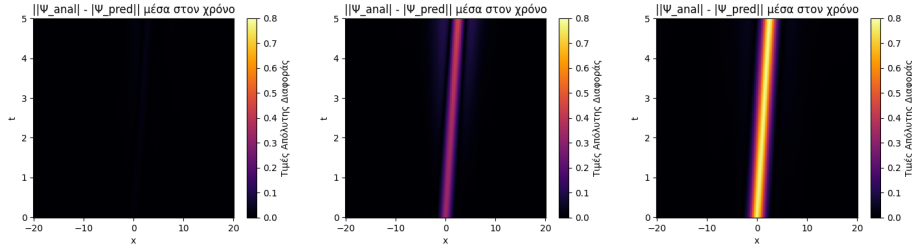
Σχήμα 7.5: Γραφήματα της Προσεγγιστικής λύσης PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400,500,2000.

Στο Σχήμα (7.5) παρατηρούμε ότι, για διαφορετικές επιλογές του υποδιαστήματος (α, β) , η Loss Function συγκλίνει όλο και πιο γρήγορα στην ελάχιστη τιμή. Όμως, όπως φαίνεται στο Σχήμα (7.6), η ελαχιστοποίηση της συνολικής Loss Function, από μόνη της, δεν αποτελεί ούτε μοναδική ούτε επαρκή συνθήκη για να εξασφαλιστεί ότι η εκπαίδευση έγινε προς την σωστή λύση.



(α') $\bar{\psi}_\theta(x, t)$ με Α.Σ. στο $[-20, -0.5] \cup [0.5, 20]$ (β') $\bar{\psi}_\theta(x, t)$ με Α.Σ. στο $[-20, -1.5] \cup [1.5, 20]$ (γ') $\bar{\psi}_\theta(x, t)$ με Α.Σ. στο $[-20, -2.5] \cup [2.5, 20]$

Σχήμα 7.6: Γραφήματα της προσέγγισης της λύσης με PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400,500,2000.



(α') $\max |\text{Αναλυτική} - \text{PINN}|$ με Α.Σ. στο $[-20, -0.5] \cup [0.5, 20]$ (β') $\max |\text{Αναλυτική} - \text{PINN}|$ με Α.Σ. στο $[-20, -1.5] \cup [1.5, 20]$ (γ') $\max |\text{Αναλυτική} - \text{PINN}|$ με Α.Σ. στο $[-20, -2.5] \cup [2.5, 20]$

Σχήμα 7.7: Μέγιστη Απόλυτη Διαφορά μεταξύ της Αναλυτικής Λύσης και της Προσεγγιστικής Λύσης PINN με κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. Η εκπαίδευση έγινε για το πλήθος των N_0 , N_b , και N_u 400, 500, 2000.

Στο Σχήμα (7.7) παρατηρούμε ότι υπάρχει ένα όριο στο πόσο μεγάλο μέρος μπορούμε να εξαιρέσουμε από το σύνολο εκπαίδευσης των Αρχικών Συνθηκών, για το συγκεκριμένο πρόβλημα. Αυτό το όριο, από τα παραπάνω αποτελέσματα δεν μπορεί να είναι μεγάλο, συμπαιραίνοντας ότι το πρόβλημα είναι εύαισθητο σε όλο και λιγότερα δεδομένα.

7.2β' Εκπαίδευση σε CUDA

Η αρχιτεκτονική του προγράμματος PINN (5.4) καθώς ο Ρυθμός Μάθησης (7.5) και τα βάρη της ολικής Συνάρτησης Απώλειας (7.4) παρέμειναν ίδια.

Για τις παραμέτρους α και β επιλέχθηκαν συμμετρικές τιμές ως προς το κέντρο του διαστήματος $[-L, L]$, ώστε το αφαιρούμενο υποδιάστημα (α, β) να οδηγεί σε ομοιόμορφη διάσπαση του χωρίου ορισμού.

Με βάση τα (5.4, 7.5, 7.4) προσεγγίζουμε την (7.6) και τα αποτελέσματα από την ολική Συνάρτηση Απώλειας μέσω βελτιστοποίησης Adam και ύστερα από την L-BFGS παρουσιάζονται στον Πίνακα 7.4

$[-L, \alpha] \cup [\beta, L]$	$Loss_{Adam}$	$Loss_{LBFGS}$	$Time_{Adam}$	$Time_{LBFGS}$
$[-20, -0.0] \cup [0.0, 20]$	2.73256493e-03	1.98757654e-04	1:24	0:25
$[-20, -0.5] \cup [0.5, 20]$	3.04127368e-03	2.81040440e-04	1:25	0:25
$[-20, -1.0] \cup [1.0, 20]$	3.34600173e-03	3.24779947e-04	1:25	0:25
$[-20, -1.5] \cup [1.5, 20]$	4.04406805e-03	1.99986709e-04	1:24	0:25
$[-20, -2.0] \cup [2.0, 20]$	5.30570373e-03	2.03668693e-04	1:25	0:25
$[-20, -2.5] \cup [2.5, 20]$	5.59728500e-03	3.52276897e-04	1:25	0:25
$[-20, -3.0] \cup [3.0, 20]$	2.27769930e-03	3.78917845e-04	1:25	0:26

Πίνακας 7.4: Αποτελέσματα συνολικών Συναρτήσεων Απώλειας για την ΜΔΕ Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Περιοδικές Συνοριακές Συνθήκες με εκπαίδευση σε GPU και πλήθος σημείων $N_0 = 400$, $N_b = 500$, $N_u = 2000$

Κατά την εκπαίδευση του PINN στην GPU με χρήση CUDA, ο συνολικός χρόνος εκπαίδευσης παρουσιάζει μικρές διακυμάνσεις για διαφορετικές επιλογές

πλήθους σημείων εκπαίδευσης. Συγκεκριμένα, η μέθοδος Adam απαιτεί περίπου 1:30 λεπτά, ενώ η μέθοδος L-BFGS ολοκληρώνεται σε περίπου 25 δευτερόλεπτα, γεγονός που υποδηλώνει περιορισμένη ευαισθησία του χρόνου εκπαίδευσης ως προς το πλήθος και την σύνθεση των σημείων εκπαίδευσης.

Κατά την εκτέλεση των υπολογισμών στην CPU, ο χρόνος εκπαίδευσης επίσης δεν εξαρτάται από το πλήθος και την σύνθεση των σημείων εκπαίδευσης, στο συγκεκριμένο πρόβλημα. Συγκεκριμένα, η βελτιστοποίηση με Adam απαιτεί περίπου 1:30 λεπτά, ενώ η L-BFGS χρειάζεται περίπου 20 δευτερόλεπτα.

Ως προς την τάξη της συνολικής Loss Function, στην CPU παρατηρούμε ότι η μέθοδος Adam έχει ίδια τάξη μεγέθους 10^{-3} , όπως και στην GPU με χρήση CUDA. Επίσης, στη μέθοδο L-BFGS, στη CPU έχει τάξη μεγέθους 10^{-4} , όπως και στην GPU με χρήση CUDA, γεγονός που υποδηλώνει ομοιόμορφη συμπεριφορά της διαδικασίας βελτιστοποίησης και στις δύο περιπτώσεις.

Καταλήγουμε στο συμπέρασμα ότι, η χρήση CPU είναι επαρκής για προβλήματα μικρού συνόλου δεδομένων, καθώς επιτυγχάνονται συγκρίσιμες τιμές της συνολικής Loss Function. Ωστόσο, καθώς αυξάνεται το πλήθος των σημείων εκπαίδευσης, ο χρόνος εκπαίδευσης στην CPU αυξάνεται αισθητά. Αντίθετα, η GPU με χρήση CUDA επιτρέπει ταχύτερη και πιο αποδοτική εκπαίδευση, με τον χρόνο να παραμένει σχεδόν σταθερός ακόμη και για μεγαλύτερα σύνολα δεδομένων, χωρίς να επηρεάζεται αρνητικά η σύγκλιση της συνάρτησης απώλειας. Για τον λόγο αυτό, στο συγκεκριμένο πρόβλημα, η GPU αποτελεί πιο κατάλληλη επιλογή για την εκπαίδευση των PINNs, ιδίως όταν έχουμε μεγάλο αριθμό σημείων εκπαίδευσης.

Κεφάλαιο 8

Συνοπτικά Συμπεράσματα

Στην παρούσα εργασία παρουσιάστηκε συνοπτικά το ιστορικό και θεωρητικό υπόβαθρο της Τεχνητής Νοημοσύνης και των Νευρωνικών Δικτύων, με έμφαση στα Physics-Informed Neural Networks (PINNs) για την προσέγγιση λύσεων γραμμικών και μη γραμμικών μερικών διαφορικών εξισώσεων.

Στην περίπτωση γραμμικών προβλημάτων μελετήθηκε η Εξίσωση Κύματος υπό περιοδικές, καθώς και Dirichlet και Neumann συνοριακές συνθήκες. Για τις μη γραμμικές εξισώσεις εξετάστηκαν οι KdV και NLS με περιοδικές συνοριακές συνθήκες. Οι προσεγγιστικές λύσεις των PINNs συγκρίθηκαν τόσο με αναλυτικές λύσεις (όπου αυτές ήταν διαθέσιμες) όσο και με αριθμητικές λύσεις που προέκυψαν μέσω της Μεθόδου Πεπερασμένων Διαφορών σε συνδυασμό με σχήμα Runge–Kutta τέταρτης τάξης.

Από τα αριθμητικά αποτελέσματα προέκυψε ότι, σε όλα τα προβλήματα που μελετήθηκαν, οι κλασικές μέθοδοι εμφανίζουν καλύτερη προσέγγιση και ακρίβεια και μικρότερο υπολογιστικό κόστος σε σύγκριση με τα PINNs.

Αυτο όμως ισχύει για προβλήματα ορισμένα σε χαμηλές διαστάσεις. Στις κλασικές αριθμητικές μεθόδους, όπως οι Πεπερασμένες Διαφορές ή τα Πεπερασμένα Στοιχεία, το υπολογιστικό κόστος αυξάνει εκθετικά με την αύξηση των διαστάσεων του προβλήματος.

Αν θεωρήσουμε ένα πρόβλημα σε d χωρικές διαστάσεις και επιλέξουμε N σημεία διακριτοποίησης ανά διάσταση, το υπολογιστικό κόστος των κλασικών αριθμητικών μεθόδων αυξάνεται εκθετικά ως προς τη διάσταση d ,

$$(8.1) \quad \mathcal{O}(N^d)$$

Στην βιβλιογραφία η εκθετική αύξηση του κόστους αναφέρεται ως Curse of Dimensionality και καθιστά την εφαρμογή των κλασικών μεθόδων απαγορευτική σε προβλήματα υψηλής διάστασης $d \geq 5$.

Αντιθέτως, τα PINNs δεν απαιτούν ρητή κατασκευή πλέγματος στον χώρο. Η υπολογιστική τους πολυπλοκότητα εξαρτάται κυρίως από:

- τον αριθμό των collocation points,
- το πλήθος των σημείων εκπαίδευσης στα σύνορα,

- τον αριθμό των βημάτων εκπαίδευσης,
- το μέγεθος του δικτύου (πλήθος νευρώνων και πλήθος hidden layers)

Το συνολικό υπολογιστικό κόστος των PINNs μπορεί να εκτιμηθεί ως

$$(8.2) \quad \mathcal{O}((dN_e + N_l N_e^2 + N_e)(N_{\text{epochs}})(N_0 + N_b + N_u))$$

- d : Αριθμός διάστασης προβλήματος,
- N_e : Πλήθος συνδέσεων του Input Layer με το πρώτο Hidden Layer
- N_l : Πλήθος συνδέσεων μεταξύ των Hidden Layers
- N_e : Πλήθος συνδέσεων μεταξύ του τελευταίου Hidden Layer με το Output Layer
- N_{epochs} : Πλήθος επαναλήψεων εκπαίδευσης (π.χ. Adam και L-BFGS)
- N_0 : Πλήθος δεδομένων εκπαίδευσης Αρχικής Συνθήκης
- N_b : Πλήθος δεδομένων εκπαίδευσης Συνοριακής Συνθήκης
- N_u : Πλήθος δεδομένων στο εσωτερικό της MΔΕ, Collocation Points

Η εκτίμηση 8.2 δεν παρουσιάζει εκθετική εξάρτηση από τη διάσταση d του προβλήματος. Έτσι, σε υψηλές διαστάσεις, όπου οι πλεγματικές μέθοδοι επιβαρύνονται εκθετικά, τα PINNs παρουσιάζουν συγκριτικό πλεονέκτημα. [46]

Συνολικά, τα PINNs δεν αντικαθιστούν τις κλασικές αριθμητικές μεθόδους, αλλά αποτελούν ένα συμπληρωματικό εργαλείο, ιδιαίτερα υποσχόμενο σε περιβάλλοντα όπου το Curse of Dimensionality περιορίζει την αποτελεσματικότητα των πλεγματικών τεχνικών.

Παράρτημα Α

Παρακάτω παρουσιάζεται ο κώδικας υλοποίησης ενός Physics-Informed Neural Network (PINN) για την επίλυση της εξίσωσης Κύματος με Περιοδικές Συνοριακές Συνθήκες. Όλοι οι σχετικοί κώδικες μπορούν να βρεθούν στο [GitHub repository](https://github.com/Wave_Periodic_Adam_LBFGS_matlab_master).

```
1  # -*- coding: utf-8 -*-
2  """Wave_Periodic_Adam_LBFGS_matlab_master.ipynb
3
4  Automatically generated by Colab.
5
6  Original file is located at
7      https://colab.research.google.com/drive/1GOVNUuyNd14_G55MrcWw1IG_-2Dc-6PF
8
9  # Μερική** Διαφορική Εξίσωση Κύματος με PINN**
10
11  ### Προσεγγίζουμε** την εξίσωση κύματος:**
12
13  $$
14  u_{tt} = c^2 u_{xx}
15  $$
16
17  ### Με**:**
18
19  $$c = 1$$
20  $$x \in [-L, L], \ ; \ L = 2 \ $$
21  $$t \in [0, T], \ ; \ T = 5 \ $$
22
23  ---
24
25  ### Αρχικές** συνθήκες: θέση( και ταχύτητα)**
26
27  $$
28  u(x, 0) = f(x) = A \sin \left(\frac{\pi}{L} x \right)
29  $$
30
31  $$
32  u_t(x, 0) = g(x) = -A \frac{\pi}{L} c \cos \left(\frac{\pi}{L} x \right)
33  $$
34
35  ---
36
37  ### Συνοριακές** συνθήκες: Περιοδικές(**
38
39  $$
40  u(-L, t) = u(L, t)
41  $$
42
43  $$
```

```

44 u_x(-L, t) = u_x(L, t)
45 $$
46 """
47
48 import torch #βασικό πακέτο PyTorch
49 import torch.nn as nn #έτοιμες συναρτήσεις για nn
50 import torch.nn.functional as f #συναρτήσεις ενεργοποίησης
51 import torch.optim as optim #βελτιστοποιητές
52 import numpy as np #βιβλιοθήκη αριθμητικών υπολογισμών
53 from time import time #μέτρηση χρόνου
54 import gc #για καθαρισμό cpu
55 import platform #για το τι συσκευή cpu χρησιμοποιώ
56
57
58 use_cuda = input("Να κεράσω CUDA; ναλοχι(/): ").strip().lower()
59
60 if use_cuda == "ναι":
61     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
62 else:
63     device = torch.device("cpu")
64
65 print("Χρήση συσκευής:", device)
66 print(torch.__version__)
67
68 print(torch.cuda.is_available())
69 if torch.cuda.is_available():
70     print("Κερασμένη η CUDA!")
71     print("Current device:", torch.cuda.current_device())
72     print("Device name:", torch.cuda.get_device_name(0))
73     torch.cuda.empty_cache() # άδειασμα gpu
74 else:
75     print("Μας τελείωσε η CUDA...")
76     print(platform.uname())
77     gc.collect() # άδειασμα cpu
78
79 #Ζέσταμα GPU για να μην πετάει warning
80 if device.type == "cuda":
81     torch.cuda.init()
82     torch.cuda.empty_cache()
83     torch.randn(8, 8, device=device) #warm-up cuBLAS
84
85 #ορισμός ακρίβειας
86 dtype = torch.float32
87
88 #σταθερές
89 pi = torch.tensor(np.pi, dtype=dtype, device=device) #π
90 wave_speed = torch.tensor(1, dtype=dtype, device=device) #ταχύτητα διάδοσης c
91 L = 2 #πλάτος χώρου
92 L_min = torch.tensor(-L, dtype=dtype, device=device) #πλάτος χώρου ελάχιστο
93 L_max = torch.tensor(L, dtype=dtype, device=device) #πλάτος χώρου μέγιστο
94 T_max = torch.tensor(5, dtype=dtype, device=device) #χρόνος
95 m = torch.tensor(1, dtype=dtype, device=device) #δείκτης μορφής κύματος
96 k = m*pi/L #κυματάρθρωμος
97 A = torch.tensor(1, dtype=dtype, device=device) #σταθερά πλάτους κύματος
98
99 #Αρχικές Συνθήκες
100 def fun_u_0_th(t,x): #thθέση=
101     return A * torch.sin(k*x)
102
103 def fun_u_0_ta(t,x): #ταταχύτητα=
104     #n = t.shape[0] πλήθος# γραμμών
105     #return torch.zeros((n,1), dtype=dtype)

```

```

106     return -A * k * wave_speed * torch.cos(k*x)
107
108 #Συνοριακές Συνθήκες
109 def fun_u_b_left(t,x):
110     #Είτις περιοδικές δεν έχω αντίστοιχη συνάρτηση
111     #Ενσωματώνονται στο training loop και loss
112     return None
113
114 def fun_u_b_right(t,x):
115     #Είτις περιοδικές δεν έχω αντίστοιχη συνάρτηση
116     #Ενσωματώνονται στο training loop και loss
117     return None
118
119 #ΜΔΕ Εξίσωση Κύματος
120 def fun_r(u_tt, u_xx):
121     return u_tt - wave_speed**2 * u_xx
122
123 #Αριθμός σημείων
124 N_0 = 400 #πλήθος αρχικής συνθήκης  $u(x,0)$  και  $u_x(x,0)$ 
125 N_b = 500 #πλήθος συνοριακών συνθηκών  $u(0,t)$  και  $u(L,t)$ 
126 N_r = 2000 #πλήθος στο εσωτερικό του πεδίου εκπαίδευσης της ΜΔΕ
127
128 #Συνοριακές τιμές
129 tmin = torch.tensor(0, dtype=dtype, device=device)
130 tmax = T_max
131 xmin = L_min
132 xmax = L_max
133
134 print(f'Τα όρια του χωροχρονικού πεδίου σε μορφή tensor:\n')
135 print(f'[tmin,tmax] x [xmin,xmax]')
136 print(f'[{tmin}, {tmax.item()}] x [{xmin}, {xmax.item()}]\n')
137
138 #Κάτω όριο
139 lb = torch.tensor([tmin, xmin], dtype=dtype, device=device)
140 #Πάνω όριο
141 ub = torch.tensor([tmax, xmax], dtype=dtype, device=device)
142
143 print(f'lb:', lb.device)
144 print(f'ub:', ub.device)
145
146 #Σταθεροποίηση γεννήτριας τυχαίων αριθμών
147 torch.manual_seed(67)
148
149 #Δημιουργία σημείων αρχικών συνθηκών
150
151
152 #στήλη με την αρχική τιμή του χρόνου
153 t_0 = torch.ones((N_0,1), dtype=dtype, requires_grad=True, device=device)*lb[0]
154 ###t_0 = torch.ones((N_0,1), dtype=dtype, device=device)*lb[0]
155
156 #στήλη με τυχαίους στο διάστημα [xmin,xmax]
157 x_0 = torch.rand((N_0,1), dtype=dtype, requires_grad=True, device=device)*(ub[1]-lb[1]) + lb[1]
158 ###x_0 = torch.rand((N_0,1), dtype=dtype, device=device)*(ub[1]-lb[1]) + lb[1]
159
160 #Συνδιασμός σε tensor (N0,2) δύο## στήλες t0,x0
161 X_0 = torch.concat([t_0, x_0], dim=1)
162
163 print(f't_0: ', t_0.shape)
164 print(f'x_0: ', x_0.shape)
165 print(f'X_0: ', X_0.shape)
166 print(f't_0: ', t_0.device)

```

```

167 print(f'x_0: ',x_0.device)
168 print(f'X_0: ',X_0.device)
169
170 #Υπολογισμός αρχικών συνθηκών
171 u_0_th = fun_u_0_th(t_0, x_0)
172 u_0_ta = fun_u_0_ta(t_0, x_0)
173
174 #Συνδιασμός σε tensor (N0,2)
175 u_0 = torch.concat([u_0_th, u_0_ta], dim=1)
176
177 print(f'u_0_th: ',u_0_th.shape)
178 print(f'u_0_ta: ',u_0_ta.shape)
179 print(f'u_0: ',u_0.shape)
180 print(f'u_0_th: ',u_0_th.device)
181 print(f'u_0_ta: ',u_0_ta.device)
182 print(f'u_0: ',u_0.device)
183
184 #Δημιουργία σημείων Συνοριακών Συνθηκών
185
186
187 #στήλη με τυχαίους στο διάστημα [tmin, tmax]
188 t_b = torch.rand((N_b,1), dtype=dtype, requires_grad=True, device=device)*(ub[0]-
    lb[0]) + lb[0]
189 ##t_b = torch.rand((N_b,1), dtype=dtype, device=device)*(ub[0]-lb[0]) + lb[0]
190
191 #στήλη με N_b στοιχεία για το xmin σύνορο αριστερά()
192 x_b_left = torch.ones((N_b, 1), dtype=dtype, requires_grad=True, device=device)*lb
    [1]
193 ##x_b_left = torch.ones((N_b, 1), dtype=dtype, device=device)*lb[1]
194
195 #στήλη με N_b στοιχεία για το xmax σύνορο δεξιά()
196 x_b_right = torch.ones((N_b, 1), dtype=dtype, requires_grad=True, device=device)*
    ub[1]
197 ##x_b_right = torch.ones((N_b, 1), dtype=dtype, device=device)*ub[1]
198
199 #Συνδιασμός σε tensor (N_b,2)
200 X_b = torch.concat([t_b, x_b_left, x_b_right], dim=1)
201
202 print(f't_b: ',t_b.shape)
203 print(f'x_b_left: ',x_b_left.shape)
204 print(f'x_b_right: ',x_b_right.shape)
205 print(f'X_b: ',X_b.shape)
206 print(f't_b: ',t_b.device)
207 print(f'x_b_left: ',x_b_left.device)
208 print(f'x_b_right: ',x_b_right.device)
209 print(f'X_b: ',X_b.device)
210
211 #Υπολογισμός Συνοριακών Συνθηκών
212
213 #Αυτό το κομμάτι κώδικα στις περιοδικές δεν χρειάζεται.
214 #Δεν έχουμε να υπολογίσουμε κάτι, θέλουμε απλά να είναι ίσες.
215
216 #Τυχαία σημεία λύσης
217
218 #Δημιουργία τυχαίων τιμών στο χρόνο [tmin, tmax]
219 t_r = torch.rand((N_r,1), dtype=dtype, requires_grad=True, device=device)*(ub[0]-
    lb[0]) + lb[0]
220
221 #Δημιουργία τυχαίων τιμών στο χώρο [xmin, xmax]
222 x_r = torch.rand((N_r,1), dtype=dtype, requires_grad=True, device=device)*(ub[1]-
    lb[1]) + lb[1]
223

```

```

224 #Συνδυασμός σημείων σε tensor (N_r,2)
225 X_r = torch.concat([t_r, x_r], dim=1)
226
227 print(f't_r: ', t_r.shape)
228 print(f'x_r: ', x_r.shape)
229 print(f'X_r: ', X_r.shape)
230 print(f't_r: ', t_r.device)
231 print(f'x_r: ', x_r.device)
232 print(f'X_r: ', X_r.device)
233
234 #Λίστες
235
236 #N(_0,2) αρχικά σημεία, N(_b,3) συνοριακά σημεία
237 X_data = [X_0, X_b]
238
239 #N(_0,2) αρχικές τιμές, N(_b,2) συνοριακές τιμές
240 u_data = [u_0]
241
242 print(f'X_data[0]==X_0: ', X_data[0].shape)
243 print(f'X_data[1]==X_b: ', X_data[1].shape)
244 print(f'u_data[0]==u_0: ', u_data[0].shape)
245 print(f'X_data[0]==X_0: ', X_data[0].device)
246 print(f'X_data[1]==X_b: ', X_data[1].device)
247 print(f'u_data[0]==u_0: ', u_data[0].device)
248
249 import matplotlib.pyplot as plt #βιβλιοθήκη γραφήματα
250
251 fig = plt.figure(figsize=(9,6)) #καμβάς 9x6 ίντσες
252 color = 'plasma' #επιλογή χρώματος γραφίματος
253
254 #σημεία αρχικής συνθήκης
255 plt.scatter(t_0.cpu().detach(), x_0.cpu().detach(), c=u_0[:, [0]].cpu().detach(),
256            marker='X', vmin=-1, vmax=1, cmap=color, label='Αρχικές Τιμές Θέσης')
257
258 #σημεία τυχαία
259 plt.scatter(t_r.cpu().detach(), x_r.cpu().detach(), c='c', marker='.', alpha=0.2,
260            label='Τυχαίες Τιμές')
261
262 plt.xlabel('$t$')
263 plt.ylabel('$x$')
264 plt.legend(loc='center').get_frame().set_edgecolor('black')
265 plt.title('Σημεία αρχικών και τυχαίων τιμών');
266 plt.show()
267
268 import matplotlib.pyplot as plt
269 from mpl_toolkits.mplot3d import Axes3D
270
271 fig = plt.figure(figsize=(12,6))
272 ax = fig.add_subplot(111, projection='3d')
273
274 #Αρχική Συνθήκη
275 ax.scatter(x_0.detach().cpu().numpy(), t_0.detach().cpu().numpy(), u_0_th.detach().cpu().numpy(), color='r', label='ΑΣ', s=2);
276
277 #σημεία τυχαία
278 ax.scatter(x_r.cpu().detach(), t_r.cpu().detach(), c='c', marker='.', alpha=0.2,
279            label='Τυχαίες Τιμές')
280
281 ax.set_xlabel('$x$')
282 ax.set_ylabel('$t$')
283 ax.set_zlabel('$u(t,x)$')
284 ax.set_title(r'ΑΣ και ΣΣ της ΜΔΕ Κύματος με NN:', fontsize=14);

```

```

282 ax.legend(loc='upper left', bbox_to_anchor=(1.05, 1))
283
284 plt.tight_layout()
285 plt.show()
286
287 # Μέγιστο πλάτος στο  $t_{max}$ 
288 U0_min = u_0_th.min().item()
289 U0_max = u_0_th.max().item()
290 print(f'Στο  $tT$  έχουμε min  $U_0 = \{U0\_min:.3f\}$ , max  $U_0 = \{U0\_max:.3f\}$ ')
291
292 #Δημιουργία NN
293
294 class NeuralNetwork(nn.Module):
295     def __init__(self, num_hidden_layers=8, num_neurons_per_layer=20):
296         super(NeuralNetwork, self).__init__()
297
298         #Το input layer συνδέεται στο o1 layer με ( 20 νευρώνες)
299         self.input_layer = nn.Linear(2, num_neurons_per_layer)
300
301         #Τα layers από το o1 μέχρι το o8
302         self.hidden_layers = nn.ModuleList([
303             nn.Linear(num_neurons_per_layer, num_neurons_per_layer)
304             for _ in range(num_hidden_layers)
305         ])
306
307         #Το output layer δίνει πραγματική τιμή
308         self.output_layer = nn.Linear(num_neurons_per_layer, 1)
309
310         #Αρχικοποίηση βαρών
311         self._initialize_weights()
312
313     def _initialize_weights(self):
314         for layer in self.hidden_layers:
315             #αποτρέπει τα πολύ μεγάλα και πολύ μικρά βάρη
316             nn.init.xavier_normal_(layer.weight)
317
318     def forward(self, x):
319         #Μετασχηματισμός του  $x$  για να βρίσκεται στο  $[lb[1], ub[1]] = [xmin, xmax]$ 
320          $l = [-1, 1]$  για ταχύτερη επκπαίδευση με την  $\tanh$ 
321          $x = 2.0 * (x - lb) / (ub - lb) - 1.0$ 
322
323         #Το input layer
324         x = torch.tanh(self.input_layer(x))
325         #ο1 με ο8 hidden layer
326         for layer in self.hidden_layers:
327             x = torch.tanh(layer(x))
328         #Το output layer
329         x = self.output_layer(x)
330         return x
331
332 #Υπολογισμός Απώλειας
333
334 #Μέσο τετραγωνικό σφάλμα
335 mse_loss = nn.MSELoss()
336
337 def compute_loss(model, X_r, X_data, u_data, epochs, verbose_flag):
338
339     #Απώλεια# ΜΔΕ
340     #Σπάσιμο στηλών  $t, x$  για να υπολογιστούν ξεχωριστά οι μερικές παράγωγοι
341     t = X_r[:, 0:1].detach().requires_grad_(True)
342     x = X_r[:, 1:2].detach().requires_grad_(True)

```



```

343
344 #Εκτίμηση ΜΔΕ μοντέλου
345 u = model(torch.concat([t,x],dim=1))
346
347 #Γπολογισμός Μερικών Παραγώγων
348 u_x = torch.autograd.grad(u, x, grad_outputs=torch.ones_like(u), create_graph=
349 True, allow_unused=True)[0]
350 u_xx = torch.autograd.grad(u_x, x, grad_outputs=torch.ones_like(u),
351 create_graph=True, allow_unused=True)[0]
352 u_t = torch.autograd.grad(u, t, grad_outputs=torch.ones_like(u), create_graph=
353 True, allow_unused=True)[0]
354 u_tt = torch.autograd.grad(u_t, t, grad_outputs=torch.ones_like(u),
355 create_graph=True, allow_unused=True)[0]
356
357 #Γπολογισμός του απώλειας της ΜΔΕ
358 l_u = torch.mean(torch.square(fun_r(u_tt, u_xx)))
359
360 #Απώλεια# αρχικής συνθήκης
361 #Σπάσιμο του t0,x0 για να υπολογιστεί ξεχωριστά η μερική παράγωγος
362 t0 = X_data[0][:,0:1].clone().detach().requires_grad_(True)
363 x0 = X_data[0][:,1:2].detach().requires_grad_(True)
364
365 #Εκτίμηση μοντέλου για την θέση και ταχύτητα
366 u0 = model(torch.concat([t0, x0],dim=1))
367
368 #Γπολογισμός μερικής παραγώγου
369 u0_t0 = torch.autograd.grad(u0, t0, grad_outputs=torch.ones_like(u0),
370 create_graph=True, allow_unused=True)[0]
371
372 #Γπολογισμός απώλειας θέσης και ταχύτητας
373 l_th = torch.mean(torch.square(u_data[0][:,0:1]-u0))
374 l_ta = torch.mean(torch.square(u_data[0][:,1:2]-u0_t0))
375 l_0 = l_th + l_ta
376
377 #Απώλεια# συνοριακών τιμών Περιοδικές()
378 #Σπάσιμο στηλών t,xl,xr για να υπολογιστούν ξεχωριστά οι μερικές παράγωγοι
379 tb = X_data[1][:,0:1].clone().detach().requires_grad_(True)
380 xbl = X_data[1][:,1:2].detach().requires_grad_(True)
381 xbr = X_data[1][:,2:3].detach().requires_grad_(True)
382
383 #Εκτίμηση μοντέλου Αριστερά
384 ubl = model(torch.concat([tb, xbl],dim=1))
385 ubl_x = torch.autograd.grad(ubl, xbl, torch.ones_like(ubl), create_graph=True)
386 [0]
387
388 #Εκτίμηση μοντέλου Δεξιά
389 ubr = model(torch.concat([tb, xbr],dim=1))
390 ubr_x = torch.autograd.grad(ubr, xbr, torch.ones_like(ubr), create_graph=True)
391 [0]
392
393 #Γπολογισμός απώλειας συνοριακών συνθηκών
394 l_lr = mse_loss(ubl, ubr)
395 l_lr_x = mse_loss(ubl_x, ubr_x)
396 l_b = l_lr + l_lr_x
397
398 #Προσαρμογή Βαρών κάθε loss
399 w_u = 1
400 w_b = 1
401 w_0 = 1

```

```

398     #Συνολική απώλεια
399     loss = w_u * l_u + w_b * l_b + w_0 * l_0
400
401     if (epochs % 100 == 0) and (verbose_flag != -1):
402         print(f'Epoch {epochs:05d}: l_u = {l_u:10.8e}, l_lr = {l_lr:10.8e}, l_lr_x =
          {l_lr_x:10.8e} l_0_th = {l_th:10.8e}, l_0_ta = {l_ta:10.8e} loss = {loss
          :10.8e}')
403
404     return loss
405
406 model = NeuralNetwork().to(device)
407
408 # Έλεγχος ότι το μοντέλο πήγε στη GPU
409 print("Model is on:", next(model.parameters()).device)
410
411 #Σταθερά learning rate
412 lr_schedule = [5e-4, 1e-4, 1e-5]
413 def get_lr(epoch):
414     if epoch < 100:
415         return lr_schedule[0]
416     elif epoch < 3000:
417         return lr_schedule[1]
418     else:
419         return lr_schedule[2]
420
421 #Adam βελτιστοποιητής
422 optimizer = optim.Adam(model.parameters(), lr=lr_schedule[0])
423
424 #Αριθμός εποχών
425 epochs=0
426 epoch = 2000
427 #Λίστα απώλειας κάθε εποχής
428 losses = []
429 #Καταγραφή χρόνου σε ( δευτερόλεπτα)
430 sec = time()
431
432 for i in range(epoch):
433
434     #Ενημέρωση της σταθεράς learning rate
435     for param_group in optimizer.param_groups:
436         param_group["lr"] = get_lr(i)
437
438     #Μηδενισμός των παραγώγων του βελτιστοποιητή πριν το backpropagation
439     optimizer.zero_grad()
440
441     #Υπολογισμός σφάλματος
442     loss = compute_loss(model, X_r, X_data, u_data, epochs, verbose_flag=1)
443
444     #Έλεγχος ποιες παράμετροι δεν είναι CUDA
445     for name, param in model.named_parameters():
446         if param.device.type != 'cuda':
447             print("Parameter on CPU:", name, param.device)
448
449     #Υπολογισμός των παραγώγων τρέχουσας εποχής
450     loss.backward(retain_graph=True)
451
452     #Ενημέρωση βαρών επόμενης εποχής
453     optimizer.step()
454
455     #Αποθήκευση του σφάλματος κάθε εποχής
456     losses.append(loss.item())
457

```

```

458     #Για την εκτύπωση του loss
459     epochs = epochs + 1
460
461     #Εφαρμογή του early stopping
462     if loss.item() < 1e-5:
463         break
464
465     #Εμφάνιση χρόνου εκτέλεσης του μοντέλου
466     seconds = time() - sec
467     days = seconds // (24*60*60)
468     hours = (seconds % (24*60*60)) // (60*60)
469     minutes = (seconds % (60*60)) // 60
470     second = int(seconds % 60)
471     print(f'\Χρόνος Εκπαίδευσης Μοντέλου: {int(days)} μέρες, {int(hours)} ώρες, {int(
472         minutes)} λεπτά, {int(second)} δευτερόλεπτα')
473     print(f'\nEpochs: {epochs}, loss = {losses[epochs-1]:10.8e}')
474
475     # Fine-tuning με L-BFGS
476     #Καταγραφή χρόνου σε ( δευτερόλεπτα)
477     sec = time()
478
479     lbfgs_counter = 0 #μετρητής βημάτων LBFGS
480
481     optimizer_lbfgs = torch.optim.LBFGS(model.parameters(), #μεταβολή των βαρών
482         του μοντέλου
483         max_iter=500, #μέγιστος αριθμός
484         εσωτερικών βημάτων
485         tolerance_grad=1e-7, #έξοδος όταν η μέση
486         κλίση είναι μικρότερη από
487         tolerance_change=1e-9, #έξοδος όταν τα βάρη
488         είναι μικρότερα από
489         history_size=50, #πόσα προηγούμενα
490         βήματα να θυμάται για την εκτίμηση της Hessian
491         line_search_fn="strong_wolfe") #για σταθερή
492         σύγκλιση, αποφυγεί πολύ μικρών/μεγάλων βημάτων
493
494     def closure():
495         optimizer_lbfgs.zero_grad() #μηδενισμός προηγούμενων gradients
496         loss = compute_loss(model, X_r, X_data, u_data, epochs, verbose_flag=-1) #
497         υπολογισμός απώλειας
498         loss.backward(retain_graph=True) #υπολογισμός gradients
499         global lbfgs_counter #για να την πάρει από έξω
500         lbfgs_counter = lbfgs_counter + 1 #μετρητής βημάτων LBFGS
501         return loss
502
503     #Μία κλήση LBFGS κάνει ( εσωτερικά 500 αλληλεπιδράσεις)
504     optimizer_lbfgs.step(closure)
505
506     #Τελική αξιολόγηση μετά το LBFGS
507     loss_lbfgs = compute_loss(model, X_r, X_data, u_data, 0, verbose_flag=-1)
508
509     print(f"LBFGS Επανάληψη {lbfgs_counter}")
510     print(f"LBFGS Σφάλμα {loss_lbfgs.item():10.8e}")
511     print(f"LBFGS βελτίωση: Από {losses[-1]:10.8e} σε {loss_lbfgs.item():10.8e}")
512     print(f"Διαφορά LBFGS με Adam: {(loss_lbfgs.item() - losses[-1]):+10.8e}")
513
514     #Εμφάνιση χρόνου εκτέλεσης του LBFGS
515     seconds = time() - sec
516     days = seconds // (24*60*60)
517     hours = (seconds % (24*60*60)) // (60*60)
518     minutes = (seconds % (60*60)) // 60
519     second = int(seconds % 60)

```

```

512 print(f'\Χρόνος Βελτιστοποίησης με LBFGS: {int(days)} μέρες, {int(hours)} ώρες, {
      int(minutes)} λεπτά, {int(second)} δευτερόλεπτα')
513
514 input(f"Να συνεχίσω το πρόγραμμα' ; ναλοχι(/): ").strip().lower()
515
516 #Απώλεια κάθε εποχής
517 fig = plt.figure(figsize=(6,4))
518
519 plt.plot(range(epoch), losses, label="Loss", color='purple')
520 plt.xlabel('Epochs')
521 plt.ylabel('Loss')
522 plt.title('Loss vs Epochs')
523 plt.legend(loc='upper right').get_frame().set_edgecolor('black')
524 plt.xticks(range(0, epoch+1, 100), rotation=60)
525 plt.grid(True)
526 plt.show()
527
528 import matplotlib.pyplot as plt
529 from mpl_toolkits.mplot3d import Axes3D
530
531 fig = plt.figure(figsize=(12,6))
532 ax = fig.add_subplot(111, projection='3d')
533
534 #Τελική εκτίμηση μοντέλου Αρχικής Συνθήκης
535 u0_m = model(torch.concat([t_0, x_0],dim=1))
536 print(f'u0_m: ',u0_m.shape)
537
538 #Τελική εκτίμηση μοντέλου Συνοριακής Συνθήκης Αριστερά
539 ubl_m = model(torch.concat([t_b, x_b_left],dim=1))
540 print(f'ubl_m: ',ubl_m.shape)
541
542 #Τελική εκτίμηση μοντέλου Συνοριακής Συνθήκης Δεξιά
543 ubr_m = model(torch.concat([t_b, x_b_right],dim=1))
544 print(f'ubr_m: ',ubr_m.shape)
545
546 #Αρχική Συνθήκη
547 ax.scatter(x_0.detach().cpu().numpy(), t_0.detach().cpu().numpy(), u0_m.detach().
      cpu().numpy(), color='r', label='ΑΕ', s=2);
548
549 #Συνοριακή Συνθήκη Αριστερά()
550 ax.scatter(x_b_left.detach().cpu().numpy(), t_b.detach().cpu().numpy(), ubl_m.
      detach().cpu().numpy(), color='g', label='ΣΕ αριστερά()', s=2);
551
552 #Συνοριακή Συνθήκη Δεξιά()
553 ax.scatter(x_b_right.detach().cpu().numpy(), t_b.detach().cpu().numpy(), ubr_m.
      detach().cpu().numpy(), color='b', label='ΣΕ δεξιά()', s=2);
554
555
556 ax.set_xlabel('$x$')
557 ax.set_ylabel('$t$')
558 ax.set_zlabel('$u(t,x)$')
559 ax.set_title(r'ΑΕ και ΣΕ της ΜΔΕ Περιοδικού Κύματος με NN:', fontsize=14);
560 ax.legend(loc='upper left', bbox_to_anchor=(1.05, 1))
561
562 plt.tight_layout()
563 plt.show()
564
565 # Μέγιστο πλάτος στο  $t_{max}$ 
566 U0_min = u0_m.min().item()
567 U0_max = u0_m.max().item()
568 print(f'Στο  $tT=$  έχουμε  $\min U_0 = \{U0\_min:.3f\}$ ,  $\max U_0 = \{U0\_max:.3f\}$ ')
569

```

```

570 from mpl_toolkits.mplot3d import Axes3D #βιβλιοθήκη για 3D γραφήματα
571
572 #Πλήθος διαστημάτων
573 N = 1000
574
575 tmin = tmin.cpu().item()
576 tmax = tmax.cpu().item()
577 xmin = xmin.cpu().item()
578 xmax = xmax.cpu().item()
579
580 #Δημιουργία σημείων
581 tspace = np.linspace(tmin, tmax, N + 1) #(N+1,1)
582 xspace = np.linspace(xmin, xmax, N + 1) #(N+1,1)
583
584 #Δημιουργία πλέγματος σημείων
585 T, X = np.meshgrid(tspace, xspace) #(N+1,N+1) , (N+1,N+1)
586
587 #κάθε γραμμή ένα σημείο του πλέγματος
588 Xgrid = np.vstack([T.flatten(),X.flatten()]).T #N((+1)^2,2)
589
590 #Υπολογίζει τις προβλέψεις u(t, x) του μοντέλου
591 upred = model(torch.tensor(Xgrid,dtype=dtype).to(device)) #N((+1)^2,1)
592
593 #Από N((+1)^2,1) σε NN(+1,+1)
594 U = upred.cpu().detach().numpy().reshape(N+1,N+1)
595
596 #Γράφημα πρόβλεψης u(t,x) από δύο οπτικές γωνίες
597 fig = plt.figure(figsize=(6,4))
598 #fig.suptitleΜοντελοποίηση(' ΜΔΕ Περιοδικού Κύματος με NN:\n Αποτελέσματα
    Προβλέψεων', fontsize=14)
599
600 ax1 = fig.add_subplot(111, projection='3d')
601 ax1.plot_surface(T, X, U, cmap='plasma');
602 ax1.view_init(35,35)
603 ax1.set_xlabel('$t$')
604 ax1.set_ylabel('$x$')
605 ax1.set_zlabel('$u(t,x)$')
606
607 ax1.invert_xaxis()
608
609
610 print(f'Για t=[0,T] έχουμε min U = {U.min():.3f}, max U = {U.max():.3f}')
611
612 # Τελικό χρονικό βήμα
613 U_final = U[:, -1] # τελευταία γραμμή
614
615 # Μέγιστο πλάτος στο t_max
616 U_min = np.min(U_final)
617 U_max = np.max(U_final)
618 print(f'Στο tT= έχουμε min U = {U_min:.3f}, max U = {U_max:.3f}')
619
620 from mpl_toolkits.mplot3d import Axes3D #βιβλιοθήκη για 3D γραφήματα
621
622 #Πλήθος διαστημάτων
623 N = 1000
624
625 #tmin = tmin.cpu().item()
626 #tmax = tmax.cpu().item()
627 #xmin = xmin.cpu().item()
628 #xmax = xmax.cpu().item()
629
630 #Δημιουργία σημείων

```

```

631 tspace = np.linspace(tmin, tmax, N + 1) #(N+1,1)
632 xspace = np.linspace(xmin, xmax, N + 1) #(N+1,1)
633
634 #Δημιουργία πλέγματος σημείων
635 T, X = np.meshgrid(tspace, xspace) #(N+1,N+1) , (N+1,N+1)
636
637 #κάθε γραμμή ένα σημείο του πλέγματος
638 Xgrid = np.vstack([T.flatten(),X.flatten()]).T #N((+1)^2,2)
639
640 #Υπολογίζει τις προβλέψεις u(t, x) του μοντέλου
641 upred = model(torch.tensor(Xgrid,dtype=dtype).to(device)) #N((+1)^2,1)
642
643 #Από N((+1)^2,1) σε NN(+1,+1)
644 U = upred.cpu().detach().numpy().reshape(N+1,N+1)
645
646 #Γράφημα πρόβλεψης u(t,x) από δύο οπτικές γωνίες
647 fig = plt.figure(figsize=(6,4))
648 #fig.suptitleΜοντελοποίηση(' ΜΔΕ Περιοδικού Κύματος με NN:\n Αποτελέσματα
        Προβλέψεων', fontsize=14)
649
650 ax1 = fig.add_subplot(121, projection='3d')
651 ax1.plot_surface(T, X, U, cmap='plasma');
652 ax1.view_init(35,35)
653 ax1.set_xlabel('$t$')
654 ax1.set_ylabel('$x$')
655 ax1.set_zlabel('$u(t,x)$')
656 ax1.set_title(r'$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}, \text{quad } x \in [-L, L], t \in [0,T]$' '\n'
        r'$u(x, 0) = A \sin(\frac{\pi}{L} x)$' '\n'
        r'$\frac{\partial u}{\partial t}(x, 0) = -A \frac{\pi}{L} c \cos(\frac{\pi}{L} x)$' '\n'
        r'$u(-L, t) = u(L, t)$' '\n'
        r'$u_x(-L, t) = u_x(L, t)$',
        fontsize=12);
662 ax1.invert_xaxis()
663
664
665 ax2 = fig.add_subplot(122, projection='3d')
666 ax2.plot_surface(T, X, U, cmap='hot');
667 ax2.view_init(35,20)
668 ax2.set_xlabel('$t$')
669 ax2.set_ylabel('$x$')
670 ax2.set_zlabel('$u(t,x)$')
671 ax2.set_title(r'$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}, \text{quad } x \in [-L, L], t \in [0,T]$' '\n'
        r'$u(x, 0) = A \sin(\frac{\pi}{L} x)$' '\n'
        r'$\frac{\partial u}{\partial t}(x, 0) = -A \frac{\pi}{L} c \cos(\frac{\pi}{L} x)$' '\n'
        r'$u(-L, t) = u(L, t)$' '\n'
        r'$u_x(-L, t) = u_x(L, t)$',
        fontsize=12);
677 ax2.invert_xaxis()
678
679 # Εμφάνιση του αποτελέσματος
680 plt.tight_layout(rect=[0, 0, 1, 0.88])
681 plt.show()
682
683 print(f'Για t=[0,T] έχουμε min U = {U.min():.3f}, max U = {U.max():.3f}')
684
685 # Τελικό χρονικό βήμα
686 U_final = U[:,-1] # τελευταία γραμμή
687

```

```

688 # Μέγιστο πλάτος στο  $t_{max}$ 
689 U_min = np.min(U_final)
690 U_max = np.max(U_final)
691 print(f'Στο  $tT=$  έχουμε min  $U = \{U_{min}:.3f\}$ , max  $U = \{U_{max}:.3f\}$ ')
692
693 import plotly.graph_objects as go
694
695 #tmin = tmin.cpu().item()
696 #tmax = tmax.cpu().item()
697 #xmin = xmin.cpu().item()
698 #xmax = xmax.cpu().item()
699
700 #Δημιουργία σημείων
701 tspace = np.linspace(tmin, tmax, N + 1) #(N+1,1)
702 xspace = np.linspace(xmin, xmax, N + 1) #(N+1,1)
703
704 #Δημιουργία πλέγματος σημείων
705 T, X = np.meshgrid(tspace, xspace) #(N+1,N+1) , (N+1,N+1)
706
707 #τίτλος
708 lines = (
709     r'<b>Μοντελοποίηση> ΜΑΕ Περιοδικού Κύματος με NN<b>',
710     r' $\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$ , \quad x \in [-L, L], t \in [0, T]',
711     r' $u(x, 0) = A \sin(\frac{\pi}{L} x)$ ',
712     r' $\frac{\partial u}{\partial t}(x, 0) = -A \frac{\pi}{L} c \cos(\frac{\pi}{L} x)$ ',
713     r' $u(-L, t) = u(L, t)$ ',
714     r' $u_x(-L, t) = u_x(L, t)$ '
715 )
716
717 #δημιουργία γραμμών τίτλου
718 annotations = []
719 x_pos = 0.05
720 start_y = 1.05
721 dy = -0.055
722
723 for i, line in enumerate(lines):
724     annotations.append(
725         dict(
726             text=line,
727             x=x_pos,
728             y=start_y + i * dy,
729             xref='paper',
730             yref='paper',
731             showarrow=False,
732             font=dict(size=13 if i > 0 else 16, family="Arial"),
733             align='center'
734         )
735     )
736
737 #αποκοπή των δεκαδικών ψηφίων από μιν μαξ τιμές για εμφάνιση
738 def truncation(x, decimals):
739     factor = 10 ** decimals
740     return torch.trunc(x * factor) / factor
741
742 U_tensor = torch.tensor(U)
743
744 min_U = truncation(torch.min(U_tensor), 1).item()
745 max_U = truncation(torch.max(U_tensor), 1).item()
746
747 #δημιουργία tickvals στη ράβδο

```

```

748 tickvals = [
749     min_U + 0.001,
750     truncation(torch.tensor((min_U + max_U)/2), 3).item(),
751     max_U - 0.001
752 ]
753
754
755
756 #δημιουργία 3D γραφήματος
757 fig = go.Figure(
758     data=[
759         go.Surface(
760             x=tspace,
761             y=xspace,
762             z=U,
763             colorscale='jet',
764             colorbar=dict(
765                 title=dict(text='u(x,t)', side='top', font=dict(size=12)),
766                 tickvals = tickvals,
767                 x=0.88,          # αριστεράδεξιά/
768                 y=0.48,          # πάνωκάτω/
769                 len=0.9,         # πόσο μακρύ"" κάθετα
770                 thickness=20,    # πόσο παχύ"" οριζόντια
771                 tickfont=dict(size=10)
772             )
773         )
774     ]
775 )
776
777 fig.update_layout(
778     margin=dict(l=20, r=20, t=50, b=20),
779     scene=dict(
780         xaxis=dict(title='t', autorange='reversed'), #αντιστροφή άξονα x
781         yaxis_title='x',
782         zaxis_title='u(t,x)'
783     ),
784     annotations=annotations
785 )
786
787 fig.show()
788
789 print(f'Για t=[0,T] έχουμε min U = {U.min():.3f}, max U = {U.max():.3f}')
790
791 # Τελικό χρονικό βήμα
792 U_final = U[:, -1] # τελευταία γραμμή
793
794 # Μέγιστο πλάτος στο t_max
795 U_min = np.min(U_final)
796 U_max = np.max(U_final)
797 print(f'Στο t=T έχουμε min U = {U_min:.3f}, max U = {U_max:.3f}')
798
799 import numpy as np
800 import matplotlib.pyplot as plt
801 from mpl_toolkits.mplot3d import Axes3D
802
803 #dtype = np.float32
804 c = wave_speed.item()A
805 = 1
806 #tmin = tmin.cpu().item()
807 #tmax = tmax.cpu().item()
808 #xmin = xmin.cpu().item()
809 #xmax = xmax.cpu().item()

```



```

810 k = k.cpu()
811
812 #σημεία
813 x = np.linspace(xmin, xmax, 200)
814 t = np.linspace(tmin, tmax, 200)
815 X, T = np.meshgrid(x, t)
816
817 print(f'x: ', x.device)
818 print(f't: ', t.device)
819 print(f'X: ', X.device)
820 print(f'T: ', T.device)
821 print(f'A: ', A.device)
822 print(f'k: ', k.device)
823 print(f'wave_speed:', wave_speed.device)
824
825 #Λύση κύματος
826 U_anal = A * torch.sin(k * (X - c * T))
827
828 #γράφημα
829 fig = plt.figure(figsize=(12,6))
830 ax = fig.add_subplot(111, projection='3d')
831 ax.plot_surface(X, T, U_anal, cmap='viridis')
832
833 ax.set_xlabel('x')
834 ax.set_ylabel('t')
835 ax.set_zlabel('u(x,t)')
836 ax.set_title('Λύση  $u(x,t)=\cos(k(x-ct))$ ')
837
838 plt.show()
839
840 print(f'Για  $t=[0,T]$  έχουμε  $\min U_{anal} = \{U_{anal.min():.3f}\}$ ,  $\max U_{anal} = \{U_{anal.max}():.3f\}$ ')
841
842 # Τελικό χρονικό βήμα
843 Uanal_final = U_anal[:, -1] # τελευταία γραμμή
844
845 # Μέγιστο πλάτος στο  $t_{max}$ 
846 Uanal_min = Uanal_final.min()
847 Uanal_max = Uanal_final.max()
848 print(f'Στο  $t=T$  έχουμε  $\min U_{anal} = \{U_{anal\_min:.3f}\}$ ,  $\max U_{anal} = \{U_{anal\_max:.3f}\}$ ')
849
850 """Διαφορές### προσεγγίσεων:
851
852 *1) Αναλυτική - Pinn*
853
854 *2) Αναλυτική - RK4*
855
856 *3) Pinn - RK4*
857 """
858
859 #1)Μέγιστη Απόλυτη Διαφορά του Μοντέλου με την Αναλυτική μέσα στον χρόνο
860
861 dtype = torch.float32
862
863 N=200
864
865 #σημεία
866 x = np.linspace(xmin, xmax, N+1)
867 t = np.linspace(tmin, tmax, N+1)
868 X, T = np.meshgrid(x, t)
869
870 print(f'x: ', x.device)

```

```

871 print(f't: ', t.device)
872 print(f'X: ', X.device)
873 print(f'T: ', T.device)
874 print(f'A: ', A.device)
875 print(f'k: ', k.device)
876 print(f'wave_speed: ', wave_speed.device)
877
878 #αναλυτική λύση κύματος
879 U_anal = A * torch.sin(k * (X - c * T))
880
881 #κάθε γραμμή ένα σημείο του πλέγματος
882 Xgrid = np.vstack([T.flatten(), X.flatten()]).T #N((+1)^2,2)
883
884 #Υπολογίζει τις προβλέψεις u(t, x) του μοντέλου
885 upred = model(torch.tensor(Xgrid, dtype=dtype).to(device)) #N((+1)^2,1)
886
887 #Από N((+1)^2,1) σε NN(+1,+1) και μετατροπή σε numpy
888 U_pred = upred.detach().cpu().numpy().reshape((N+1, N+1))
889 U_anal_np = U_anal.detach().cpu().numpy()
890
891 #διαφορές
892 abs_diff = np.abs(U_pred - U_anal_np) #απόλυτη τιμή διαφοράς (N+1,N+1)
893 #mean_diff = np.mean(abs_diff, axis=1) μέση# τιμή διαφοράς N(+1,)
894 max_diff = np.max(abs_diff, axis=1) #μέγιστη τιμή διαφοράς (N+1,)
895
896
897 #γραφήματα
898
899 #Πρώτο γράφημα: pcolormesh
900
901 #Τα μέγιστα ελάχιστα για την μπάρα
902 #vmin = np.min(abs_diff)
903 #vmax = np.max(abs_diff)
904 vmin = 0.0
905 vmax = 0.014
906
907 plt.figure(figsize=(6,4))
908 pcm = plt.pcolormesh(X, T, abs_diff, shading='auto', cmap='inferno', vmin=vmin,
909                      vmax=vmax)
910 plt.title('|U_anal - U_pred| μέσα στον χρόνο')
911 plt.xlabel('x')
912 plt.ylabel('t')
913 plt.colorbar(pcm, label='Τιμές Απόλυτης Διαφοράς')
914
915 #Δεύτερο γράφημα: γραμμικό
916 #plt.figure(figsize=(10,4))
917 #plt.plot(t, mean_diff, label='mean|U_anal - U_pred|', color='purple')
918 #plt.xlabel('t')
919 #plt.ylabel('Τιμές Μέσης Απόλυτης Διαφοράς')
920 #plt.title('Μέση Απόλυτη Διαφορά του Μοντέλου με την Αναλυτική μέσα στον χρόνο')
921 #plt.legend(loc='upper center').get_frame().set_edgecolor('black')
922 #plt.xticks(np.arange(0, tmax+0.1, 0.5))
923 #plt.grid(True)
924
925 #Τρίτο γράφημα: γραμμικό
926 plt.figure(figsize=(6,4))
927 plt.plot(t, max_diff, label='max|U_anal - U_pred|', color='purple')
928 plt.xlabel('t')
929 plt.ylabel('Τιμές Μέγιστης Απόλυτης Διαφοράς')
930 plt.title('Μέγιστη Απόλυτη Διαφορά του Μοντέλου με την Αναλυτική μέσα στον χρόνο')
931 plt.legend(loc='upper center').get_frame().set_edgecolor('black')
932 plt.xticks(np.arange(0, tmax+0.1, 0.5))

```

```

932 plt.grid(True)
933
934 plt.tight_layout() # για να μην κολλάνε οι τίτλοι/άξονες/
935 plt.show()
936
937 #Τελικό χρονικό βήμα
938 U_final = U_pred[:, -1] # τελευταία γραμμή
939 Uanal_final = U_anal_np[:, -1]
940
941 #Μέγιστο πλάτος στο  $t_{max}$ 
942 U_min = np.min(U_final)
943 U_max = np.max(U_final)
944 print(f'Στο  $tT$ = έχουμε min  $U_{pred} = \{U_{min}:.3f\}$ , max  $U_{pred} = \{U_{max}:.3f\}$ ')
945
946 Uanal_min = np.min(Uanal_final)
947 Uanal_max = np.max(Uanal_final)
948 print(f'Στο  $tT$ = έχουμε min  $U_{anal} = \{U_{anal\_min}:.3f\}$ , max  $U_{anal} = \{U_{anal\_max}:.3f\}$ '
949       )
950
951 #ανέβασμα αρχείου Matlab
952
953 from google.colab import files
954 import os
955 import pandas as pd
956 import torch
957
958 filename = "u_wave_p.csv"
959
960 print(os.listdir()) # εμφανίζει όλα τα αρχεία στον φάκελο
961
962 #ρωτάω για αντικατάσταση αρχείου
963 if os.path.exists(filename):
964     answer = input(f"Θέλεις να διαγράψω το αρχείο '{filename}' ; ναι/χι(/): ").
965     strip().lower()
966
967     if answer == "ναι":
968         os.remove(filename)
969         print(f"Το αρχείο '{filename}' διαγράφηκε.")
970     else:
971         print("Η διαγραφή ακυρώθηκε.")
972 else:
973     print(f"Το αρχείο '{filename}' δεν υπάρχει.")
974
975 if os.path.exists(filename):
976     print(f"Το αρχείο '{filename}' υπάρχει στον τρέχοντα φάκελο ({os.getcwd()})." )
977     df = pd.read_csv(filename, header=None)
978 else:
979     print(f"Το αρχείο '{filename}' ΔΕΝ υπάρχει στον τρέχοντα φάκελο ({os.getcwd()}
980           ).")
981     upload = files.upload() # θα ανοίξει παράθυρο για να ανεβάσω το CSV
982     upload_filename = list(upload.keys())[0] # το όνομα του πρώτου αρχείου
983     df = pd.read_csv(upload_filename, header=None)
984
985 # Μετατροπή σε tensor
986 U_RK4 = torch.tensor(df.values, dtype=torch.float)
987
988 print("Μέγεθος  $u$ :", U_RK4.shape)
989
990 #2) Διαφορές μεταξύ Μοντέλου και RK4 μέσα στον χρόνο
991 import numpy as np

```

```

991 #παίρνω τις διαστάσεις από το Matlab
992 Nx_rk4, Nt_rk4 = U_RK4.shape
993
994 #φτιάχνω ίδιες διαστάσεις για το μοντέλο για να επιτευχθεί η αφαίρεση
995 Nx = Nx_rk4
996 Nt = Nt_rk4
997
998 #σημεία
999 x = np.linspace(xmin, xmax, Nx)
1000 t = np.linspace(tmin, tmax, Nt)
1001 X, T = np.meshgrid(x, t)
1002
1003 #αναλυτική λύση κύματος
1004 U_anal = A * np.sin(k * (X - c * T))
1005
1006 #μετατροπή σε numpy
1007 U_anal_np = U_anal.detach().cpu().numpy()
1008 U_RK4_np = U_RK4.detach().cpu().numpy()
1009
1010 print(U_anal_np.shape)
1011 print(U_RK4_np.shape)
1012
1013 abs_diff = np.abs(U_anal_np - U_RK4_np.T) #απόλυτη διαφορά
1014 #mean_diff = np.mean(abs_diff, axis=1) μέση# τιμή διαφοράς
1015 max_diff = np.max(abs_diff, axis=1) #μέγιστη τιμή διαφοράς
1016
1017
1018 #γράφηματα
1019
1020 #Πρώτο γράφημα: pcolormesh
1021 plt.figure(figsize=(6,4))
1022 pcm = plt.pcolormesh(X, T, abs_diff, shading='auto', cmap='inferno')
1023 plt.title('|U_anal - U_RK4| μέσα στον χρόνο')
1024 plt.xlabel('x')
1025 plt.ylabel('t')
1026 plt.colorbar(pcm, label='Τιμές Απόλυτης Διαφοράς')
1027
1028 #Δεύτερο γράφημα: γραμμικό
1029 #plt.figure(figsize=(10,4))
1030 #plt.plot(t, mean_diff, label='mean|U_anal - U_RK4|', color='purple')
1031 #plt.xlabel('t')
1032 #plt.ylabel('Τιμές Μέσης Απόλυτης Διαφοράς')
1033 #plt.title('Μέση Απόλυτη Διαφορά της Αναλυτικής με την RK4 μέσα στον χρόνο')
1034 #plt.legend(loc='upper center').get_frame().set_edgecolor('black')
1035 #plt.xticks(np.arange(0, tmax+0.1, 0.5))
1036 #plt.grid(True)
1037
1038 #Τρίτο γράφημα: γραμμικό
1039 plt.figure(figsize=(6,4))
1040 plt.plot(t, max_diff, label='max|U_anal - U_RK4|', color='cyan')
1041 plt.xlabel('t')
1042 plt.ylabel('Τιμές Μέγιστης Απόλυτης Διαφοράς')
1043 plt.title('max|U_anal - U_RK4| μέσα στον χρόνο')
1044 plt.legend(loc='upper center').get_frame().set_edgecolor('black')
1045 plt.xticks(np.arange(0, tmax+0.1, 0.5))
1046 plt.grid(True)
1047
1048 plt.tight_layout() # για να μην κολλάνε οι τίτλοι/άξονες/
1049 plt.show()
1050
1051 #Τελικό χρονικό βήμα
1052 Uanal_final = U_anal_np[:, -1] # τελευταία γραμμή

```

```

1053 URK4_final = U_RK4_np[:, -1]
1054
1055 #Μέγιστο πλάτος στο  $t_{max}$ 
1056 Uanal_min = np.min(Uanal_final)
1057 Uanal_max = np.max(Uanal_final)
1058 print(f'Στο  $tT$ = έχουμε min U = {Uanal_min:.3f}, max U = {Uanal_max:.3f}')
1059
1060 URK4_min = np.min(URK4_final)
1061 URK4_max = np.max(URK4_final)
1062 print(f'Στο  $tT$ = έχουμε min U_RK4 = {URK4_min:.3f}, max U_RK4 = {URK4_max:.3f}')
1063
1064 #3)Διαφορές του Μοντέλου με την RK4 μέσα στον χρόνο
1065
1066 #dtype = torch.float32
1067
1068 #παίρνω τις διαστάσεις από το Matlab
1069 Nx_rk4, Nt_rk4 = U_RK4.shape
1070
1071 #φτιάχνω ίδιες διαστάσεις για το μοντέλο για να επιτευχθεί η αφαίρεση
1072 Nx = Nx_rk4
1073 Nt = Nt_rk4
1074
1075 #σημεία
1076 x = np.linspace(xmin, xmax, Nx)
1077 t = np.linspace(tmin, tmax, Nt)
1078 X, T = np.meshgrid(x, t)
1079
1080 #κάθε γραμμή ένα σημείο του πλέγματος
1081 Xgrid = np.vstack([T.flatten(), X.flatten()]).T #N((+1) 2,2)
1082
1083 #Υπολογίζει τις προβλέψεις  $u(t, x)$  του μοντέλου
1084 upred = model(torch.tensor(Xgrid, dtype=dtype).to(device)) #N((+1) 2,1)
1085
1086 #Από N((+1) 2,1) σε NN(+1,+1) και μετατροπή σε numpy
1087 U_pred = upred.detach().numpy().reshape(Nt, Nx)
1088 U_RK4_np = U_RK4.detach().cpu().numpy()
1089
1090 abs_diff = np.abs(U_pred - U_RK4_np.T) #απόλυτη διαφορά NN(+1,+1)
1091 #mean_diff = np.mean(abs_diff, axis=1) μέση# τιμή διαφοράς N(+1,)
1092 max_diff = np.max(abs_diff, axis=1) #μέγιστη τιμή διαφοράς N(+1,)
1093
1094
1095 #γραφήματα
1096
1097 #Πρώτο γράφημα: pcolormesh
1098
1099 #Για μέγιστα ελάχιστα για την μπάρα
1100 #vmin = np.min(abs_diff)
1101 #vmax = np.max(abs_diff)
1102 vmin = 0.0
1103 vmax = 0.014
1104
1105 plt.figure(figsize=(6,4))
1106 pcm = plt.pcolormesh(X, T, abs_diff, shading='auto', cmap='inferno', vmin=vmin,
1107                      vmax=vmax)
1108 plt.title('|U_RK4 - U_pred| μέσα στον χρόνο')
1109 plt.xlabel('x')
1110 plt.ylabel('t')
1111 plt.colorbar(pcm, label='Τιμές Απόλυτης Διαφοράς')
1112
1113 #Δεύτερο γράφημα: γραμμικό
1114 #plt.figure(figsize=(10,4))

```

```

1114 #plt.plot(t, mean_diff, label='mean/U_RK4 - U_pred/', color='purple')
1115 #plt.xlabel('t')
1116 #plt.ylabel('Τιμές Απόλυτης Διαφοράς')
1117 #plt.title('Μέση Απόλυτη Διαφορά του Μοντέλου με την RK4 μέσα στον χρόνο')
1118 #plt.legend(loc='upper center').get_frame().set_edgecolor('black')
1119 #plt.xticks(np.arange(0, tmax+0.1, 0.5))
1120 #plt.grid(True)
1121
1122 #Τρίτο γράφημα: γραμμικό
1123 plt.figure(figsize=(6,4))
1124 plt.plot(t, max_diff, label='max|U_RK4 - U_pred|', color='cyan')
1125 plt.xlabel('t')
1126 plt.ylabel('Τιμές Μέγιστης Απόλυτης Διαφοράς')
1127 plt.title('max|U_RK4 - U_pred| μέσα στον χρόνο')
1128 plt.legend(loc='upper center').get_frame().set_edgecolor('black')
1129 plt.xticks(np.arange(0, tmax+0.1, 0.5))
1130 plt.grid(True)
1131
1132 plt.tight_layout() # για να μην κολλάνε οι τίτλοι/άξονες/
1133 plt.show()
1134
1135 #Τελικό χρονικό βήμα
1136 U_final = U_pred[:, -1] # τελευταία γραμμή
1137 URK4_final = U_RK4_np[:, -1]
1138
1139 #Μέγιστο πλάτος στο t_max
1140 U_min = np.min(U_final)
1141 U_max = np.max(U_final)
1142 print(f'Στο tT= έχουμε min U_pred = {U_min:.3f}, max U_pred = {U_max:.3f}')
1143
1144 URK4_min = np.min(URK4_final)
1145 URK4_max = np.max(URK4_final)
1146 print(f'Στο tT= έχουμε min U_RK4 = {URK4_min:.3f}, max U_RK4 = {URK4_max:.3f}')

```

Listing 1: Εφαρμογή PINN στην Εξίσωση Κύματος με Περιοδικές Συνοριακές Συνθήκες

Βιβλιογραφία

- [1] B. Κάλφας and Γ. Ζωγραφίδης. Αρχαίοι Έλληνες Φιλόσοφοι, 2012.
- [2] STEAMBuilders (Erasmus+). Archimedes' principle: Pedagogical sequence. Technical report, STEAMBuilders Project, 2022.
- [3] Δήμητρα Μήττα. Μορφές και Θέματα της Αρχαίας Ελληνικής Μυθολογίας. Online Άρθρο, 2012. Ινστιτούτο Νεοελληνικών Σπουδών.
- [4] Στέργιος Χατζηκυριακίδης. *Τεχνητή νοημοσύνη και μεγάλα γλωσσικά μοντέλα: Ιστορία, χρήσεις, ανησυχίες*. Δίαυλος, Αθήνα, 2024.
- [5] Δημήτριος Καλλιεγρόπουλος. Τα Αυτόματα του Έρωνα. Online PDF, 2023. <https://edabyt.gr/1o-synedrio-praktika/>.
- [6] Isaac Newton Institute for Mathematical Sciences. Isaac newton's life, 1998.
- [7] Scott McLaughlan. Why rene descartes believed that machines will never be able to genuinely “think”. Online Άρθρο, 2026.
- [8] Oscar Schwartz. In the 17th century, leibniz dreamed of a machine that could calculate ideas. Online Άρθρο, 2019.
- [9] Roger Bishop Jones. Leibniz and the automation of reason. Online Άρθρο, 2006.
- [10] Luigi Federico Menabrea. Sketch of the analytical engine invented by charles babbage. In *Scientific Memoirs*. Longman, Rees, Orme, Brown, and Green, 1843.
- [11] History of Computing Project. Ada lovelace – complete biography, history and inventions. Online Άρθρο, 2026.
- [12] J. Fuegi and J. Francis. Lovelace & babbage and the creation of the 1843 'notes'. *IEEE Annals of the History of Computing*, 25(4):16–26, 2003.
- [13] Science Museum Group. Charles babbage's difference engines and the science museum. Online Άρθρο, 2023.
- [14] Britannica Editors. Ada lovelace: The first computer programmer. Online Άρθρο, 2025.
- [15] History of Computing Project. Alan turing — complete biography, history and inventions. Online Άρθρο, 2025.

- [16] Stanford Encyclopedia of Philosophy. Alan turing. Ηλεκτρονική Εγκυκλοπαίδεια, 2020.
- [17] Encyclopaedia Britannica, Inc. Alan turing — computer designer. Ηλεκτρονική Εγκυκλοπαίδεια, 2026.
- [18] Alan M. Turing. Intelligent machinery. Research report, National Physical Laboratory, London, 1948.
- [19] Alan M. Turing. Computing machinery and intelligence. Διαδικτυακό PDF, μάθημα UMBC, 2026.
- [20] W. S. McCulloch and W. Pitts. A logical calculus of ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.
- [21] Theodore H. Abraham. (re) fashioning the brain: The connective link between mcculloch and pitts. *Isis*, 93(2):221–241, 2002.
- [22] B. Jack Copeland and Diane Proudfoot. Alan turing, father of the modern computer. *The Rutherford Journal*, 4, 2011–2012.
- [23] Machine Learning Times. Machine, learning, 1951. Online Άρθρο, 2025.
- [24] Trustees of Dartmouth College. Artificial intelligence coined at dartmouth 1956. Online Άρθρο, 2026.
- [25] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [26] David H. Hubel and Torsten N. Wiesel. Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex. *The Journal of Physiology*, 160:106–154, 1962.
- [27] Kunihiro Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, 1980.
- [28] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [29] Geoffrey E. Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. *Neural Computation*, 18(1527–1554), 2006.
- [30] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [31] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [32] GeeksforGeeks. Keras vs tensorflow vs pytorch. Online Άρθρο, 2025.

-
- [33] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed deep learning (part ii): Data-driven discovery of nonlinear partial differential equations. *arXiv preprint*, 2017.
 - [34] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations. *arXiv preprint*, 1711.10561, 2017.
 - [35] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
 - [36] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.
 - [37] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.
 - [38] Αθανάσιος Χασιαχός and Παναγιώτης Τσίχας. 8-artificial neural networks (ann). Σημειώσεις μαθήματος Μέθοδοι Βελτιστοποίησης, Πανεπιστήμιο Πατρών, 2019. Πρόσβαση μέσω e-class.
 - [39] Adrian Rosebrock. Implementing the perceptron neural network with python. Online Άρθρο, 2021.
 - [40] Koushik Ahmed Kushal. Mnist(hand written digit) classification using neural network(step by step) from scratch. Online Άρθρο, 2023.
 - [41] David Alber. Viewing mnist images. Online Άρθρο, 2023.
 - [42] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
 - [43] Patrick Kidger and Terry J. Lyons. Universal approximation with deep narrow networks. In *Proceedings of the 33rd Conference on Learning Theory (COLT)*, Proceedings of Machine Learning Research. PMLR, 2020.
 - [44] Shengze Cai, Zhiping Mao, Zhicheng Wang, Minglang Yin, and George Em Karniadakis. Physics-informed neural networks (pinns) for fluid mechanics: A review. *Acta Mechanica Sinica*, 37(12):1727–1738, 2022.
 - [45] Jason J. Bramburger. *Data-Driven Methods for Dynamic Systems: A New Perspective on Old Problems*. Other Titles in Applied Mathematics. Society for Industrial and Applied Mathematics, 2023.
 - [46] Tamara G. Grossmann, Urszula Julia Komorowska, Jonas Latz, and Carola-Bibiane Schönlieb. Can physics-informed neural networks beat the finite element method? *IMA Journal of Applied Mathematics*, 89(1):143–174, 2024.
 - [47] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.

- [48] Chuqi Chen, Yahong Yang, Yang Xiang, and Wenrui Hao. Automatic differentiation is essential in training neural networks for solving differential equations. *CoRR*, abs/2405.14099, 2024. <https://arxiv.org/abs/2405.14099>.
- [49] Diederik P. Kingma and Jimmy Ba. Adam a method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [50] Dania Sana. Approximating the wave equation via physics-informed neural networks: Various forward and inverse problems. Internship report, Friedrich-Alexander-Universität Erlangen–Nürnberg, Research Center for Mathematics of Data (FAU MoD) and Chair of Dynamics, Numerics and Control (DCN), 2022. Supervised by Dr. Yue Wang.
- [51] Shiqi Wang, Yuze Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021.
- [52] Ameya D. Jagtap, Kenji Kawaguchi, and George Em Karniadakis. Extended physics-informed neural networks (xpinns) a generalized space–time domain decomposition based deep learning framework for nonlinear partial differential equations. *Communications in Computational Physics*, 28(5):2002–2041, 2020.
- [53] Ehsan Kharazmi, Zhongqiang Zhang, and George Em Karniadakis. Variational physics-informed neural networks for solving partial differential equations. *Computer Methods in Applied Mechanics and Engineering*, 374:113626, 2021.
- [54] Levi McClenney and Ulisses Braga-Neto. Self-adaptive physics-informed neural networks using a soft attention mechanism, 2020. arXiv preprint.
- [55] Aditi S. Krishnapriyan, Ali Gholami, Shandian Zhe, Robert M. Kirby, and Michael W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 26548–26560, 2021.
- [56] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.
- [57] Gianluca Fabiani, Ioannis G. Kevrekidis, Constantinos Siettos, and Athanasios N. Yannacopoulos. Randonets: Shallow networks with random projections for learning linear and nonlinear operators. *Journal of Computational Physics*, 520:113433, 2025.
- [58] Brian Caulfield. What’s the difference between a cpu and a gpu? Online *Αρθρο*, 2009.
- [59] TechBloat. Pytorch cpu vs gpu: Understanding different results. Online *Αρθρο*, 2025.

-
- [60] The University of Queensland, Applied Mathematical Analysis. Wave equation d’alembert’s formula. Online Άρθρο, 0. Περιέχει και την εργασία του d’Alembert.
 - [61] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Περιοδικές Συνοριακές Συνθήκες. [Source code \(GitHub\)](#), 2026.
 - [62] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος Περιοδικές Αρχικές Συνθήκες (matlab). [Source code \(GitHub\)](#), 2026.
 - [63] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες (Περιοδικές Σ.Σ.). [Source code \(GitHub\)](#), 2026.
 - [64] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Συνοριακές Συνθήκες dirichlet. [Source code \(GitHub\)](#), 2026.
 - [65] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Συνοριακές Συνθήκες dirichlet (matlab). [Source code \(GitHub\)](#), 2026.
 - [66] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες (dirichlet Σ.Σ.). [Source code \(GitHub\)](#), 2026.
 - [67] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες dirichlet. [Source code \(GitHub\)](#), 2026.
 - [68] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες dirichlet. [Source code \(GitHub\)](#), 2026.
 - [69] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Συνοριακές Συνθήκες neumann. [Source code \(GitHub\)](#), 2026.
 - [70] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Συνοριακές Συνθήκες neumann (matlab). [Source code \(GitHub\)](#), 2026.
 - [71] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες (neumann Σ.Σ.). [Source code \(GitHub\)](#), 2026.
 - [72] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες neumann. [Source code \(GitHub\)](#), 2026.
 - [73] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες και Κατά Τμήματα Ορισμένες Συνοριακές Συνθήκες neumann. [Source code \(GitHub\)](#), 2026.
 - [74] D. J. Korteweg and G. de Vries. On the change of form of long waves advancing in a rectangular canal, and on a new type of long stationary waves. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 39(240):422–443, 1895.
 - [75] Heriot-Watt University. John scott russell and the solitary wave. Online Άρθρο, 2026.
 - [76] Σπυρίδων Σακελλαρόπουλος. Εξίσωση kdv με Περιοδικές Αρχικές Συνθήκες. [Source code \(GitHub\)](#), 2026.

- [77] Σπυρίδων Σακελλαρόπουλος. Εξίσωση kdv με Περιοδικές Αρχικές Συνθήκες (matlab). [Source code \(GitHub\)](#), 2026.
- [78] Σπυρίδων Σακελλαρόπουλος. Εξίσωση Κύματος με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες. [Source code \(GitHub\)](#), 2026.
- [79] E. Schrödinger. Quantisierung als eigenwertproblem. *Annalen der Physik*, 384(4):361–376, 1926.
- [80] Vladimir E. Zakharov. Stability of periodic waves of finite amplitude on the surface of a deep fluid. *Journal of Applied Mechanics and Technical Physics*, 9(2):190–194, 1968.
- [81] V. E. Zakharov and A. B. Shabat. Exact theory of two-dimensional self-focusing and one-dimensional self-modulation of waves in nonlinear media. *Journal of Experimental and Theoretical Physics*, 34:62–69, 1970.
- [82] A. Hasegawa and F. Tappert. Transmission of stationary nonlinear optical pulses in dispersive dielectric fibers. i. anomalous dispersion. *Applied Physics Letters*, 23(3):142–144, 1973.
- [83] T. Taniuti and N. Yajima. Perturbation method for a nonlinear wave modulation. i. *Journal of Mathematical Physics*, 10(8):1369–1372, 1969.
- [84] Σπυρίδων Σακελλαρόπουλος. Εξίσωση nls με Περιοδικές Αρχικές Συνθήκες. [Source code \(GitHub\)](#), 2026.
- [85] Σπυρίδων Σακελλαρόπουλος. Εξίσωση nls με Περιοδικές Συνοριακές Συνθήκες (matlab). [Source code \(GitHub\)](#), 2026.
- [86] Σπυρίδων Σακελλαρόπουλος. Εξίσωση nls με Κατά Τμήματα Ορισμένες Αρχικές Συνθήκες (Περιοδικές Σ.Σ.). [Source code \(GitHub\)](#), 2026.